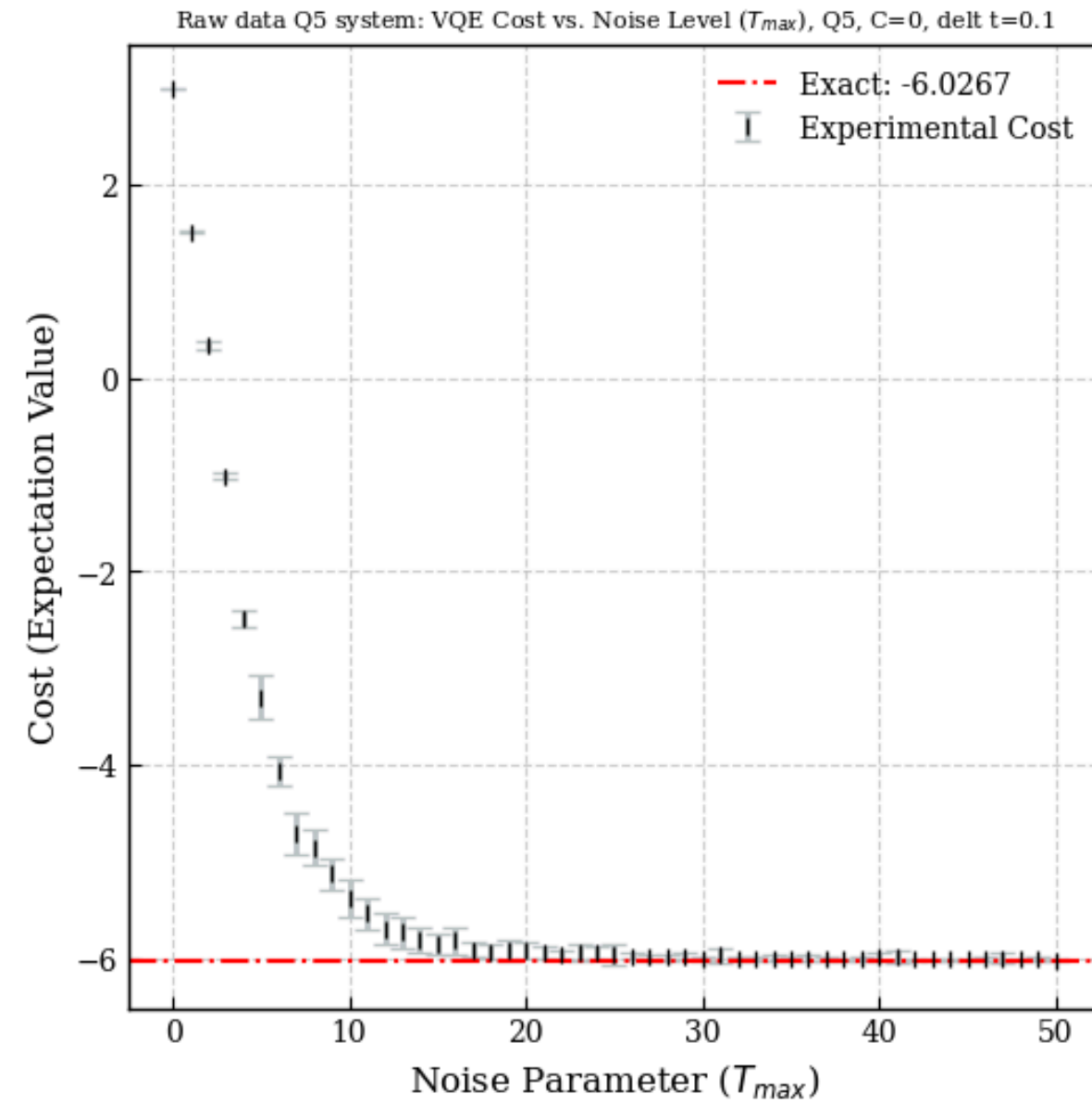


Part. 1

ZNE with LSF



Plotting simulation raw data C=0

n from noise-free $C = 0$ in T_{max} range $[0, 30]$

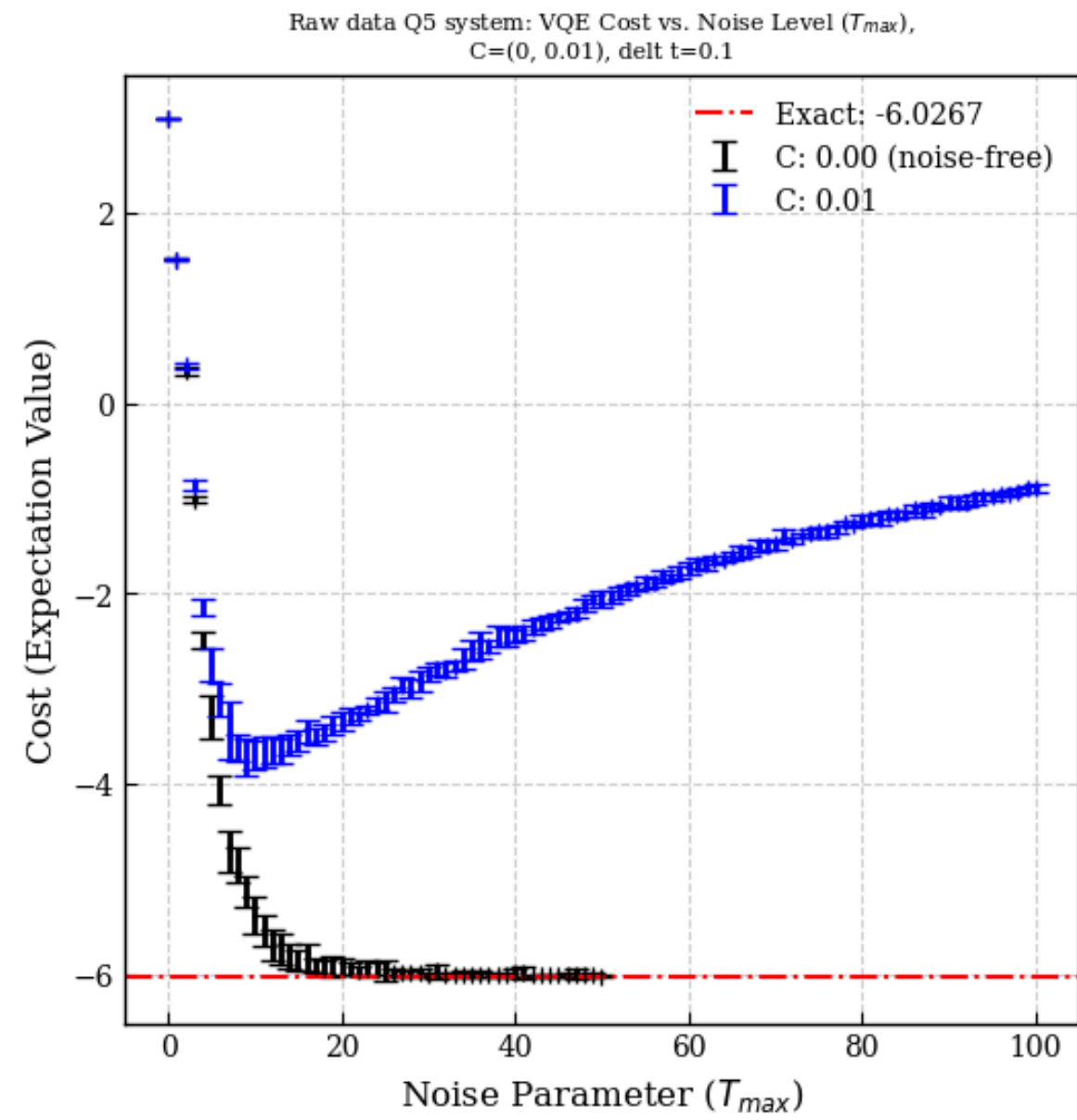
```
> # Noiseless guesses
ZNE_GUESS_NOISELESS = -5.0
A0_GUESS_NOISELESS = 10.0
T0_GUESS_NOISELESS = 8.0
```

[9] ✓ 0.0s Python

```
[10] ✓ 0.0s
... T_max range [0, 30]: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30]
```

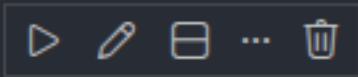
n	ZNE_init	A0_init	T0_init	ZNE_fit	A0_fit	T0_fit	LS
1	-5.000	10.000	8.00	-7.8104	10.8439	5.07	33.802
2	-5.000	10.000	8.00	-6.0875	9.0206	3.04	25.351
3	-5.000	10.000	8.00	-5.9365	8.7690	2.98	88.591
4	-5.000	10.000	8.00	-5.9073	8.6807	3.03	138.008
5	-5.000	10.000	8.00	-5.8952	8.6358	3.07	172.480
6	-5.000	10.000	8.00	-5.8876	8.6075	3.09	197.901
7	-5.000	10.000	8.00	-5.8811	8.5875	3.10	217.025
8	-5.000	10.000	8.00	-5.8746	8.5723	3.10	231.458
9	-5.000	10.000	8.00	-5.8684	8.5604	3.10	242.346
10	-5.000	10.000	8.00	-5.8627	8.5511	3.09	250.535

we decided that $n=2$



C=0.01 noisy simulation plot

γ from large $T_{max} = [50, 100]$



$$f_{\text{noisy}}(t) = B \cdot e^{-\gamma t}, \quad t \in [50, 100]$$

Iteration	Total nfev	Cost	Cost reduction	Step norm	Optimality
0	1	3.0635e+04			2.28e+05
1	2	2.7113e+04	3.52e+03	2.93e+00	2.82e-02
2	3	2.7113e+04	1.68e-04	7.30e-01	2.81e-04
3	4	2.7113e+04	5.37e-03	1.13e-01	7.51e-02
4	5	2.6904e+04	2.09e+02	1.93e-01	9.58e+02
5	6	1.6465e+03	2.53e+04	6.91e-01	5.04e+03
6	7	7.6364e+02	8.83e+02	4.07e-01	6.44e+04
7	9	5.1415e+02	2.49e+02	3.67e-01	9.97e+02
8	10	2.3252e+02	2.82e+02	7.32e-01	1.43e+03
9	11	6.7217e+01	1.65e+02	7.38e-01	9.28e+02
10	12	1.2824e+01	5.44e+01	6.52e-01	6.55e+02
11	13	3.6358e+00	9.19e+00	3.02e-01	1.06e+02
12	14	3.3196e+00	3.16e-01	7.29e-02	6.21e+00
13	15	3.3190e+00	6.33e-04	3.05e-03	5.93e-03
14	16	3.3190e+00	8.97e-09	1.44e-05	1.28e-03
15	17	3.3190e+00	5.42e-14	4.38e-08	4.20e-06
`ftol` termination condition is satisfied.					
Function evaluations 17, initial cost 3.0635e+04, final cost 3.3190e+00, first-order optimality 4.20e-06.					
Large-T Exponential Fit: $f(t) = B \cdot \exp(-\gamma t)$ with $T = [$ 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67					
68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85					
86 87 88 89 90 91 92 93 94 95 96 97 98 99 100]					
param	initial		fitted		

gamma	5.0000e-02		1.6758e-02		
B	2.9330e+00		-4.7588e+00		

(ZNE, A, T_0) from small $T_{max} = [0, 5]$

$$f_{\text{noisy}}(t) = ZNE + \frac{A}{1 + \left(\frac{t}{T_0}\right)^n} \text{ where } t \in [0, 5]$$

Iteration	Total nfev	Cost	Cost reduction	Step norm	Optimality
0	1	4.3506e+14			4.35e+15
1	2	6.0216e+12	4.29e+14	3.74e+00	4.52e+14
2	3	1.8372e+09	6.02e+12	1.82e+00	7.75e+12
3	4	1.2547e+03	1.84e+09	2.86e-01	2.45e+09
4	5	1.0653e+03	1.89e+02	2.72e-02	1.29e+03
5	6	2.9221e+02	7.73e+02	3.76e+00	4.47e+03
6	7	1.7771e+02	1.15e+02	6.31e-01	3.10e+02
7	8	1.4298e+02	3.47e+01	6.94e-01	3.30e+02
8	9	1.4163e+02	1.35e+00	1.22e-01	1.68e+01
9	10	1.4161e+02	2.15e-02	1.73e-02	1.53e+00
10	11	1.4161e+02	1.31e-04	1.33e-03	1.79e+01
11	12	1.4161e+02	7.56e-07	1.01e-04	8.47e+00
12	13	1.4161e+02	4.34e-09	7.69e-06	9.17e+00
13	14	1.4161e+02	2.43e-11	5.65e-07	9.23e+00

`ftol` termination condition is satisfied.

Function evaluations 14, initial cost 4.3506e+14, final cost 1.4161e+02, first-order optimality 9.23e+00.

Small-T Hill Fit: $f(t) = ZNE + A \cdot \frac{1}{1 + (t/T_0)^n}$ with $n=2$, $T = [0 \ 1 \ 2 \ 3 \ 4 \ 5]$

param	initial	fitted
ZNE	-5.0000	-2.0178
A	10.0000	5.0178
T0	8.0000	1.5548

⏮ ⏪ ⏩ ⏭ 🔍 🔄

All fitted parameters:

GAMMA_FROM_LARGE_T = 0.016757638541062304

B_FROM_LARGE_T = -4.758846539294926

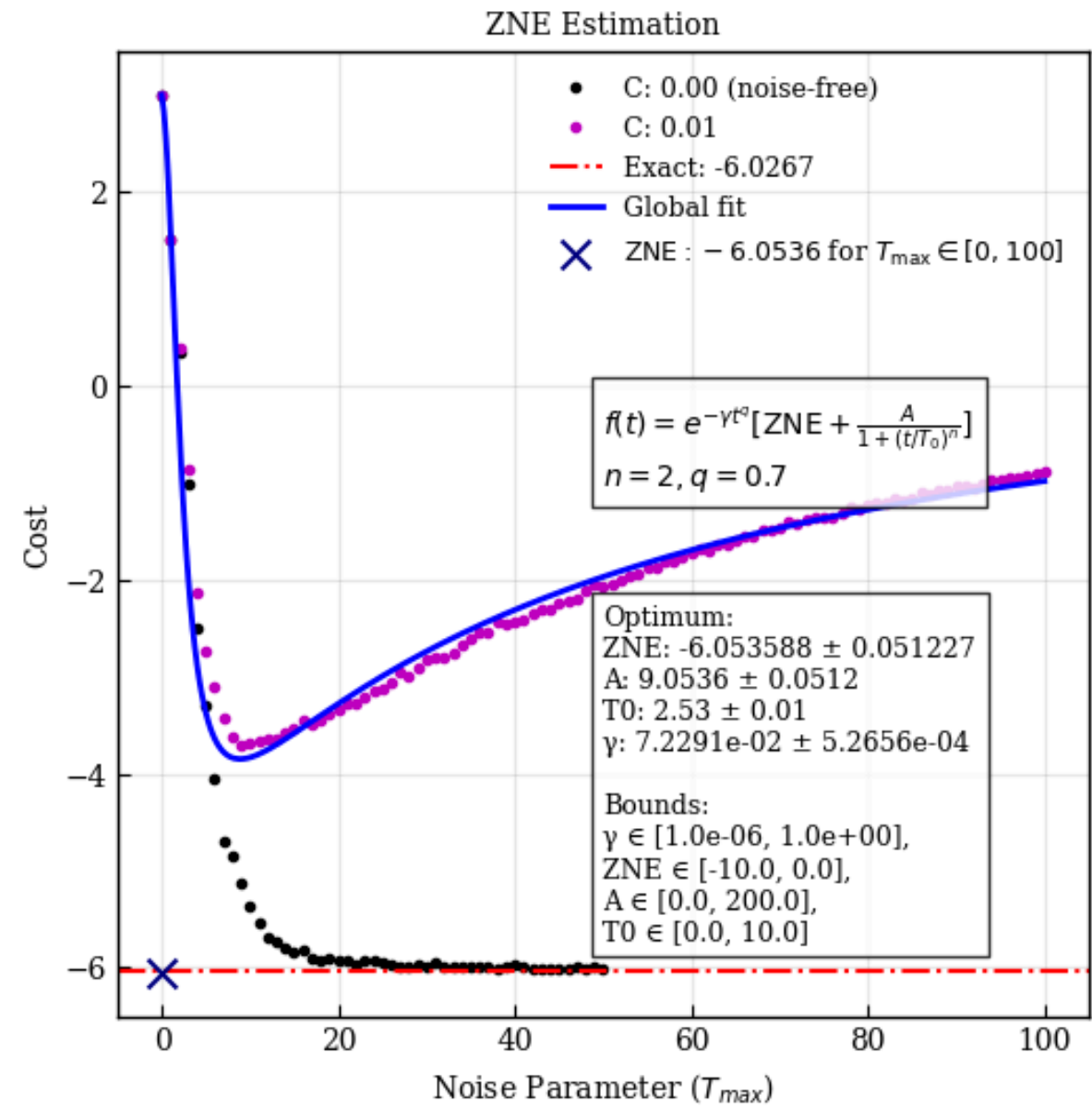
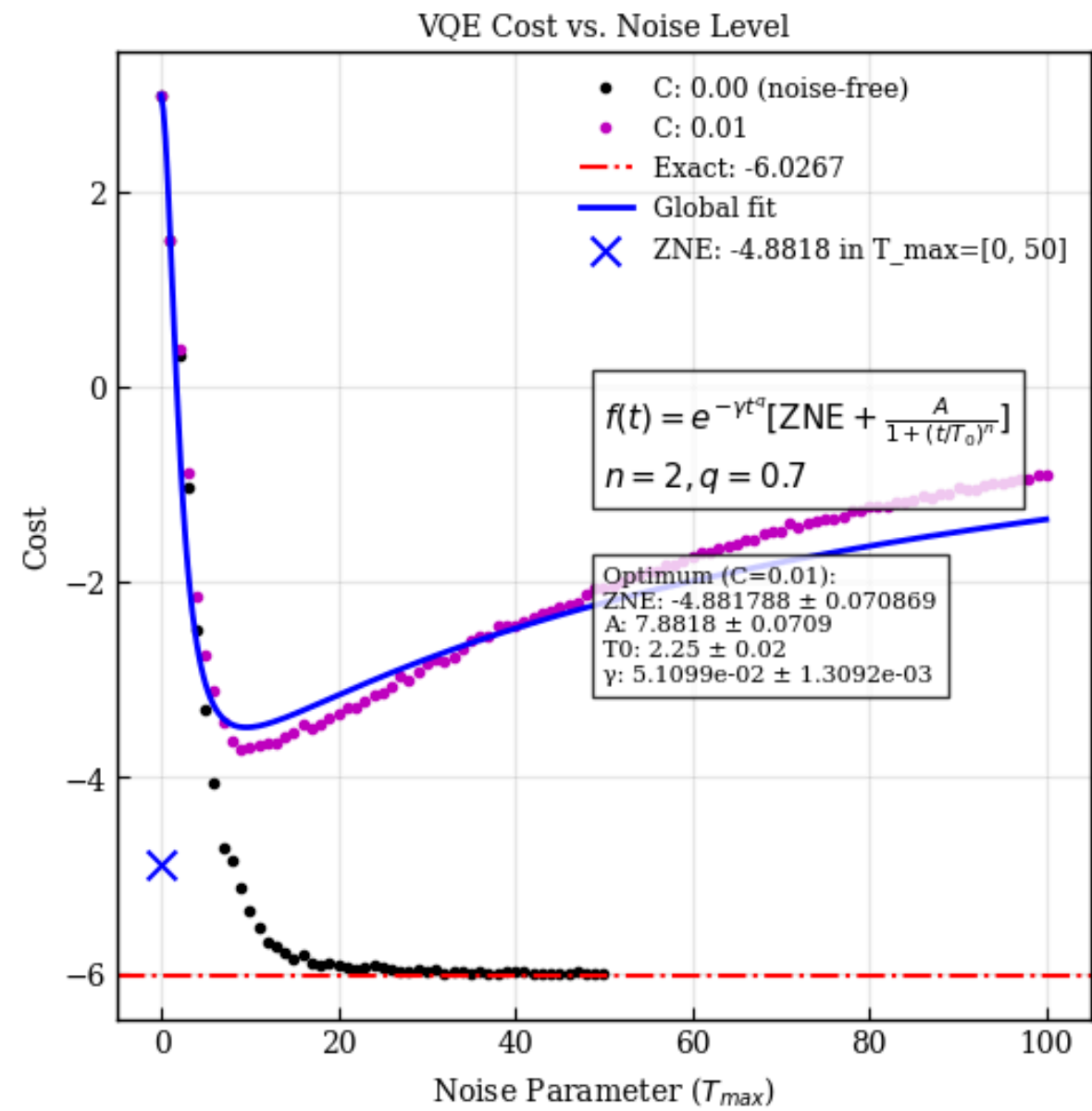
ZNE_FROM_NOISY_SMALL_T = -2.0177814739005315

A_FROM_NOISY_SMALL_T = 5.017781548473394

T0_FROM_NOISY_SMALL_T = 1.5547854398013081

End of fitted parameters.

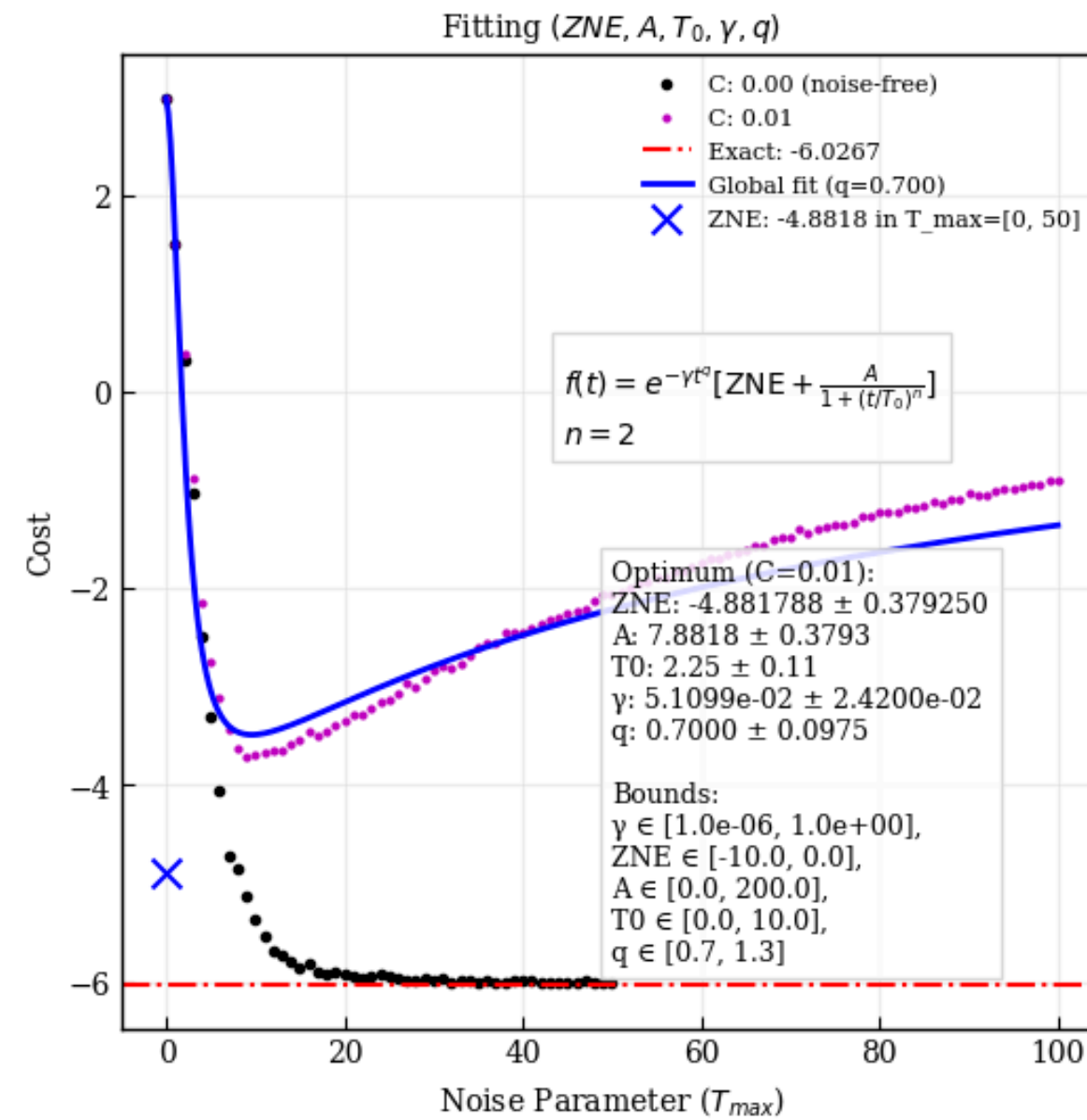
$$f_{\text{noisy}}(t) = e^{-\gamma t^q} \left[\text{ZNE} + \frac{A}{1 + \left(\frac{t}{T_0} \right)^n} \right], \quad q = 0.7 \text{ for } C = 0.01$$



Optimizing q test

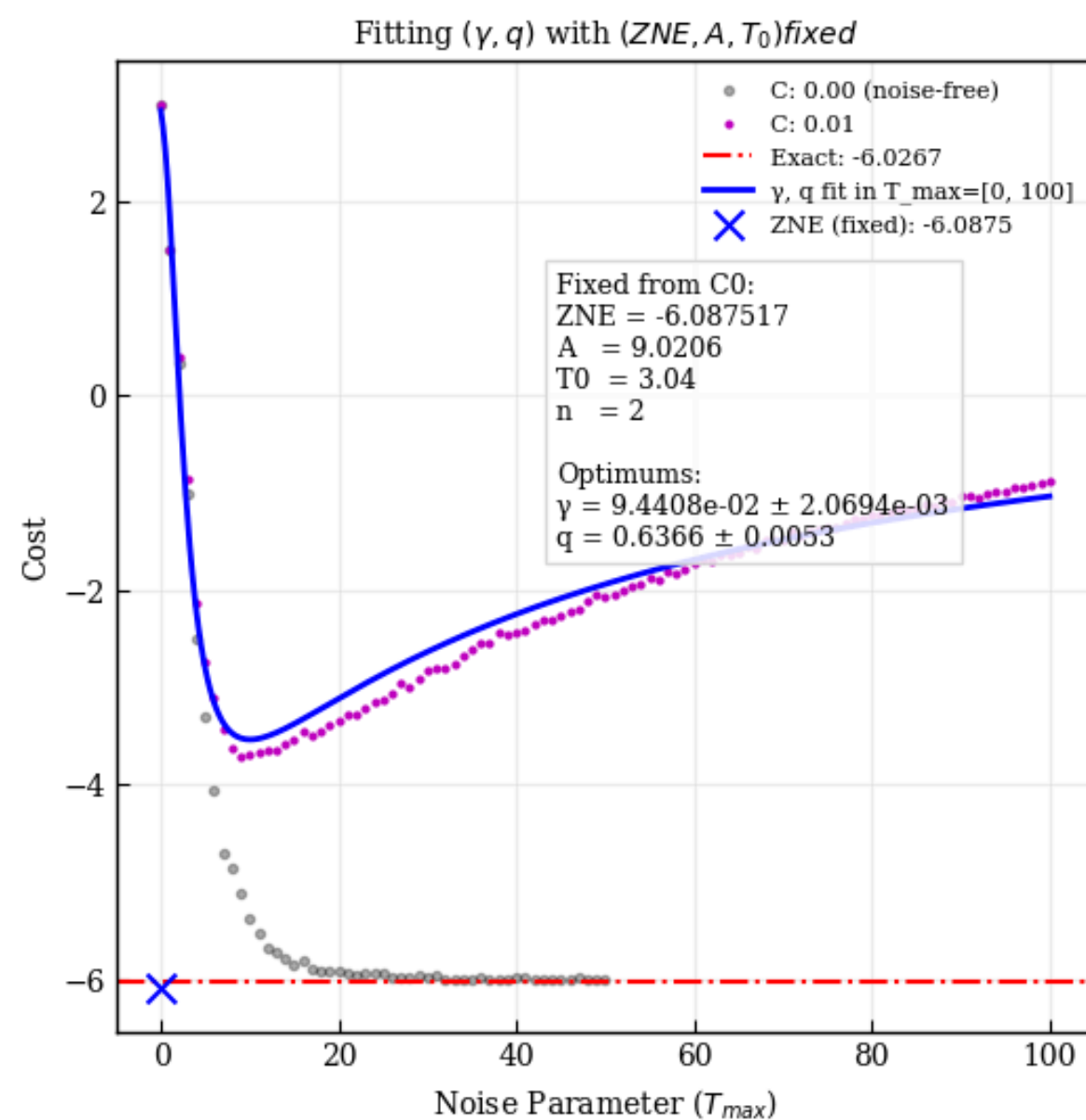
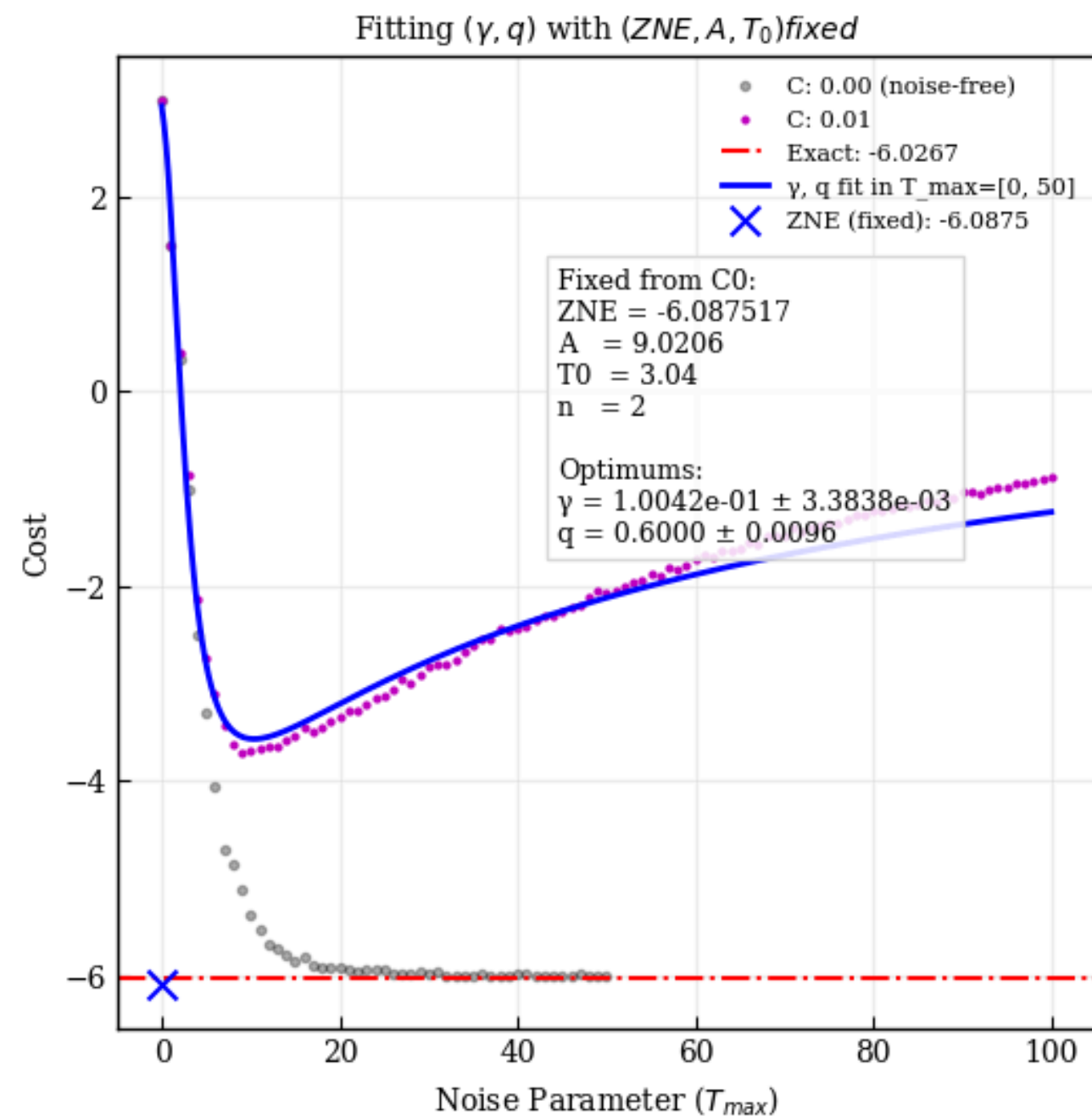
Fitting parameters (γ, ZNE, A, T_0, q)

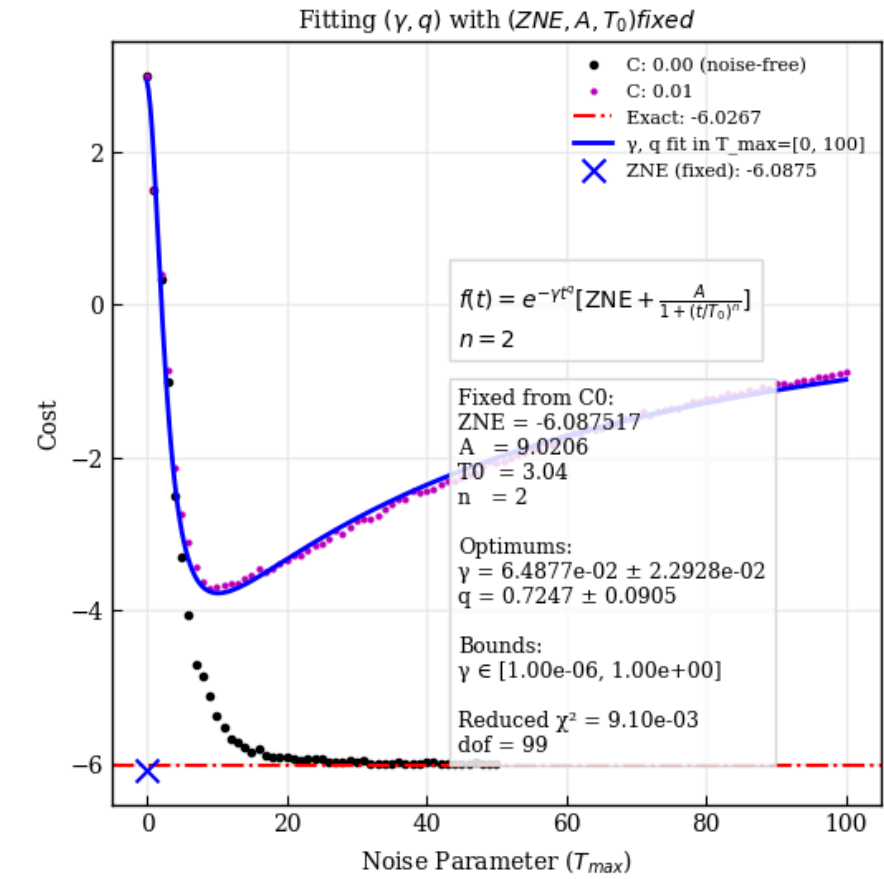
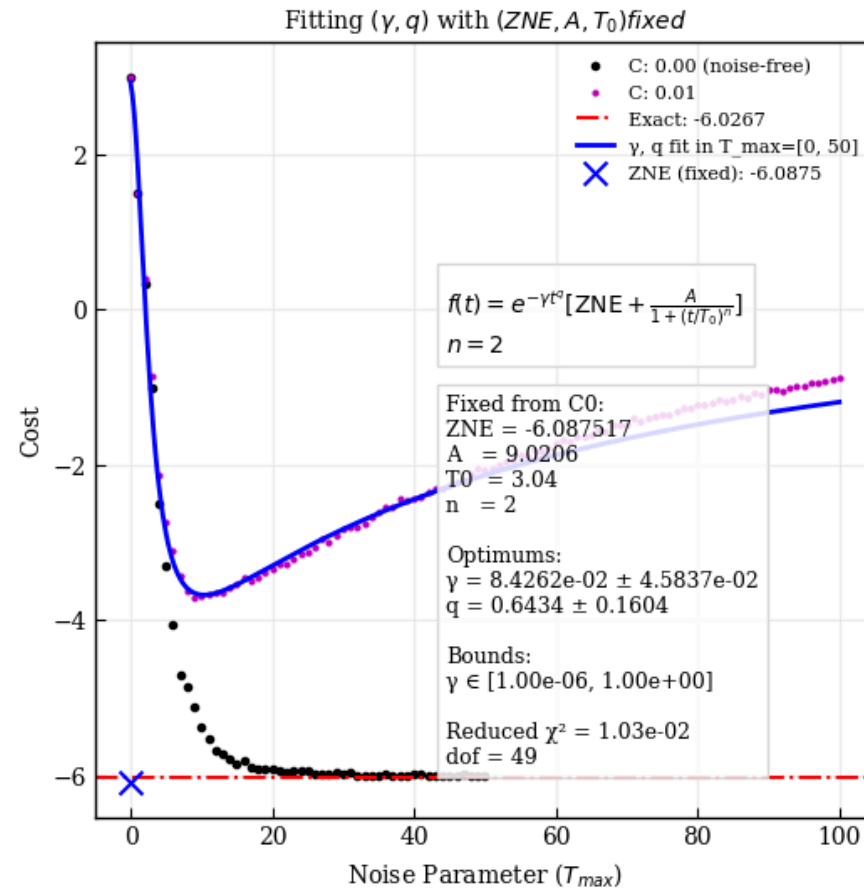
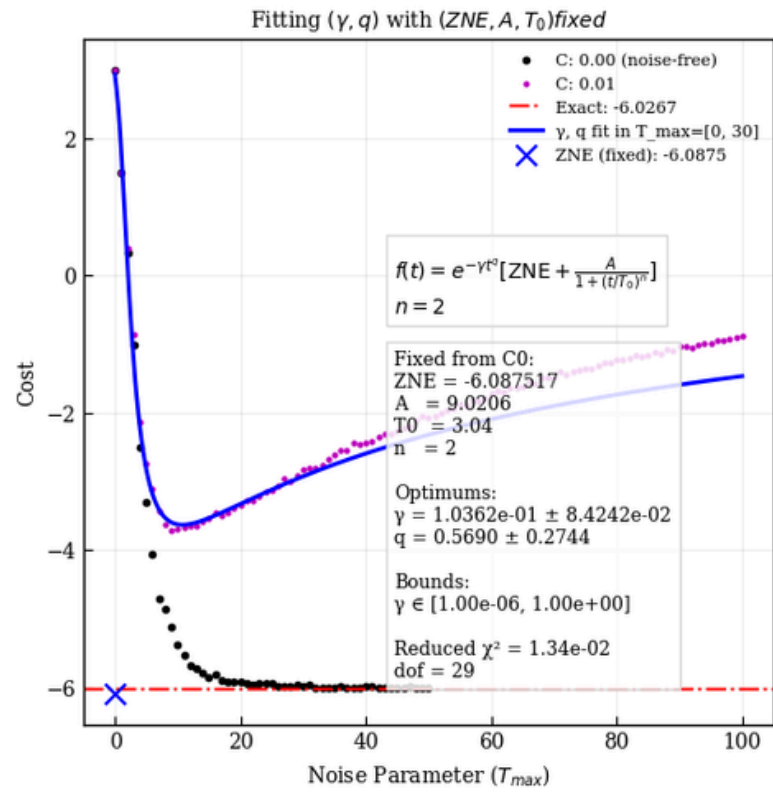
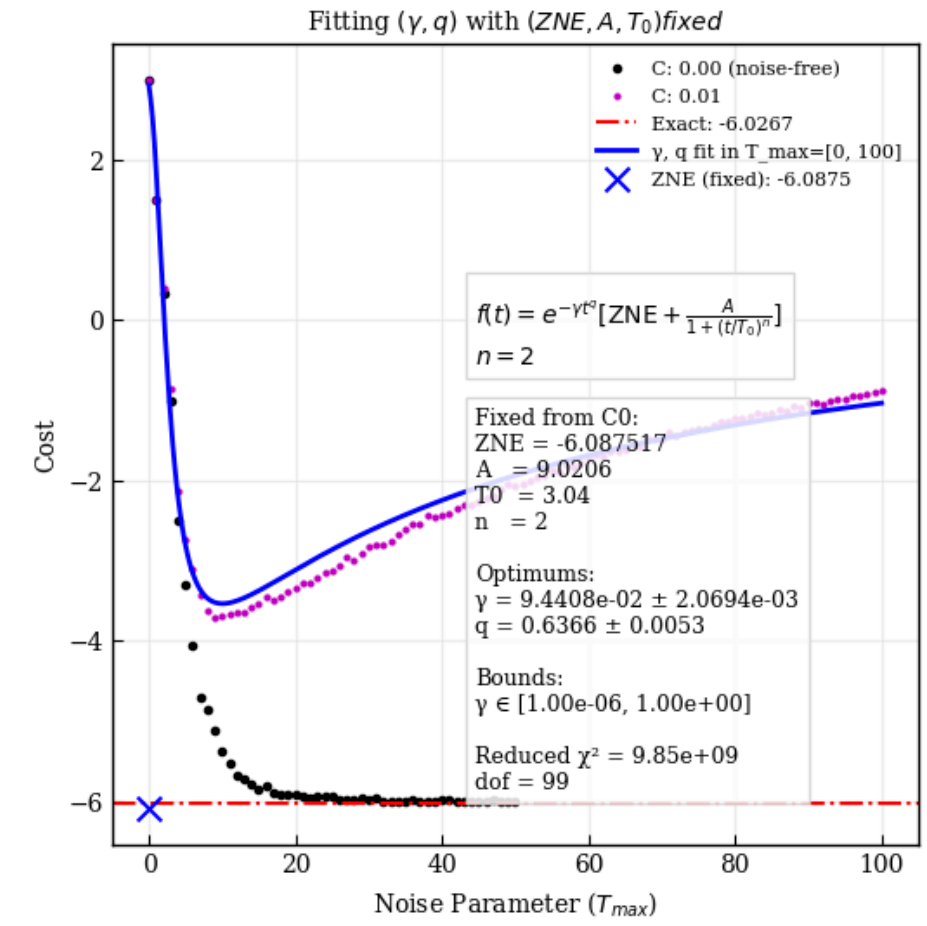
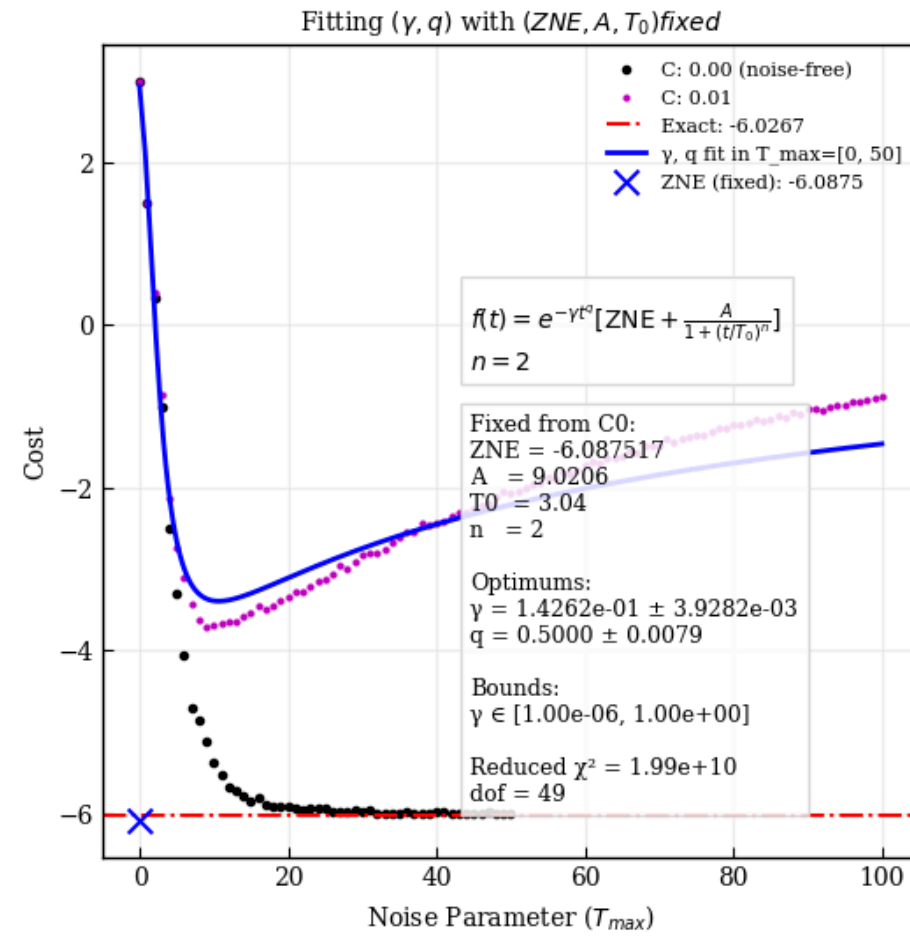
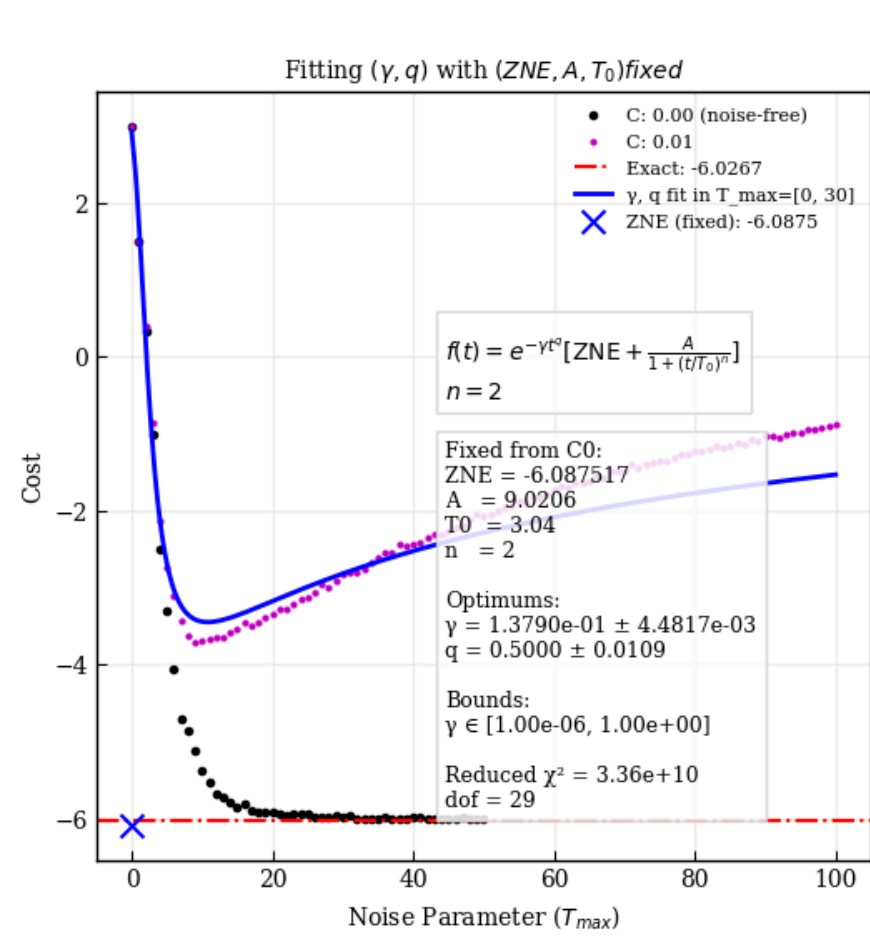
$$f_{\text{noisy}}(t) = e^{-\gamma t^q} \left[ZNE + \frac{A}{1 + \left(\frac{t}{T_0} \right)^n} \right]$$



Fitting γ, q with fixed parameters (ZNE, A, T0, n)

$$f_{\text{noisy}}(t) = e^{-\gamma t^q} \left[\text{ZNE} + \frac{A}{1 + \left(\frac{t}{T_0}\right)^n} \right]$$





Part. 2

R(T,C) analysis

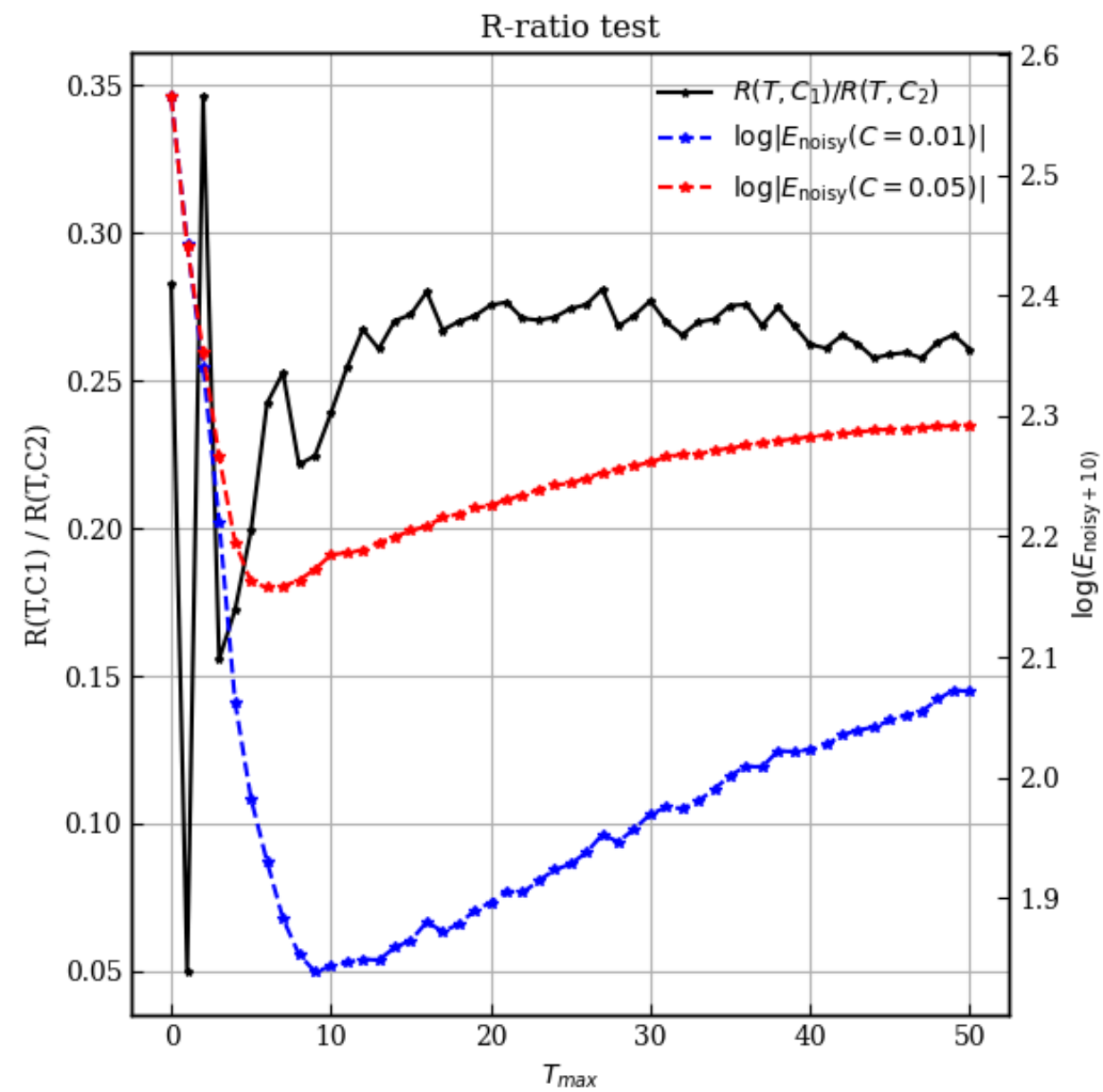
Could you calculate:

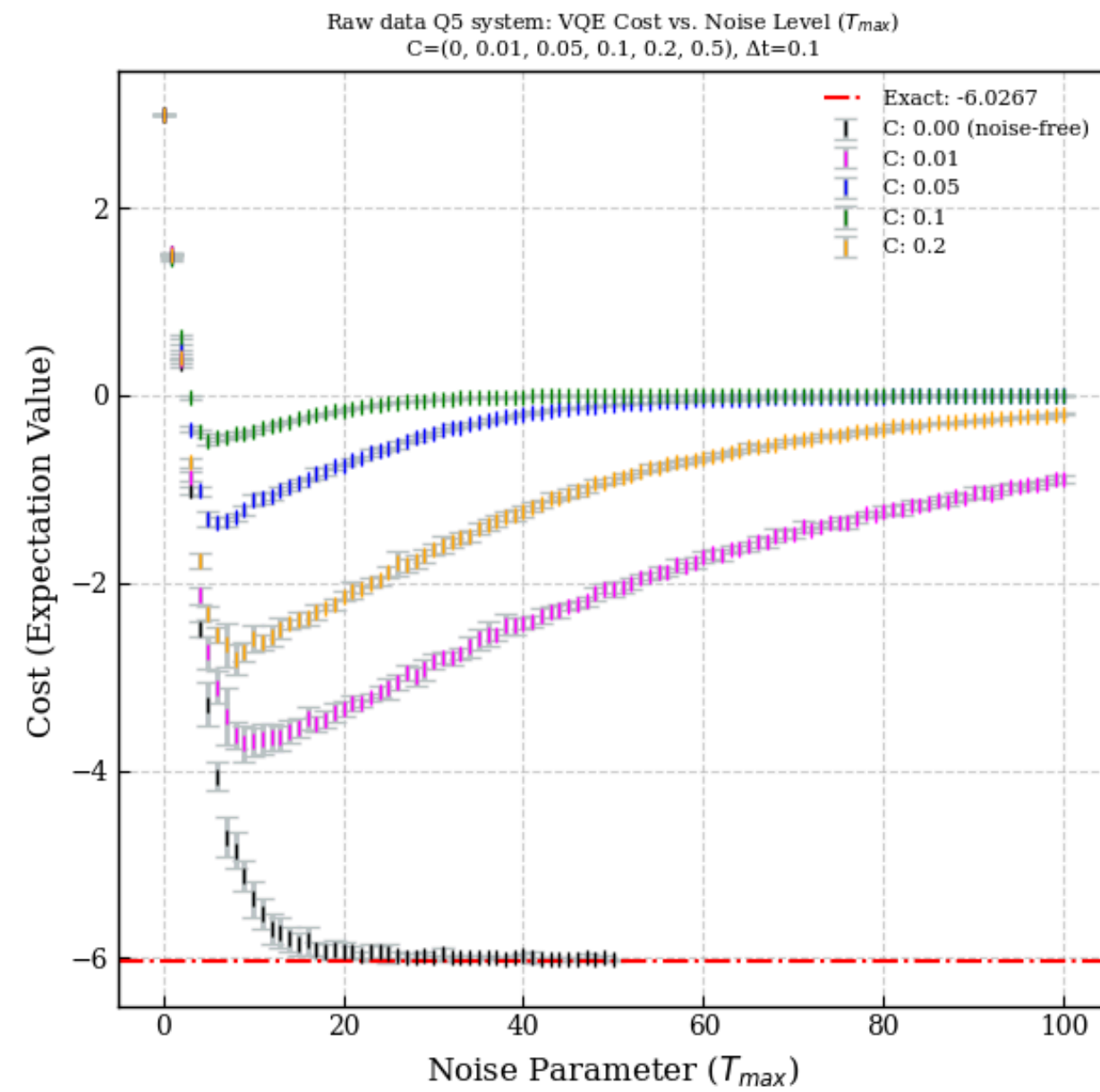
$$R(T_{\max}, C) := -\log \left(\frac{E_{\text{noisy}}(T_{\max}, C)}{E_{\text{ideal}}(T_{\max})} \right)$$

If the ratio of these values for different noise levels C_1 and C_2 is constant:

$$\frac{R(T, C_1)}{R(T, C_2)} \approx \text{const}(C_1, C_2)$$

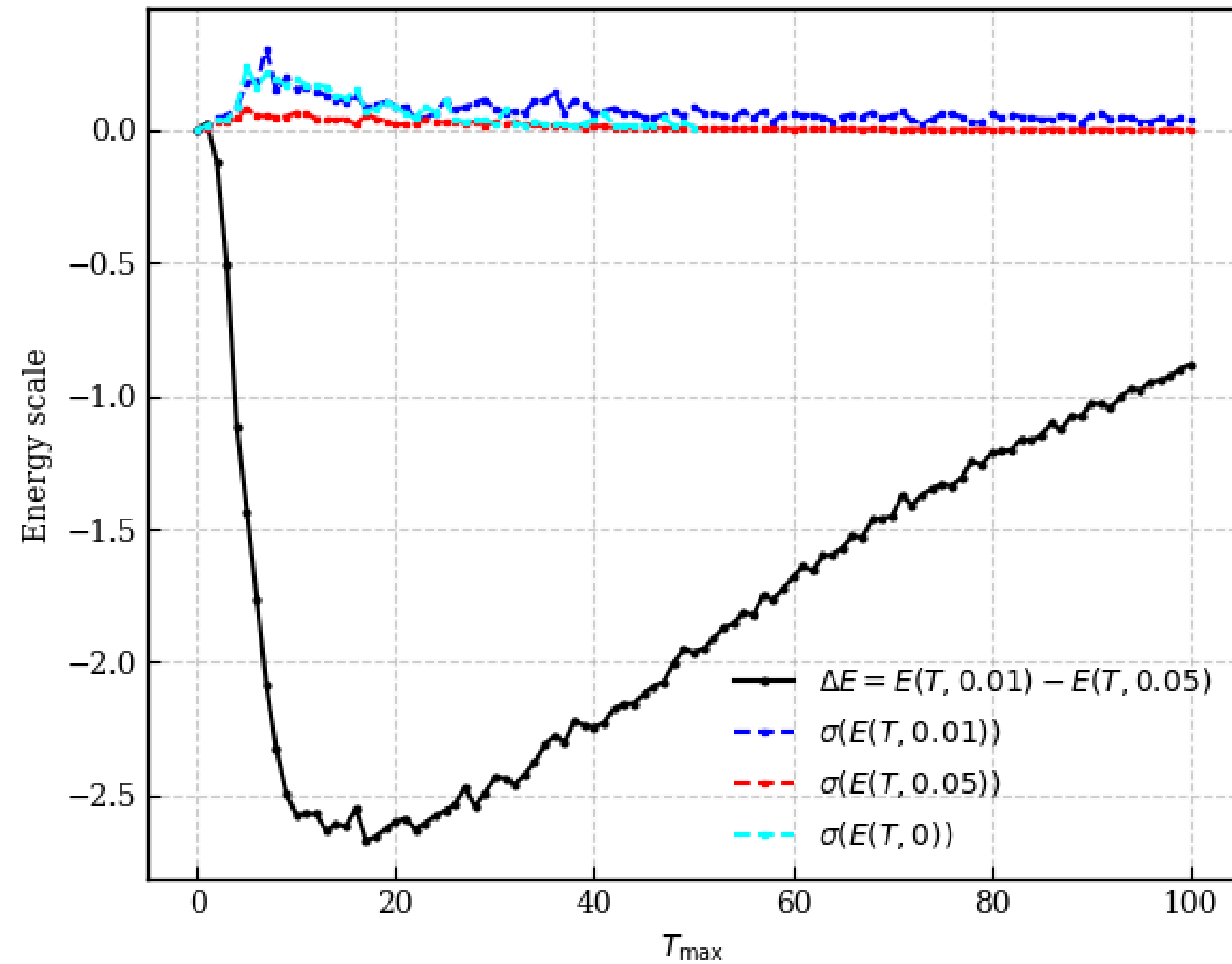
That is, if $\frac{R(T, C_1)}{R(T, C_2)}$ is almost constant with T , then we are seeing something physical.





Run more simulations using various C,
but C was not uniformly sampled

Comparison: ΔE vs STDs for $C=0.01$



Polynomial fitting of $R(T, C) = F0(T) + C \cdot F1(T) + C^2 \cdot F2(T) + C^3 \cdot F3(T)$

```
..  T      F0(T)      F1(T)      F2(T)      F3(T)
    3.000000      4.401211e-01      -4.056836e+01      1.337153e+03      -5.682684e+03
    4.000000      3.574668e-02      9.865535e+00      2.079100e+02      -1.247286e+03
    5.000000      6.990867e-02      9.334075e+00      2.191455e+02      -1.294247e+03
    6.000000      1.337955e-01      1.111326e+01      2.358865e+02      -1.415944e+03
    7.000000      1.423225e-01      1.569249e+01      1.987172e+02      -1.332172e+03
    8.000000      8.624152e-02      1.892494e+01      1.803660e+02      -1.318061e+03
    9.000000      6.252900e-02      2.511395e+01      1.004226e+02      -1.059287e+03
   10.000000      7.478775e-02      2.972897e+01      5.127189e+01      -9.181327e+02
   11.000000      1.272038e-01      2.757636e+01      1.011755e+02      -1.118390e+03
   12.000000      1.646895e-01      2.694707e+01      1.208639e+02      -1.199077e+03
   13.000000      1.495400e-01      2.906947e+01      1.078994e+02      -1.180571e+03
   14.000000      1.775816e-01      2.933183e+01      1.160028e+02      -1.227312e+03
   15.000000      1.890385e-01      3.004023e+01      1.220067e+02      -1.272383e+03
   16.000000      2.237854e-01      2.851355e+01      1.588367e+02      -1.422842e+03
   17.000000      1.816357e-01      3.322710e+01      1.118747e+02      -1.295282e+03
   18.000000      2.021374e-01      3.241433e+01      1.399300e+02      -1.414680e+03
   19.000000      2.174310e-01      3.262070e+01      1.562833e+02      -1.502167e+03
   20.000000      2.455853e-01      3.086282e+01      1.987662e+02      -1.668762e+03
   21.000000      2.579756e-01      3.170548e+01      2.089175e+02      -1.738319e+03
   22.000000      2.503525e-01      3.261518e+01      2.178820e+02      -1.803699e+03
   23.000000      2.590430e-01      3.318215e+01      2.324408e+02      -1.888008e+03
   24.000000      2.711822e-01      3.366493e+01      2.453358e+02      -1.964536e+03
   25.000000      3.023311e-01      3.142685e+01      2.980325e+02      -2.169766e+03
   26.000000      3.042420e-01      3.401986e+01      2.753217e+02      -2.114329e+03
   ...
   46.000000      4.673156e-01      4.718767e+01      6.067731e+02      -4.048510e+03
   47.000000      4.801747e-01      4.622601e+01      6.547954e+02      -4.266139e+03
   48.000000      5.177474e-01      4.629661e+01      6.710209e+02      -4.344689e+03
   49.000000      5.526682e-01      4.511422e+01      7.176844e+02      -4.553638e+03
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Could you change the degree and see how the remaining square error changes depending on degree.

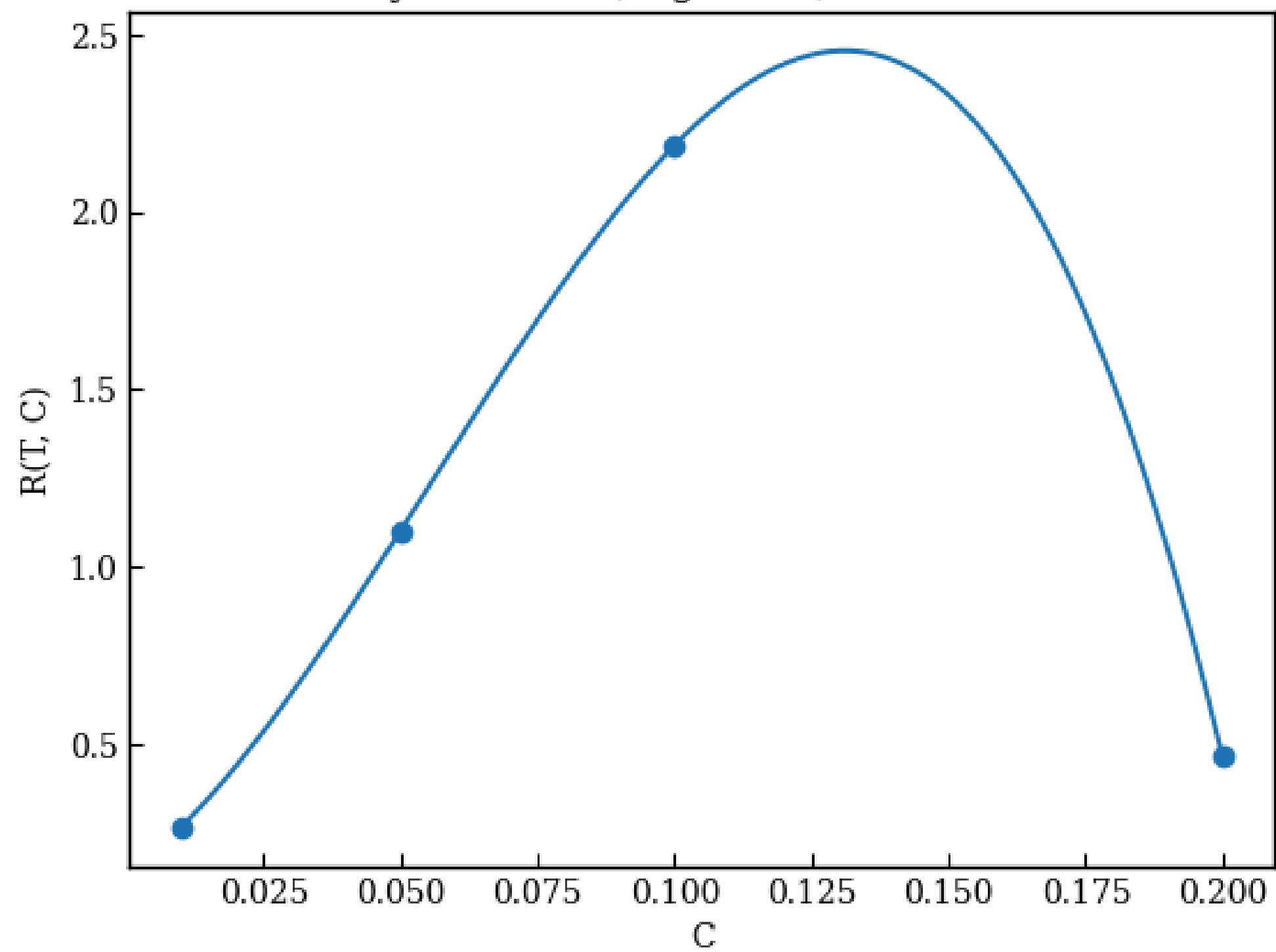
Polynomial degree vs residual error

Degree 1: RSS = 4.879212e+02

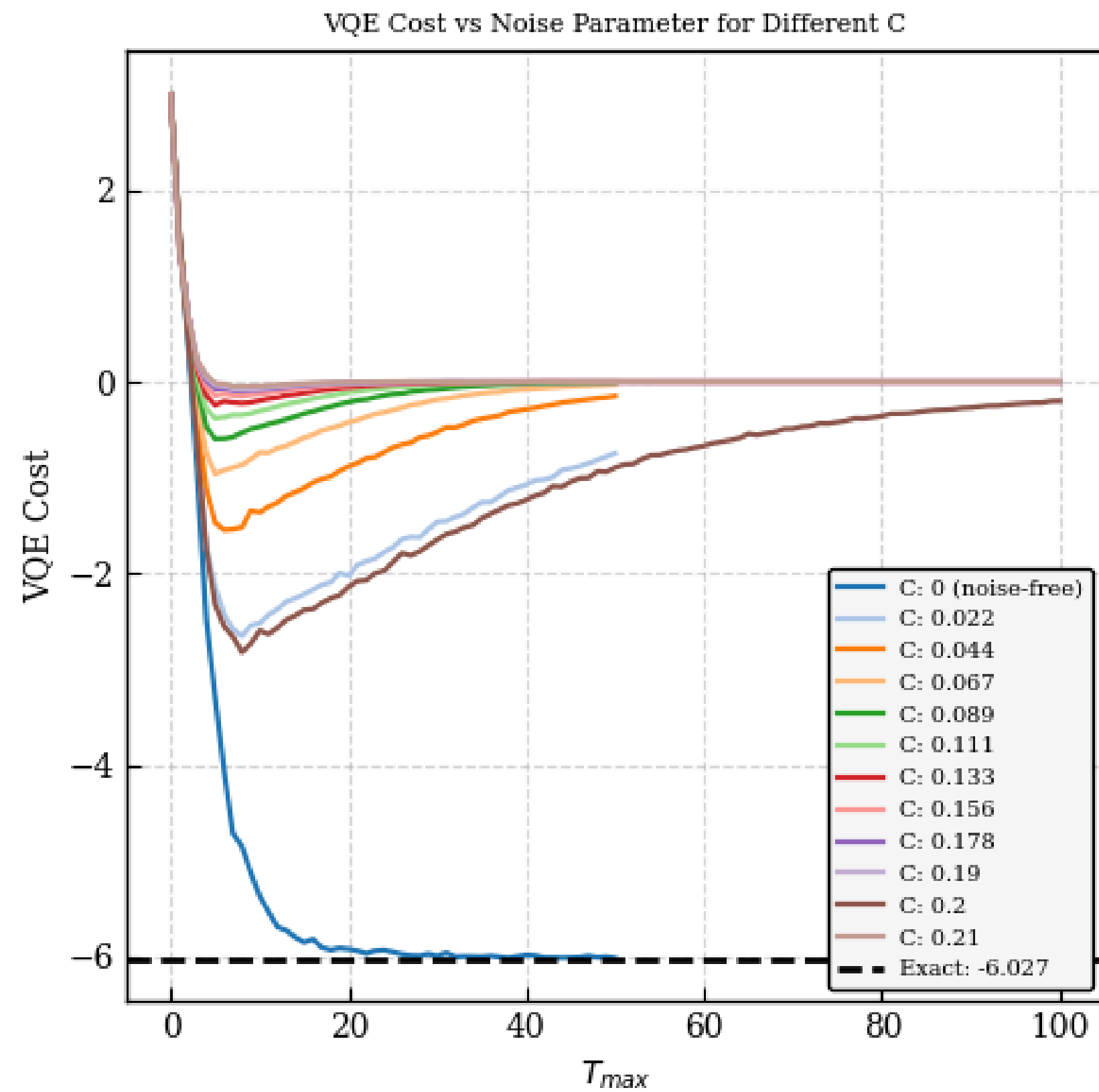
Degree 2: RSS = 1.830214e+01

Degree 3: RSS = 2.675059e-27

Polynomial fit (degree=3) at T index 3



Uniformly sampled $C = [0.0, 0.022, 0.044, 0.067, 0.089, 0.111, 0.133, 0.156, 0.178, 0.200]$



Polynomial Fitting of $R(T, C)$

Since $R(T, C = 0) = 0$ for all T ,
the constant term must vanish in the polynomial expansion.

Therefore, we impose $F_0(T) = 0$, and write

$$R(T, C) = C F_1(T) + C^2 F_2(T) + C^3 F_3(T) + C^4 F_4(T) + \dots$$

This enforces the physical constraint that the noise response vanishes when $C = 0$.

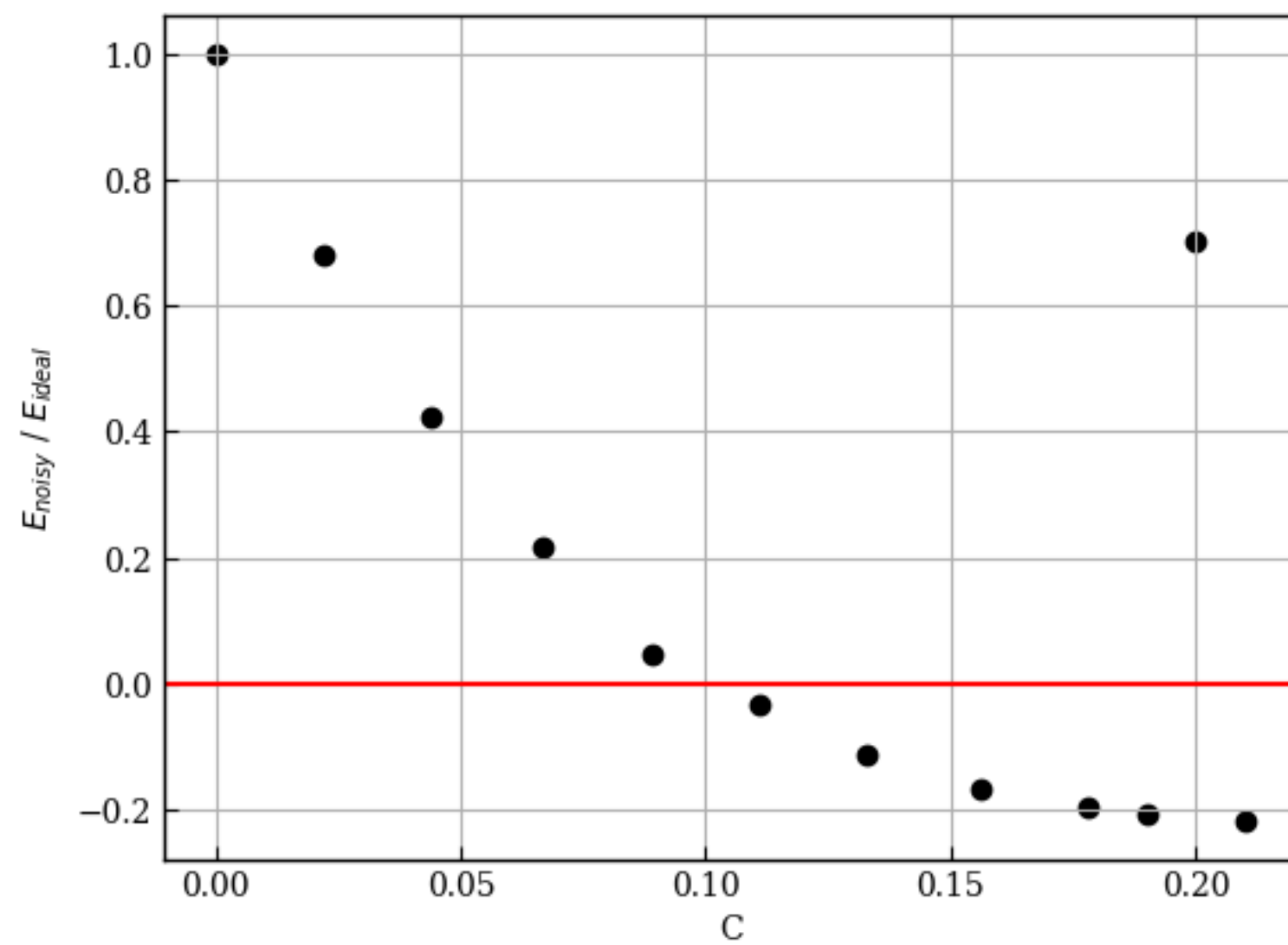
...

Degree 1 fit at $T = 3$
 $F_1(T=3) = 1.410072e+01$

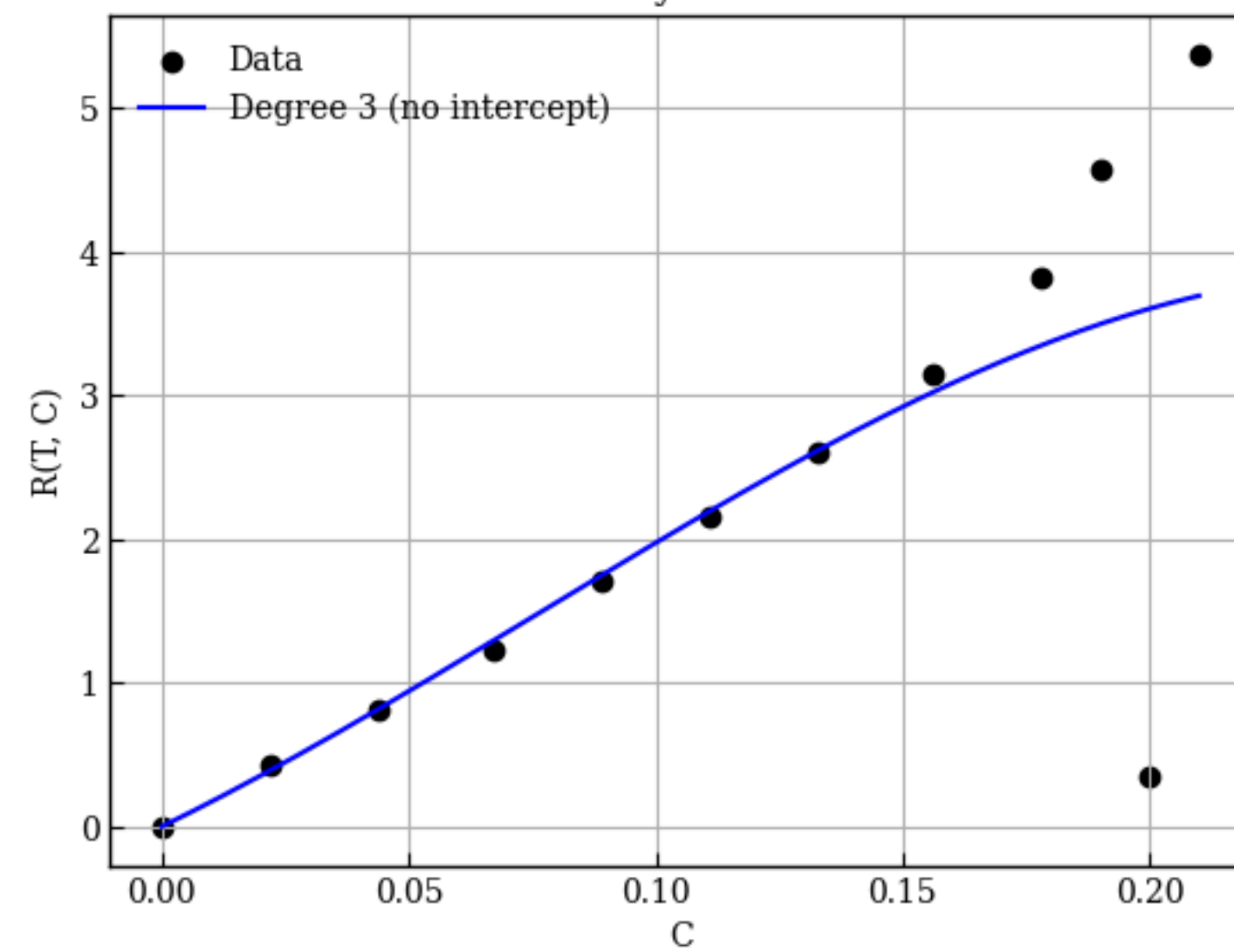
Degree 2 fit at $T = 3$
 $F_1(T=3) = 4.798220e+01$
 $F_2(T=3) = -2.233795e+02$

Degree 3 fit at $T = 3$
 $F_1(T=3) = -1.612367e+01$
 $F_2(T=3) = 9.771890e+02$
 $F_3(T=3) = -4.437050e+03$

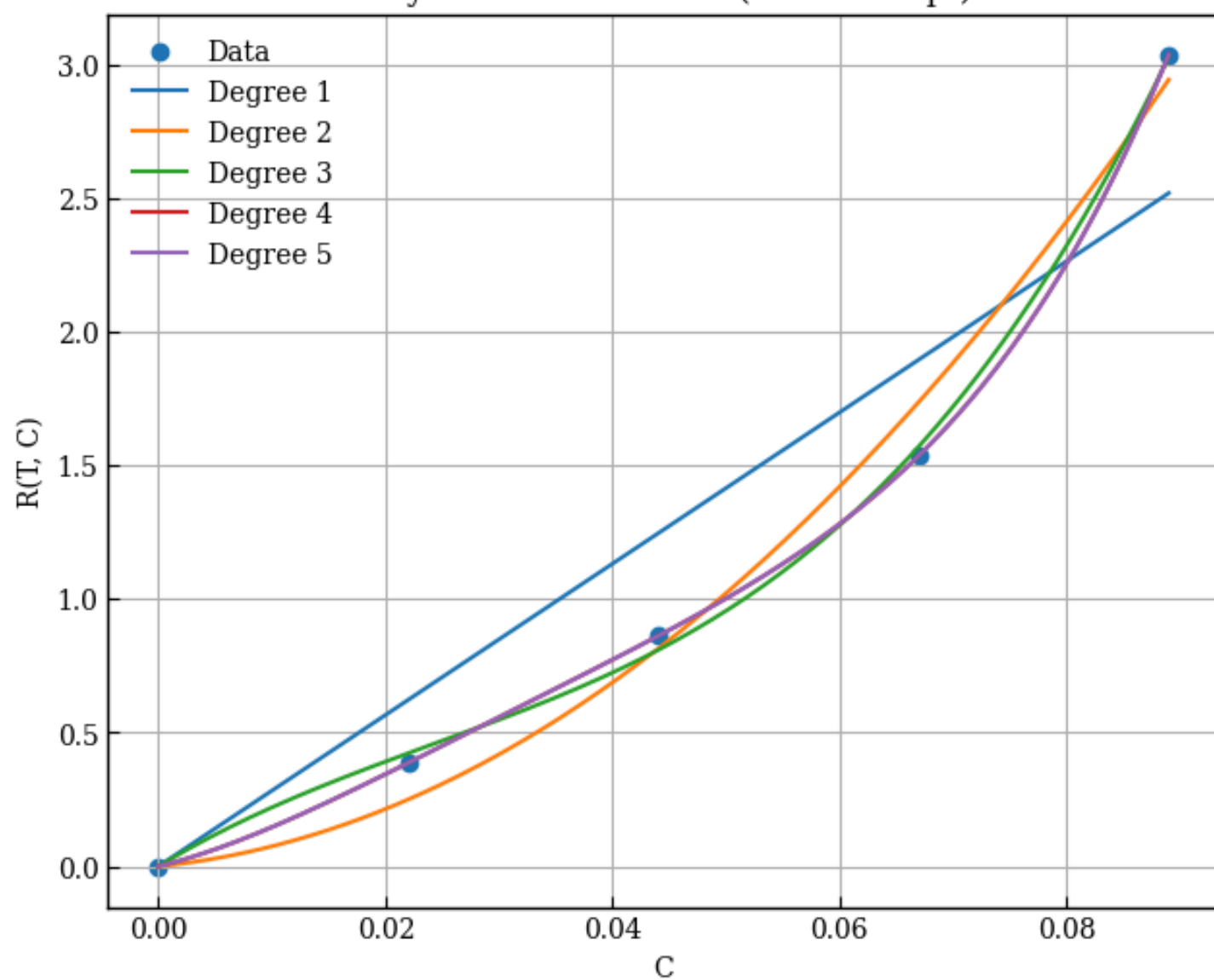
Ratio vs C at T=3



Constrained Polynomial Fit at T=5



Polynomial Fits at T=3 (No Intercept)



Degree 1
F1 = 2.829e+01

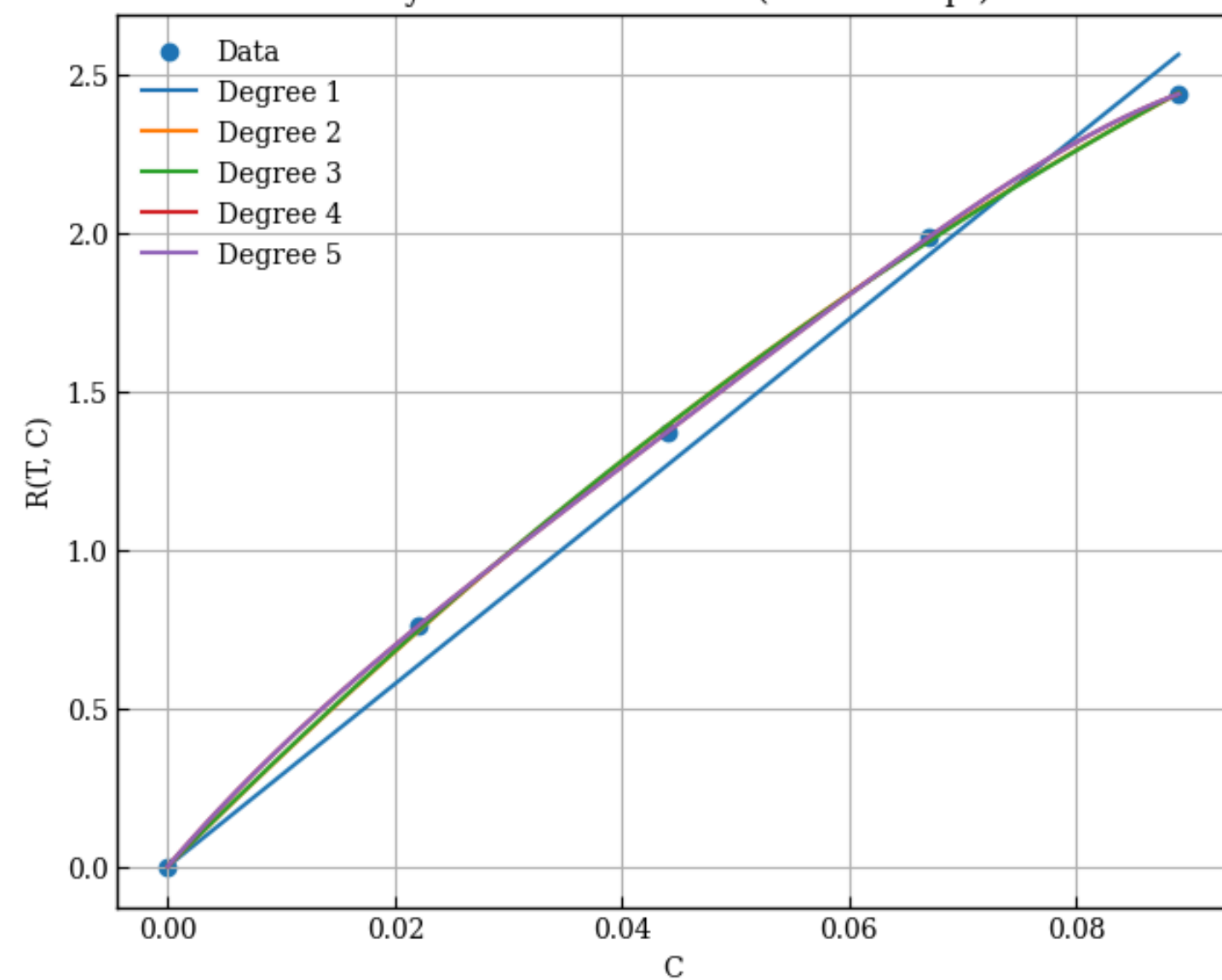
Degree 2
F1 = 4.231e+00
F2 = 3.239e+02

Degree 3
F1 = 2.565e+01
F2 = -4.190e+02
F3 = 5.764e+03

Degree 4
F1 = 1.018e+01
F2 = 5.572e+02
F3 = -1.229e+04
F4 = 1.017e+05

Degree 5
F1 = 1.031e+01
F2 = 5.454e+02
F3 = -1.191e+04
F4 = 9.684e+04
F5 = 2.170e+04

Polynomial Fits at T=10 (No Intercept)



Degree 1
F1 = 2.880e+01

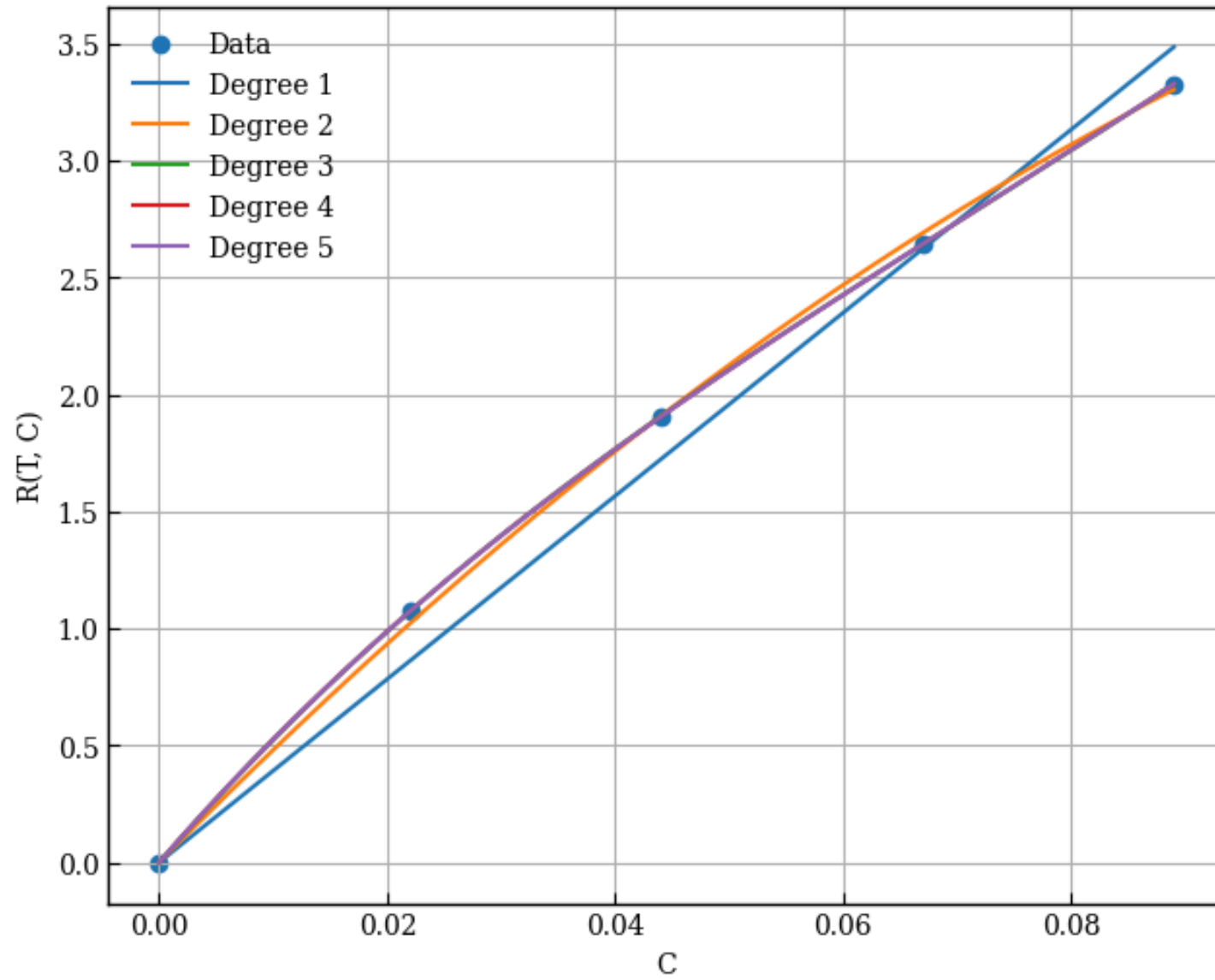
Degree 2
F1 = 3.574e+01
F2 = -9.347e+01

Degree 3
F1 = 3.606e+01
F2 = -1.043e+02
F3 = 8.442e+01

Degree 4
F1 = 4.189e+01
F2 = -4.725e+02
F3 = 6.892e+03
F4 = -3.834e+04

Degree 5
F1 = 4.184e+01
F2 = -4.681e+02
F3 = 6.751e+03
F4 = -3.652e+04
F5 = -8.223e+03

Polynomial Fits at T=20 (No Intercept)



Degree 1
F1 = 3.917e+01

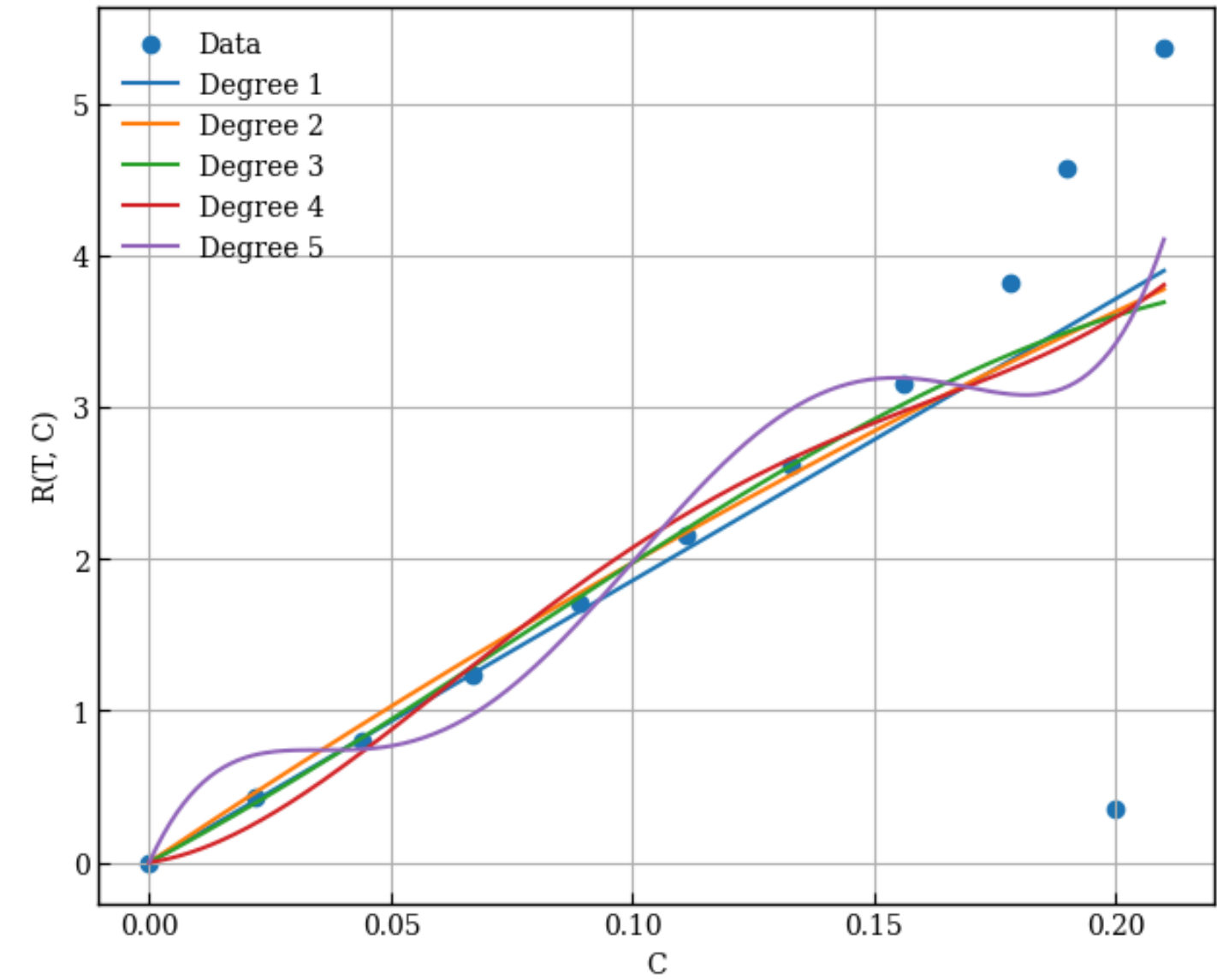
Degree 2
F1 = 4.950e+01
F2 = -1.390e+02

Degree 3
F1 = 5.578e+01
F2 = -3.569e+02
F3 = 1.691e+03

Degree 4
F1 = 5.630e+01
F2 = -3.896e+02
F3 = 2.295e+03
F4 = -3.400e+03

Degree 5
F1 = 5.630e+01
F2 = -3.892e+02
F3 = 2.282e+03
F4 = -3.232e+03
F5 = -7.571e+02

Polynomial Fits at T=5 (No Intercept) [scikit-learn ver.]



Degree 1
F1 = 1.857e+01

Degree 2
F1 = 2.135e+01
F2 = -1.602e+01

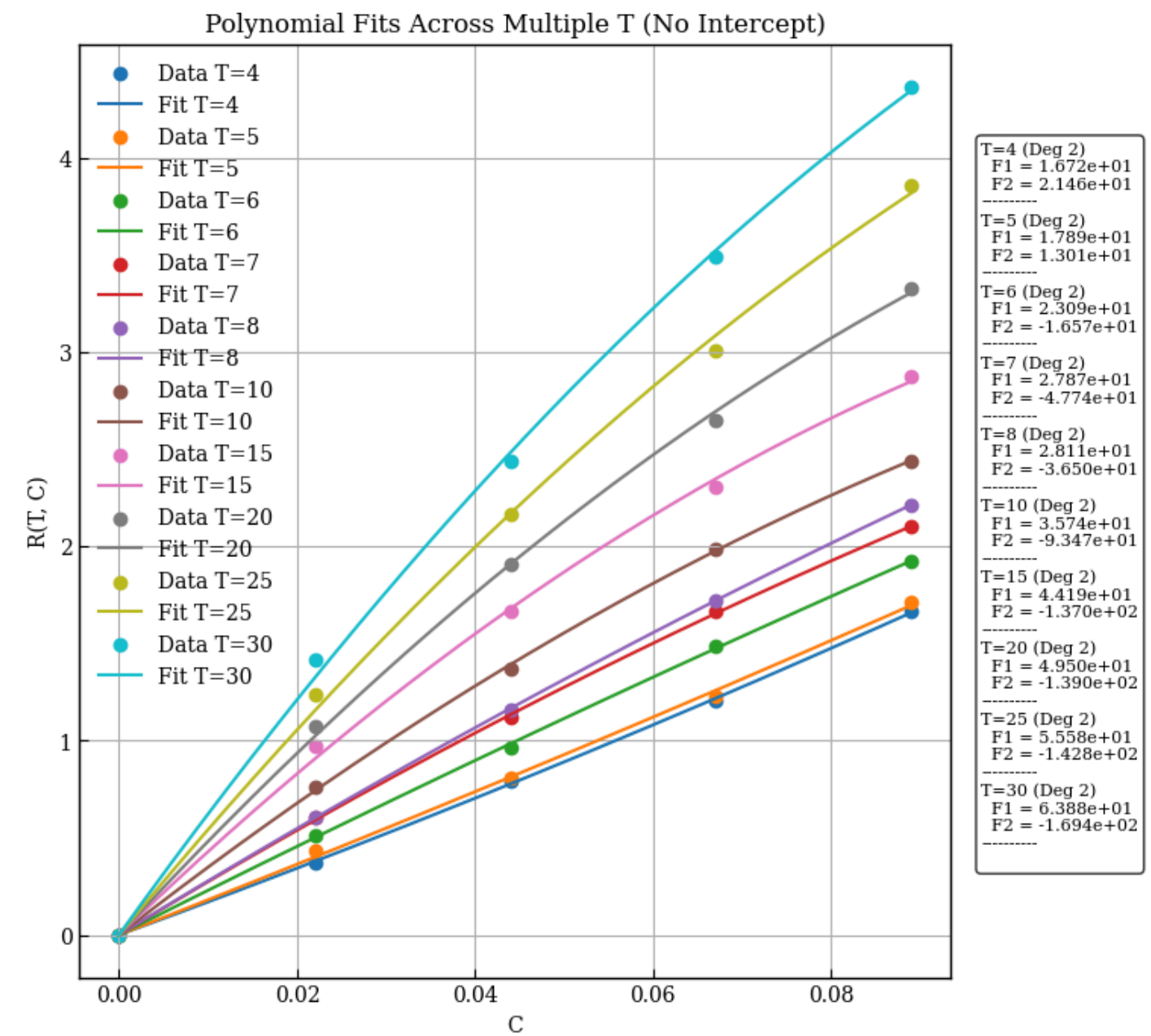
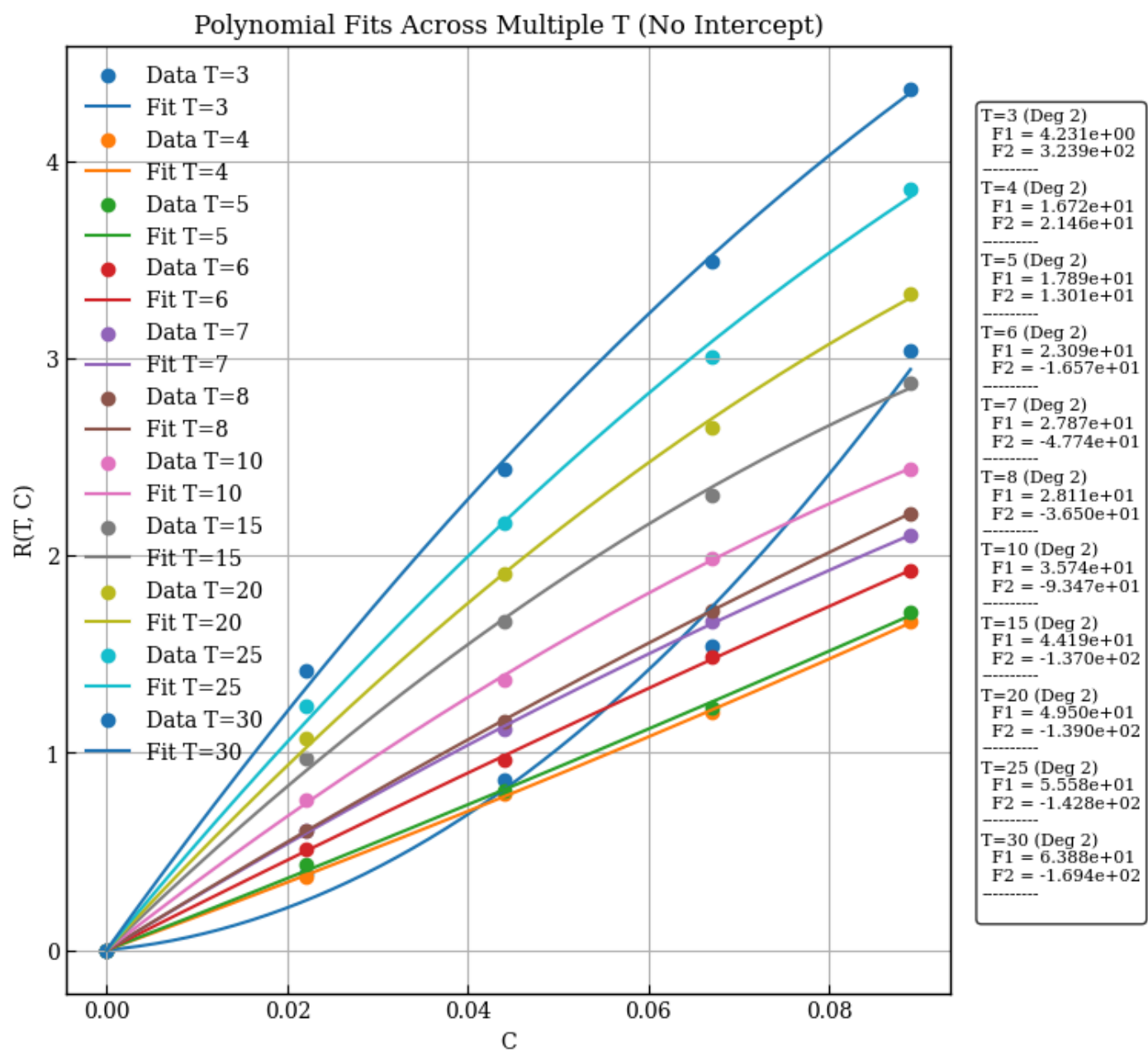
Degree 3
F1 = 1.673e+01
F2 = 5.380e+01
F3 = -2.369e+02

Degree 4
F1 = 4.597e+00
F2 = 3.846e+02
F3 = -2.874e+03
F4 = 6.425e+03

Degree 5
F1 = 6.997e+01
F2 = -2.394e+03
F3 = 3.512e+04
F4 = -2.016e+05
F5 = 3.963e+05

OK, So, Degree 2 may be enough. Let us assume Degree 2 and reconstruct $F1(t)$ and $F2(t)$. We further try to fit $F1(T)$ and $F2(T)$ Then, we can approximately get a form of function $R(T, C)$

T=3 looks anomolous too. let us remove T=3. By using T>3, let us implement polynomial fitting F1(T) and F2(T) (edited)



From these data, you can further fit:

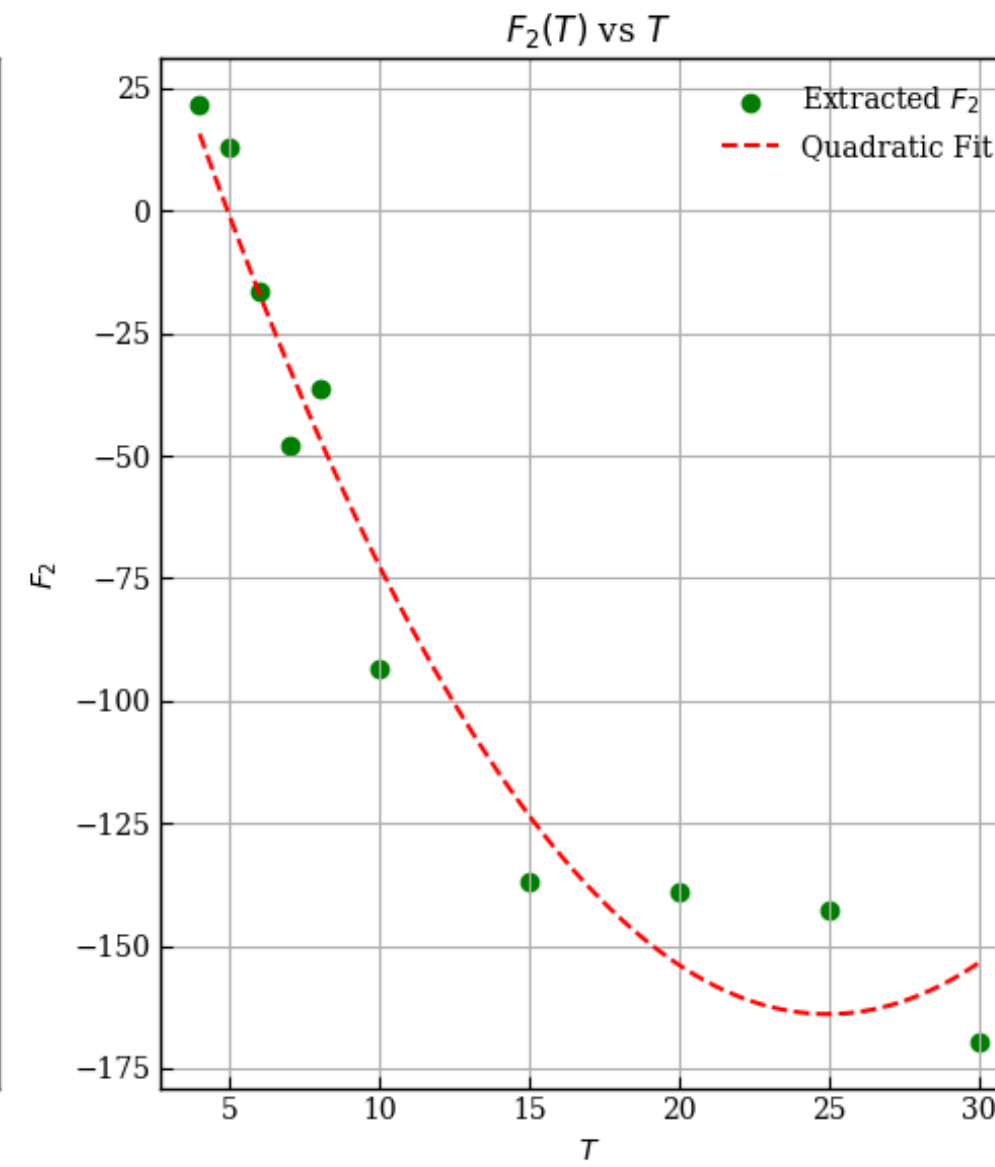
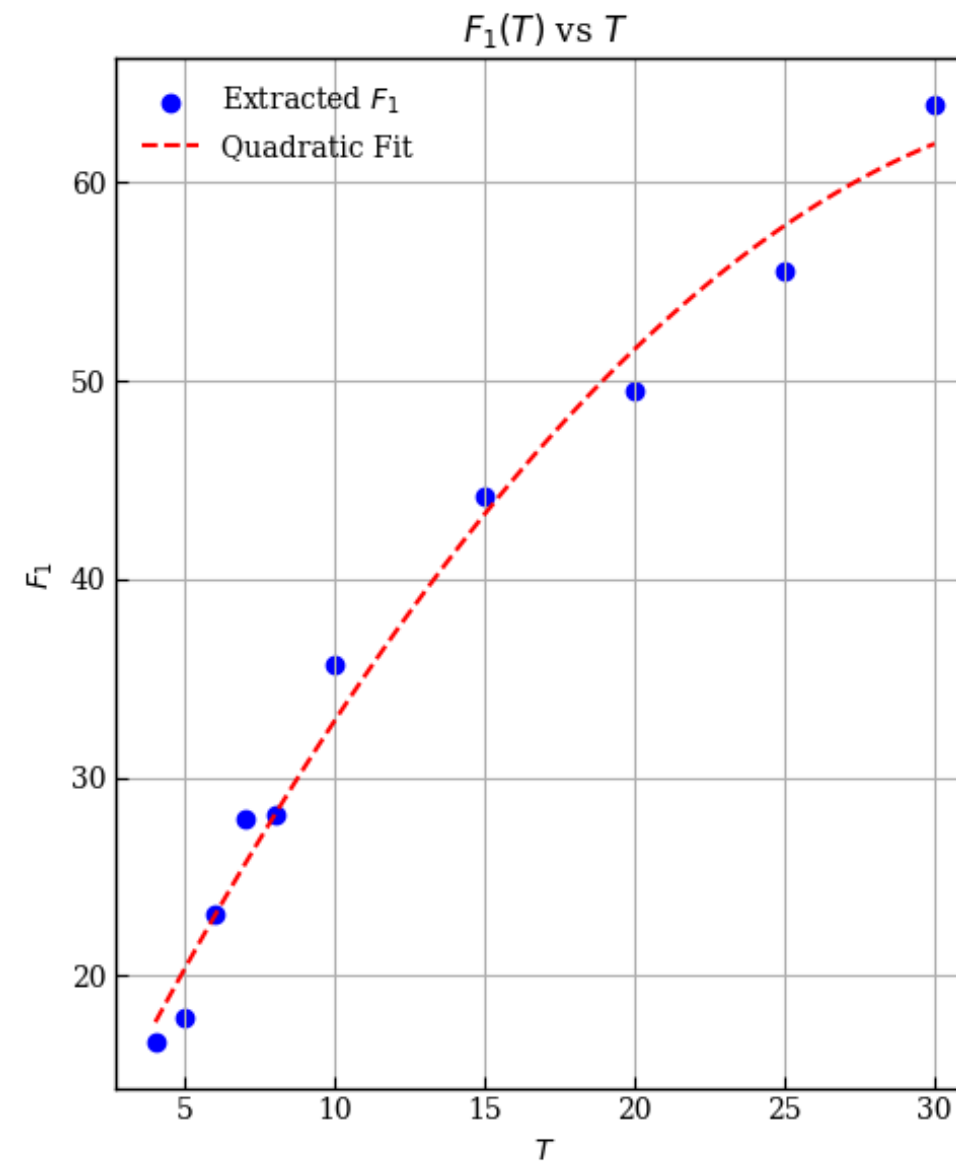
$$F_1(T) = F_{10} + F_{11}T + F_{12}T^2$$

$$F_2(T) = F_{20} + F_{21}T + F_{22}T^2$$

and find the coefficients F_{10} , F_{11} , F_{12} , F_{20} , F_{21} , and F_{22} .

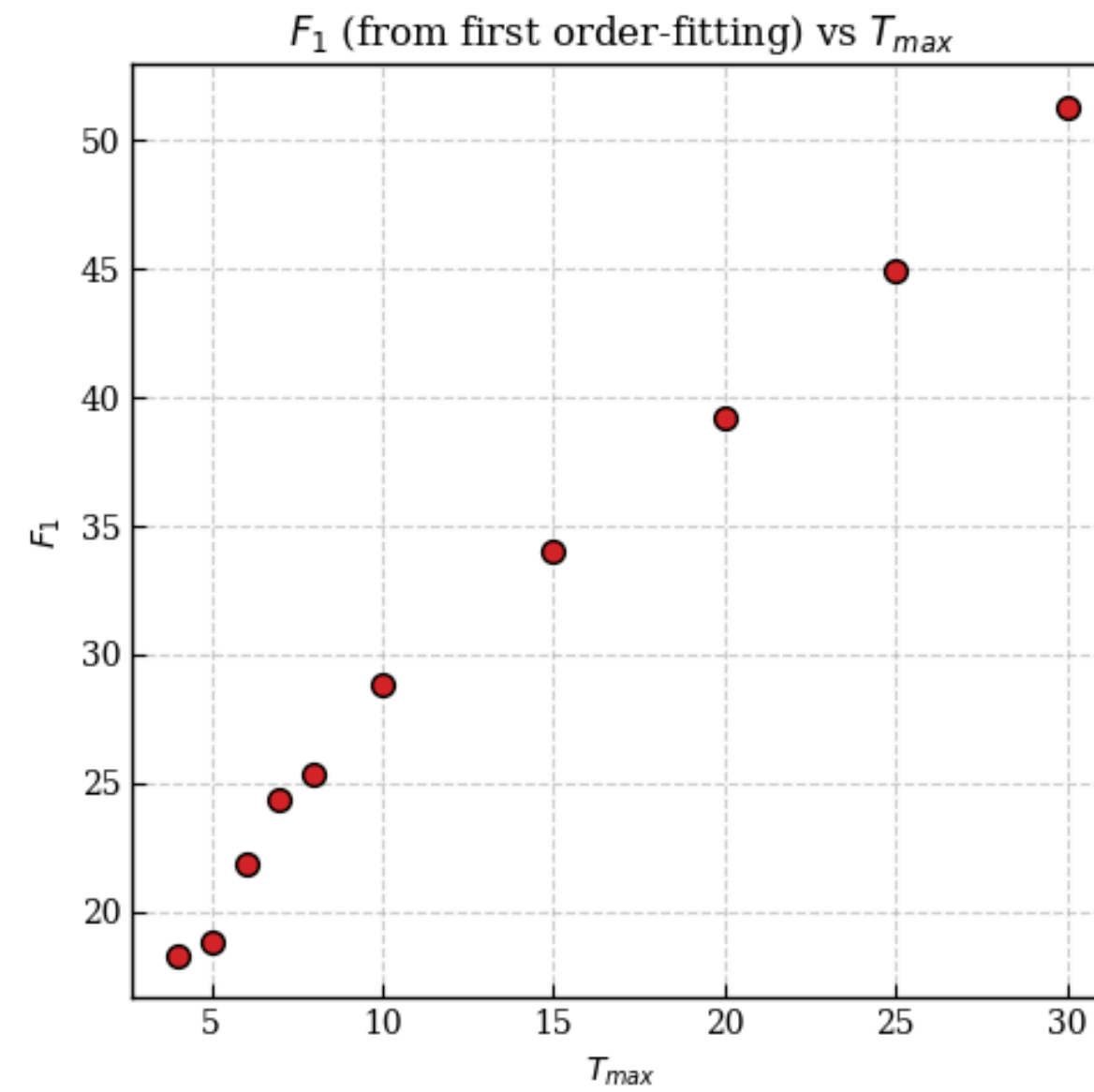
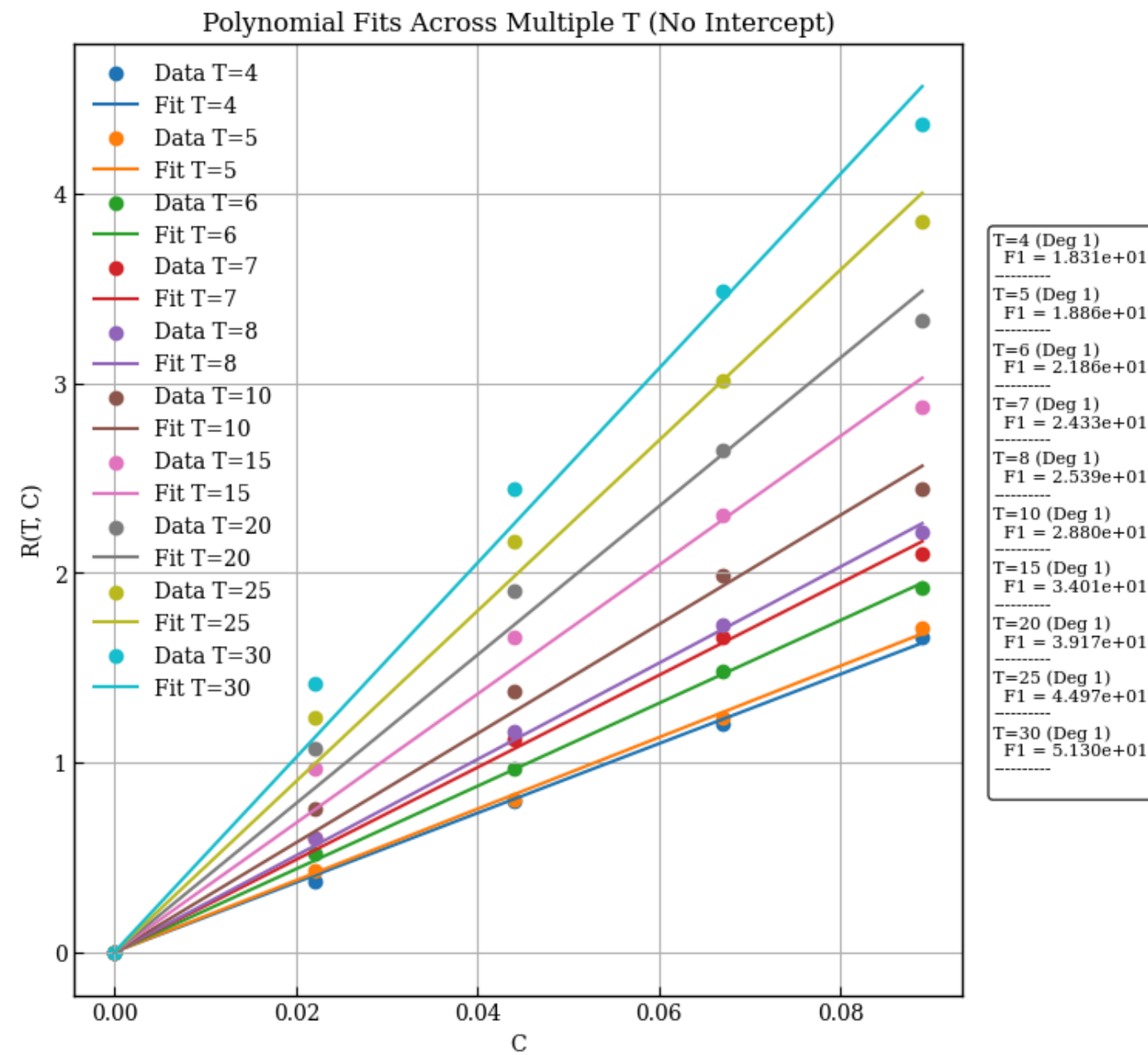
I would like you to plot $F_1(T)$ and $F_2(T)$ where:

- **Plot 1:** y-axis is $F_1(T)$ and x-axis is T .
- **Plot 2:** y-axis is $F_2(T)$ and x-axis is T .



```
.. F1 coefficients: F10=5.8391e+00, F11=3.1243e+00, F12=-4.1792e-02
   F2 coefficients: F20=9.0816e+01, F21=-2.0422e+01, F22=4.0929e-01
```

Could you plot T versus $F_1(T)$ in the above first order fitting?



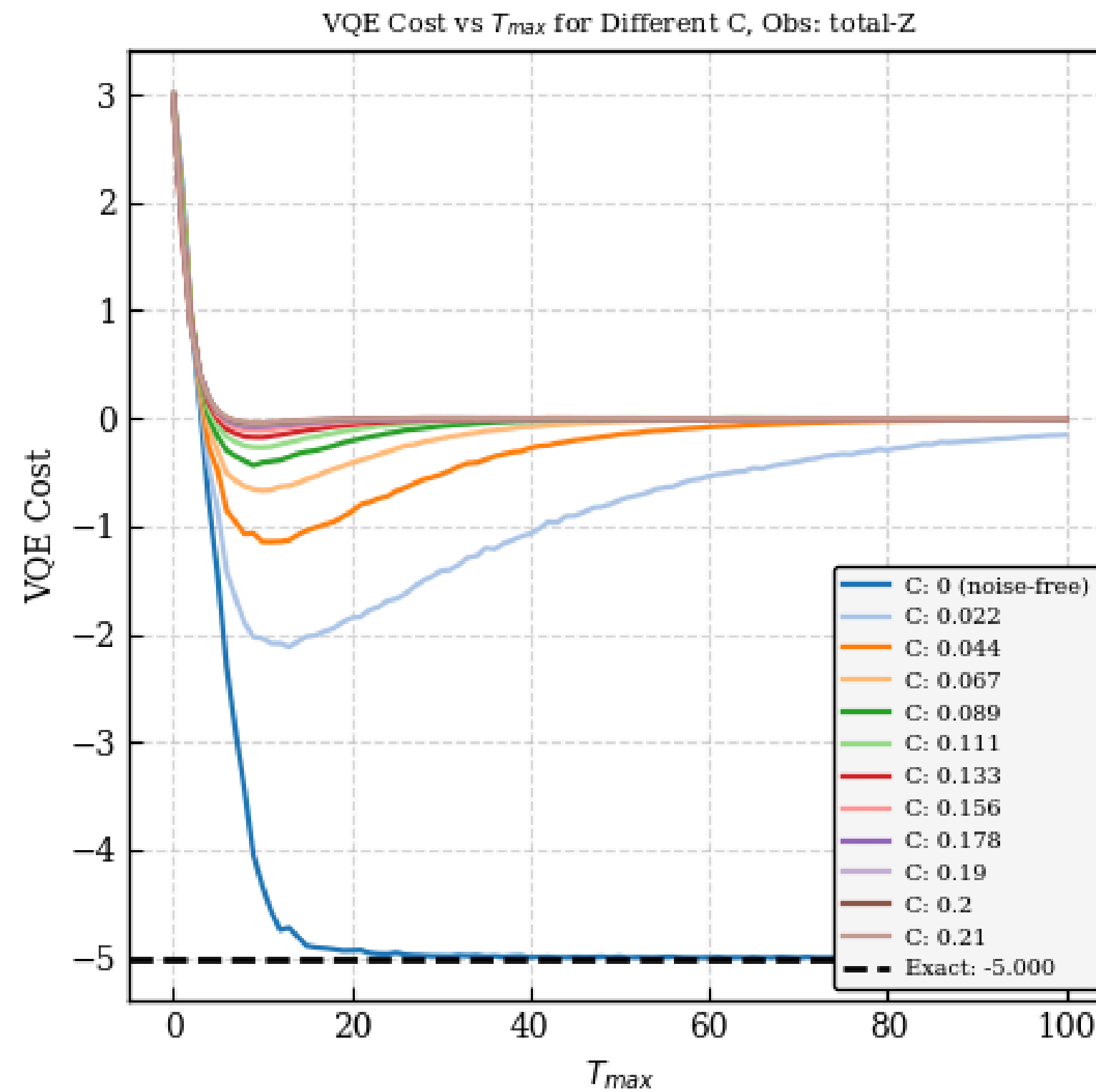
Part. 3

$R(T,C)$ analysis using a different observable

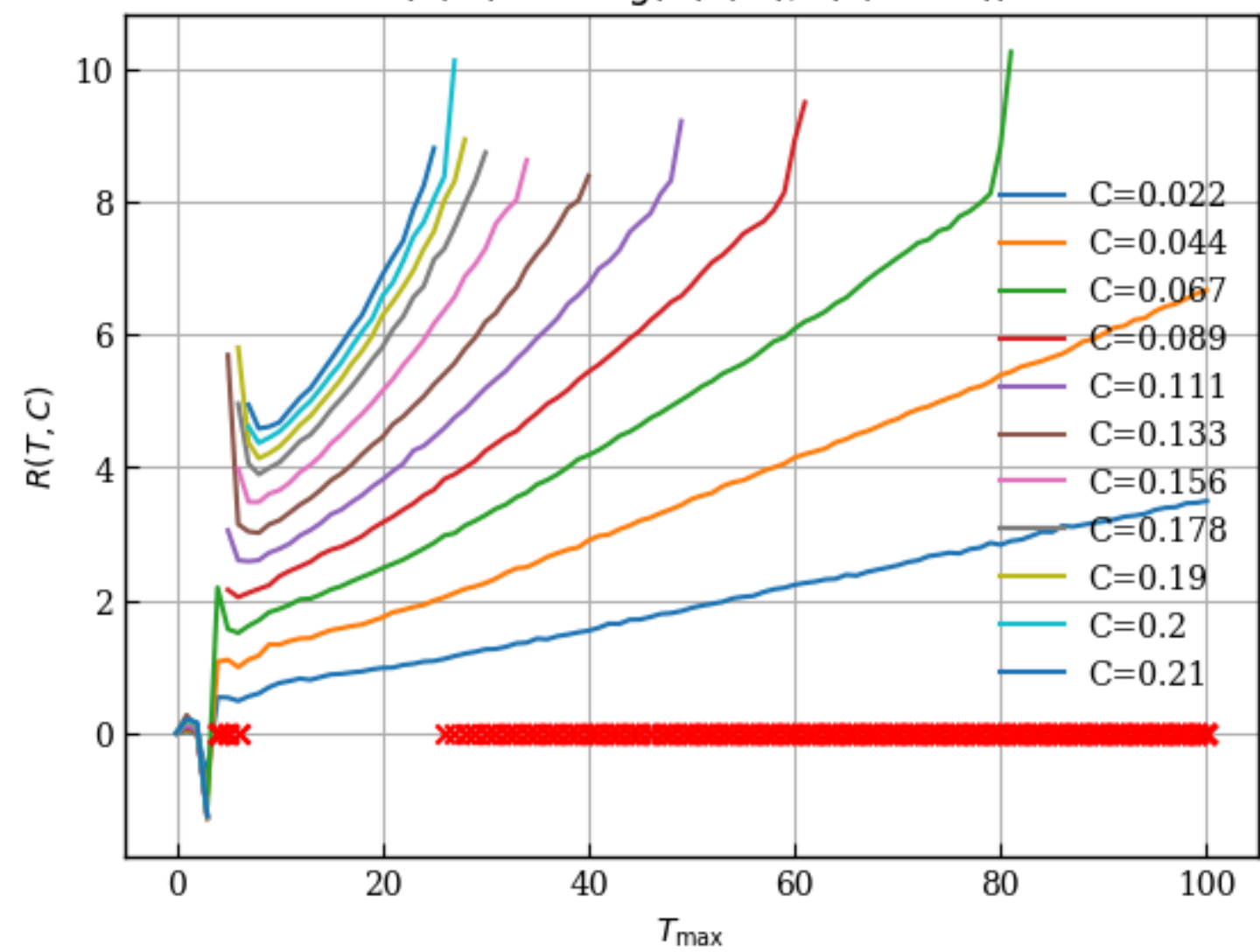
Simulation using a different observable:

$$H_{\text{Obs}} = \sum_i Z_i$$

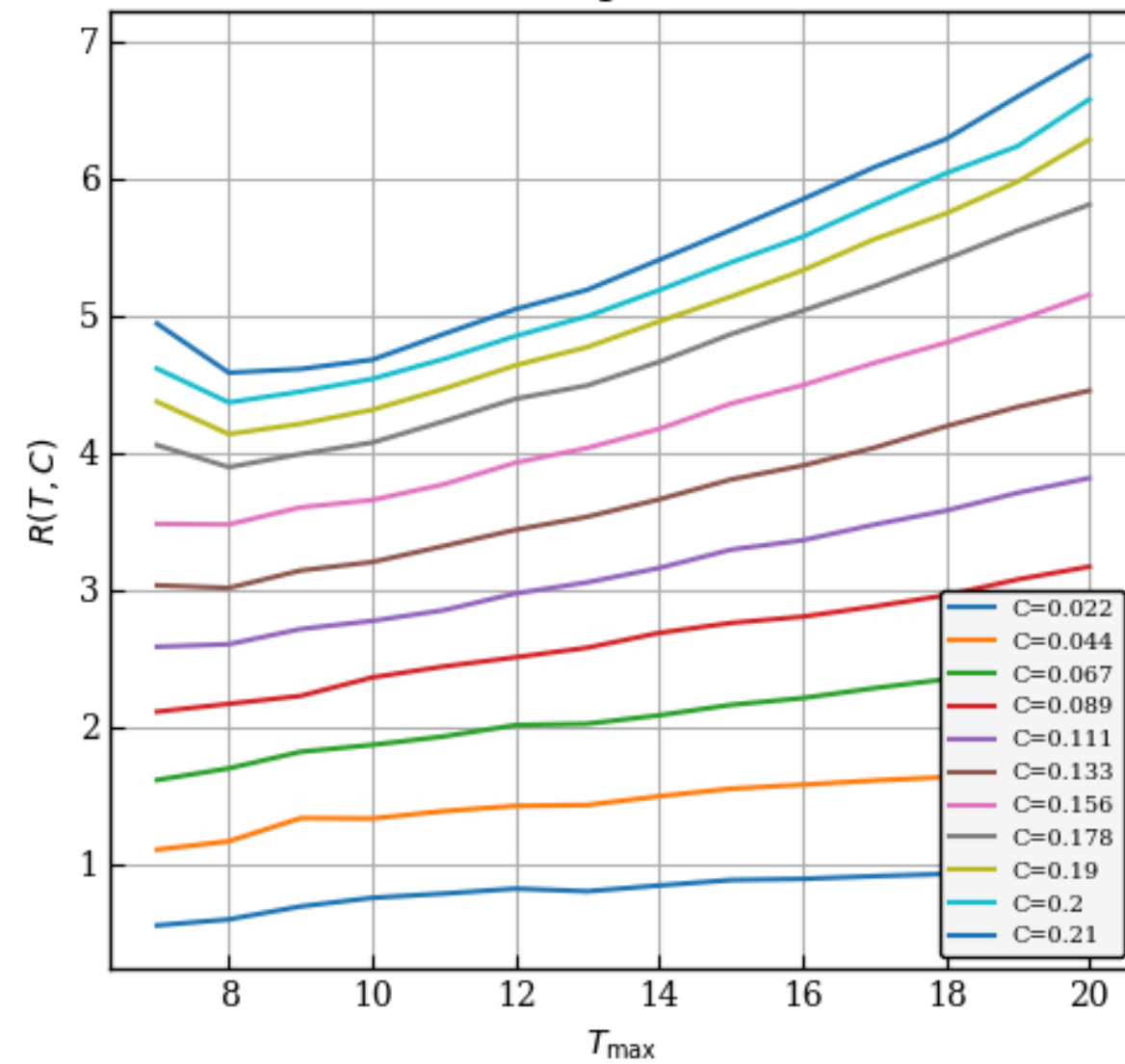
Loading simulation data for H_{Obs}



$$R(T, C) = -\log(E(T, C)/E(T, C=0))$$



$$R(T, C) = -\log(E(T, C)/E(T, 0))$$

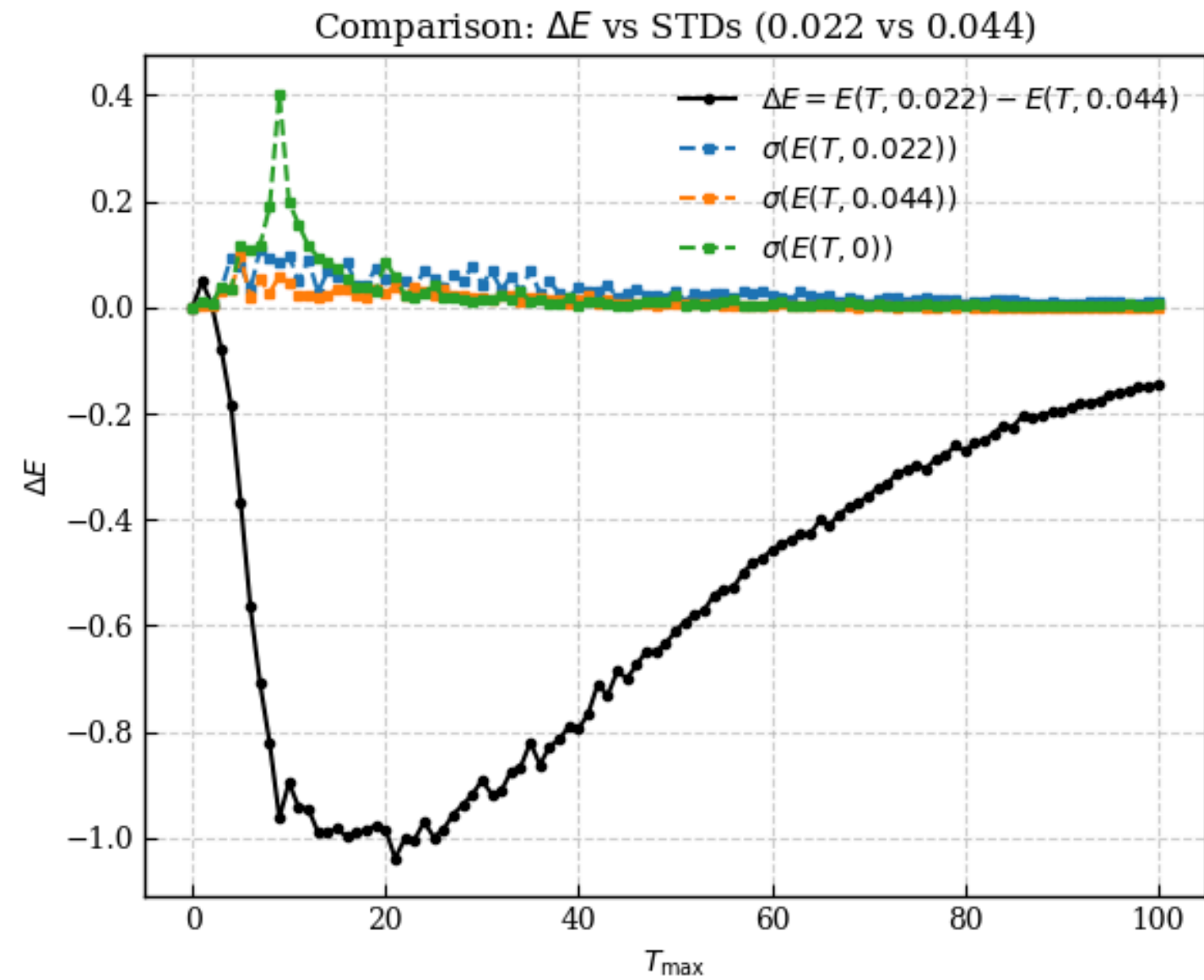


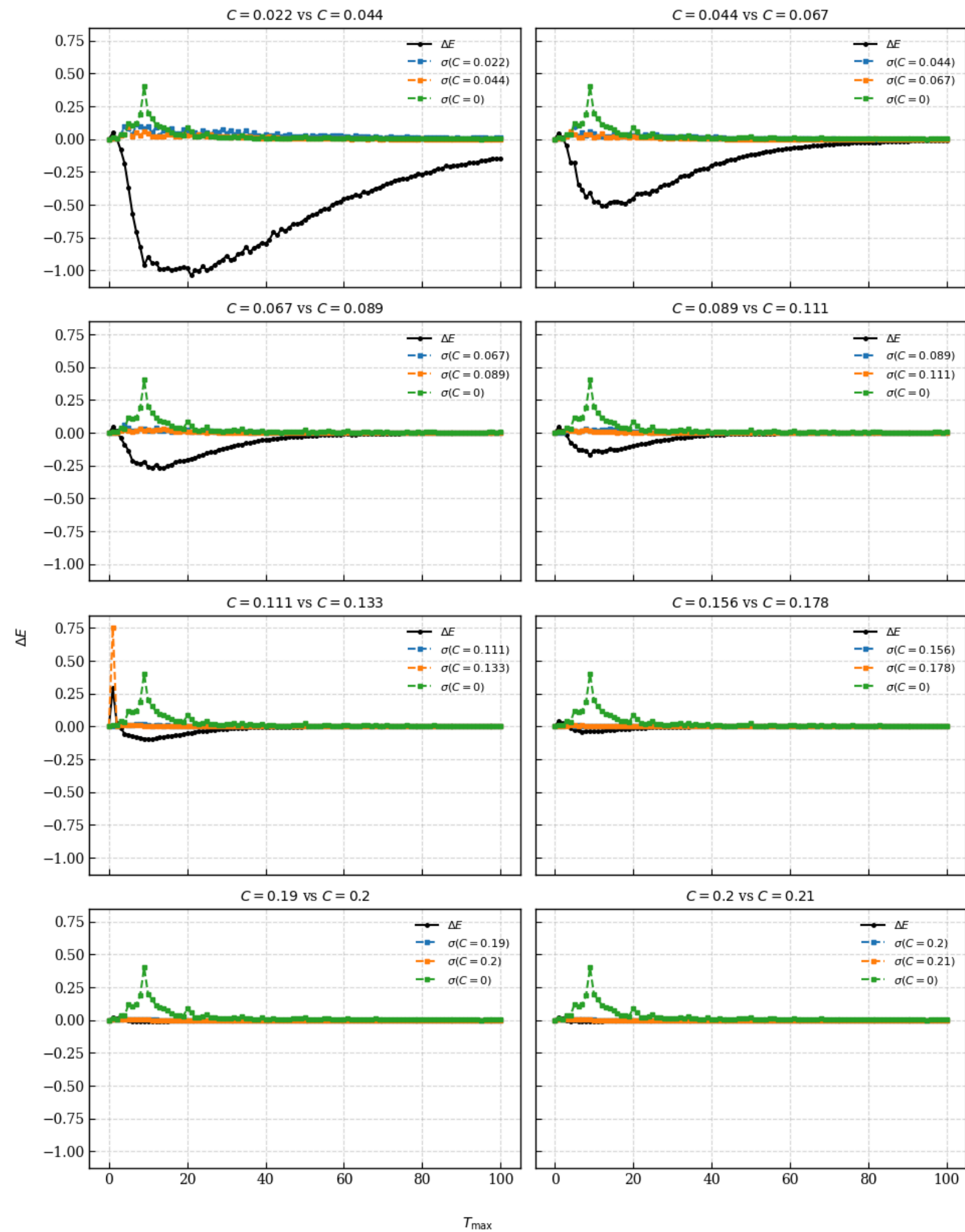
Could you compare between standard deviations and the difference of E_{noisy} and $E_{\text{noiseless}}$?

My guess is that the difference between E_{noisy} and $E_{\text{noiseless}}$ is so small in comparison to the standard deviation of E_{noisy} and $E_{\text{noiseless}}$ for large T and small T .

Such phenomenon cause this type of inconsistency

In this case the difference is the difference between E_{noisy} and $E_{\text{noiseless}}$. Not, the difference between $E_{\text{noisy}}(C1,T)$ and $E_{\text{noisy}}(C2,T)$





Sorry for the confusion about the sign issue.

The important quantity is $\Delta E_{\text{noisy}} / E_{\text{noisy}}$ (or equivalently $E_{\text{noisy}} / \Delta E_{\text{noisy}}$).

If E_{noisy} and ΔE_{noisy} are of the same order, the sign cannot be trusted.

For small T , the issue was that
 $\Delta E_{\text{noisy}} \approx E_{\text{noisy}} - E_{\text{noiseless}}$.

For large T , the problem becomes
 $\Delta E_{\text{noisy}} \approx E_{\text{noisy}}$.

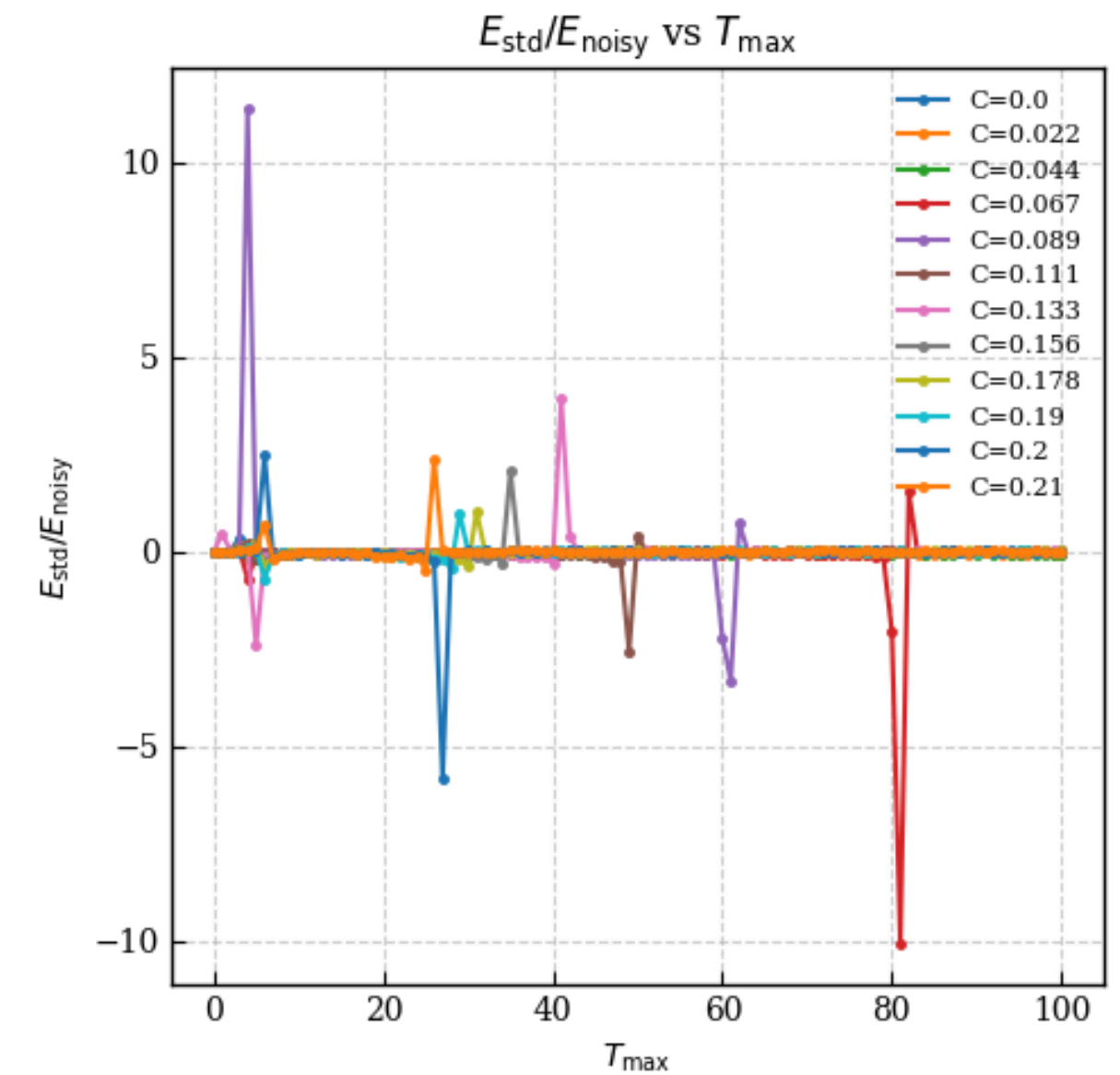
From your graph, $E_{\text{noiseless}} < 0$ for all large T .

Hence, the sign problem occurs when $E_{\text{noisy}} > 0$.

From physical intuition, this should never occur.

Therefore, this is likely a numerical or statistical error.

ΔE_{noisy} is standard deviation of E_{noisy}



In the above document, it is written that the default value of `ftol` is 10^{-6} .

If you did not change this option, the precision of the optimization is 10^{-6} .

The value of E_{noisy} for $C = 0.178$ is much smaller than `ftol`.

Therefore, this is a kind of numerical error.

Intuitively speaking, we cannot trust optimization results that are smaller than 10^{-6} .

If the optimization result is smaller than 10^{-6} , it simply means that the optimization code decides to stop because the value is sufficiently small. In some sense, the algorithm treats it as “zero.”

For large T , let us use only the data for which $E_{\text{noisy}} < -10^{-5}$.

Here, “large T ” means values of T that are larger than the T at which $E_{\text{noisy}}(T)$ takes its minimum.

For each C , there exists a minimum value of $E_{\text{noisy}}(T)$.

For all T larger than this arg-minimum T , we do not use data points where $E_{\text{noisy}} > -10^{-5}$.

After calculating $R(T, C)$, please fit it as

$$R(T, C) = C \cdot F_1(T) + C^2 \cdot F_2(T) + C^3 \cdot F_3(T) + C^4 \cdot F_4(T) + \dots$$

It may be sufficient to perform only first- and second-order fitting:

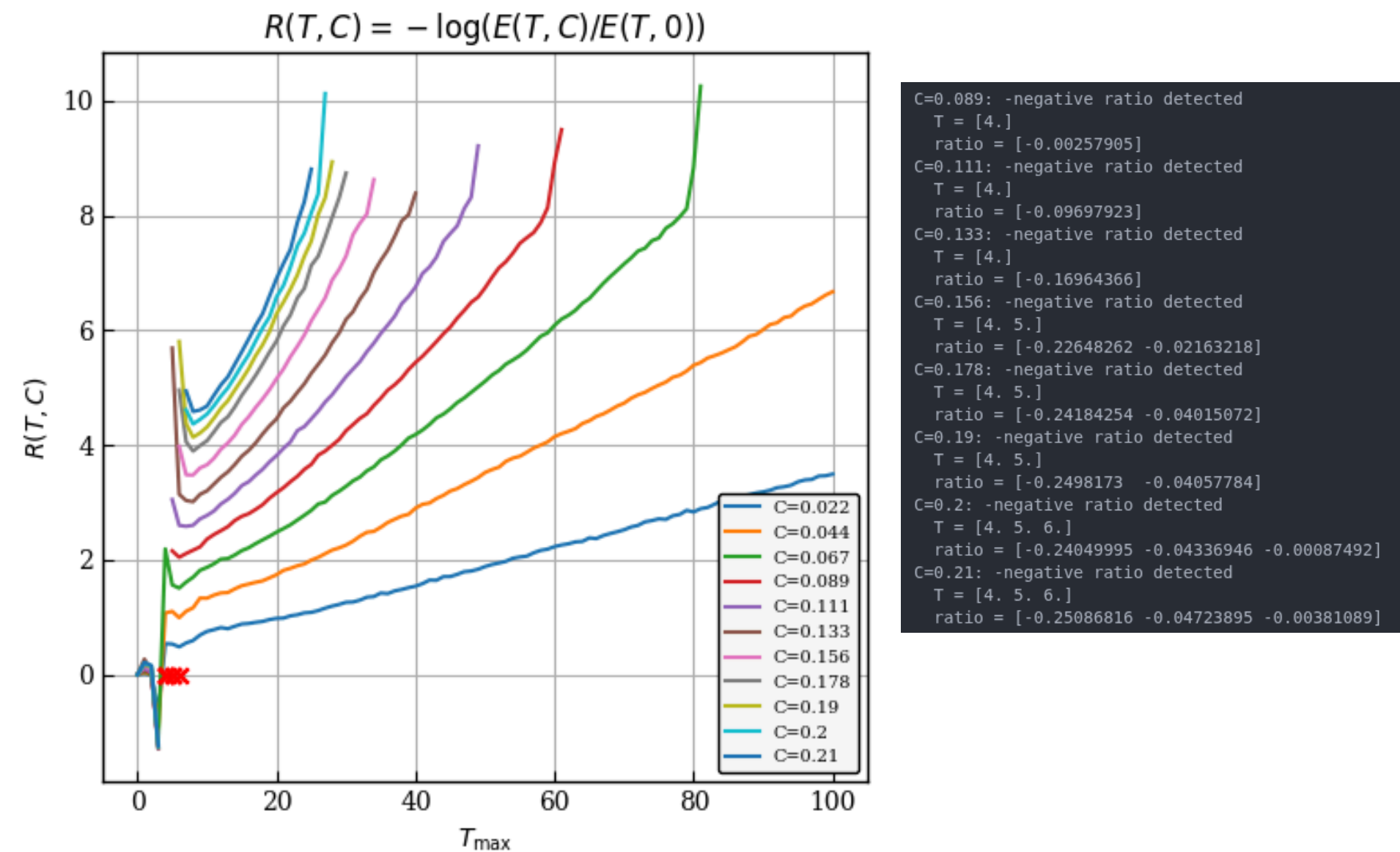
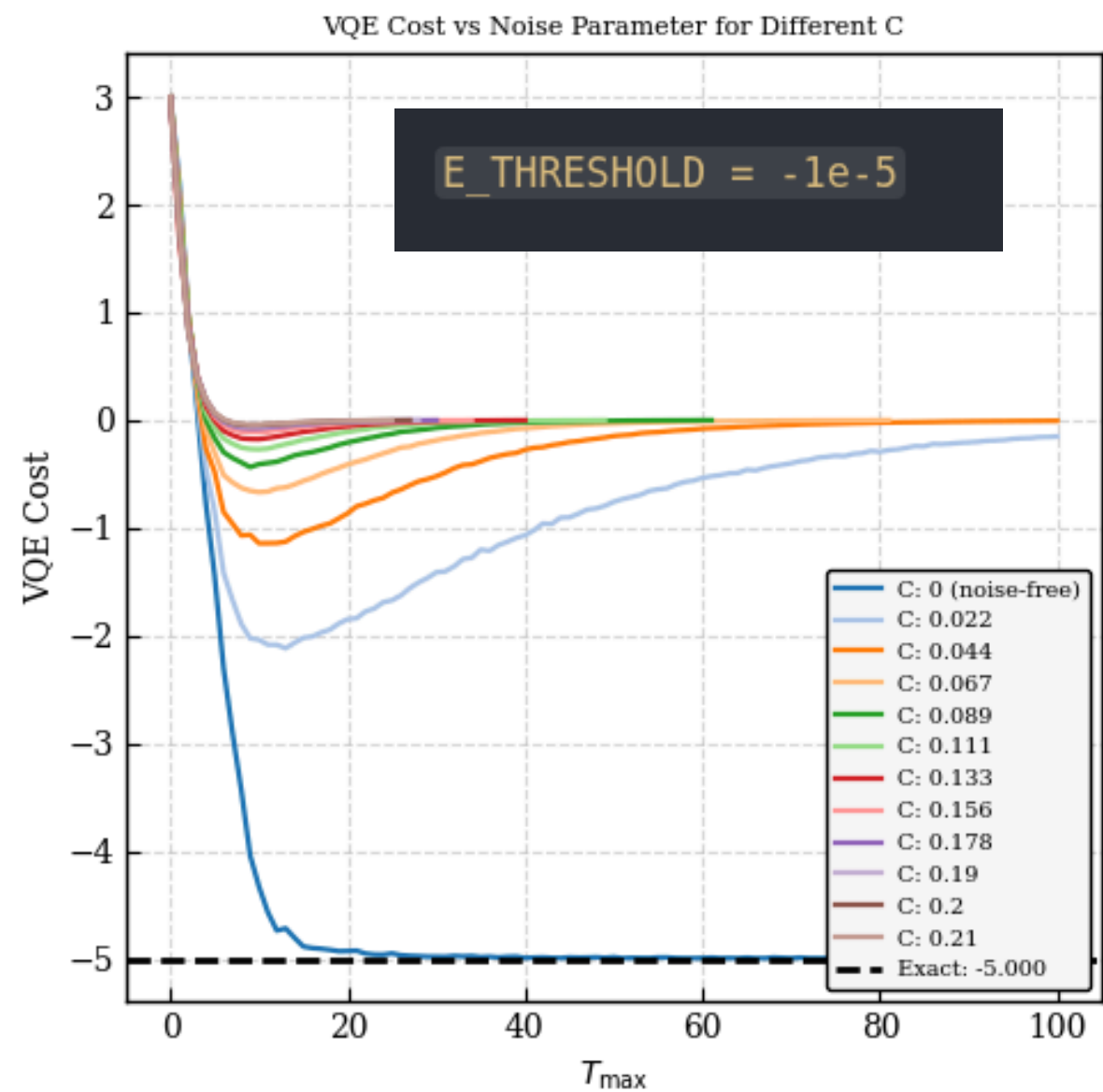
$$R(T, C) = C \cdot F_1(T)$$

or

$$R(T, C) = C \cdot F_1(T) + C^2 \cdot F_2(T).$$

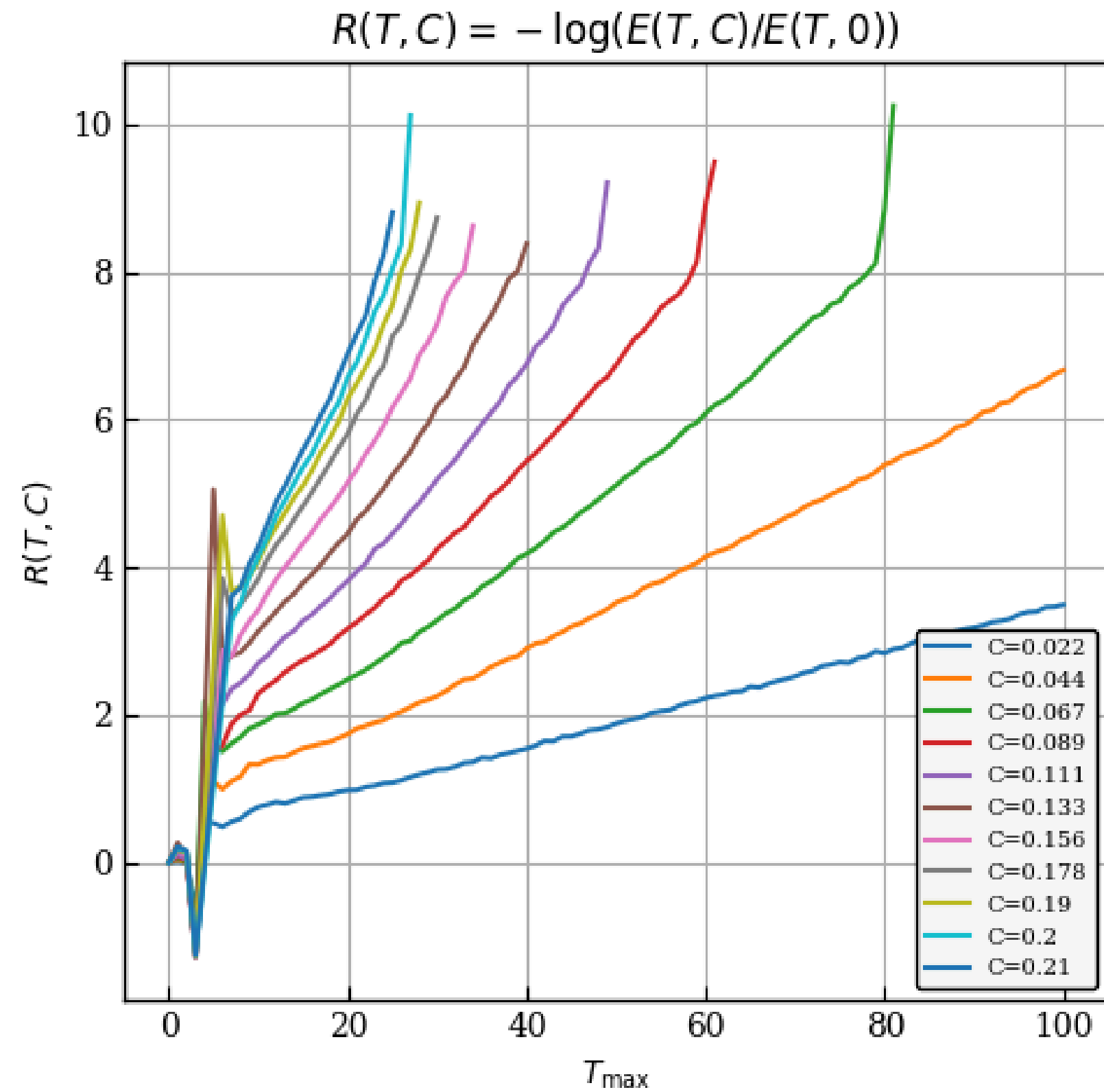
Then, you can further fit $F_1(T)$ and $F_2(T)$.

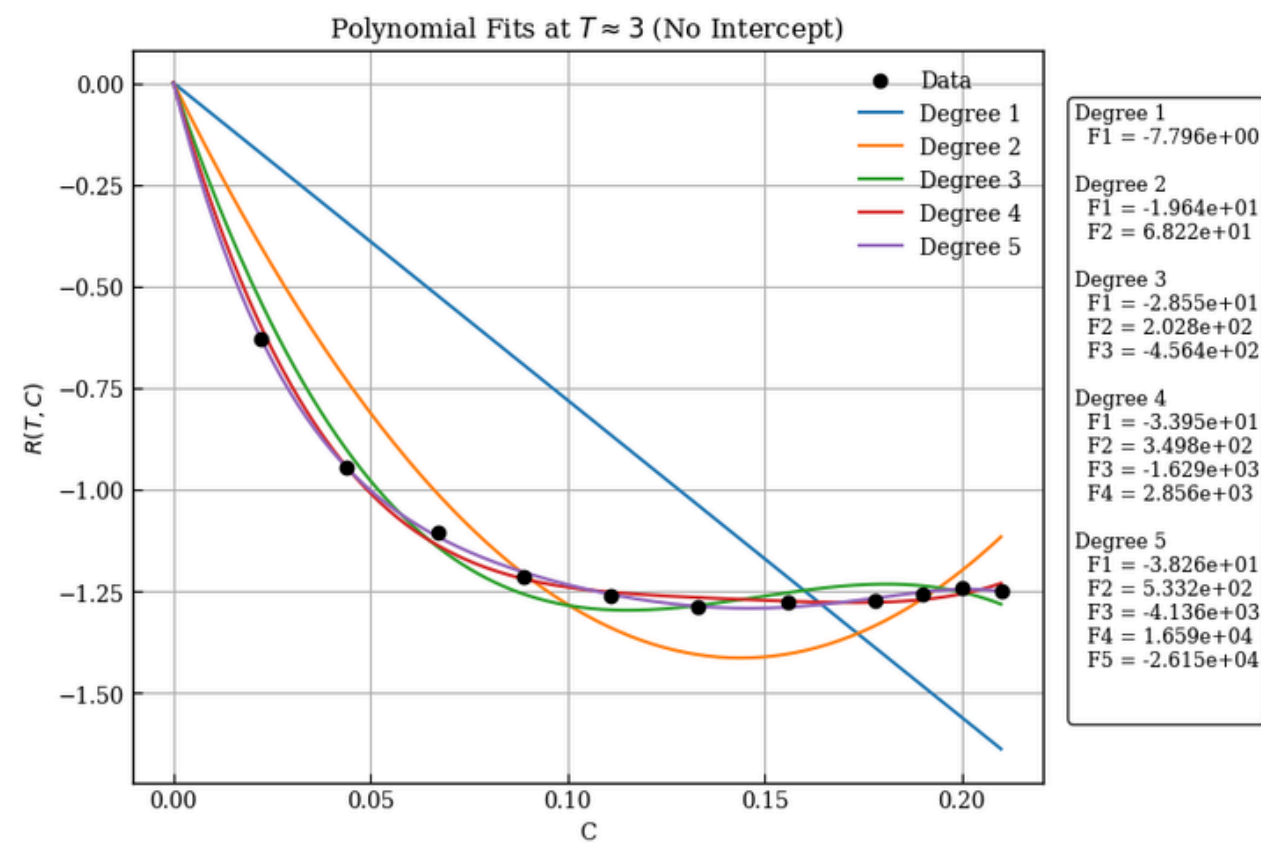
*Scipy SLSQP ftol related



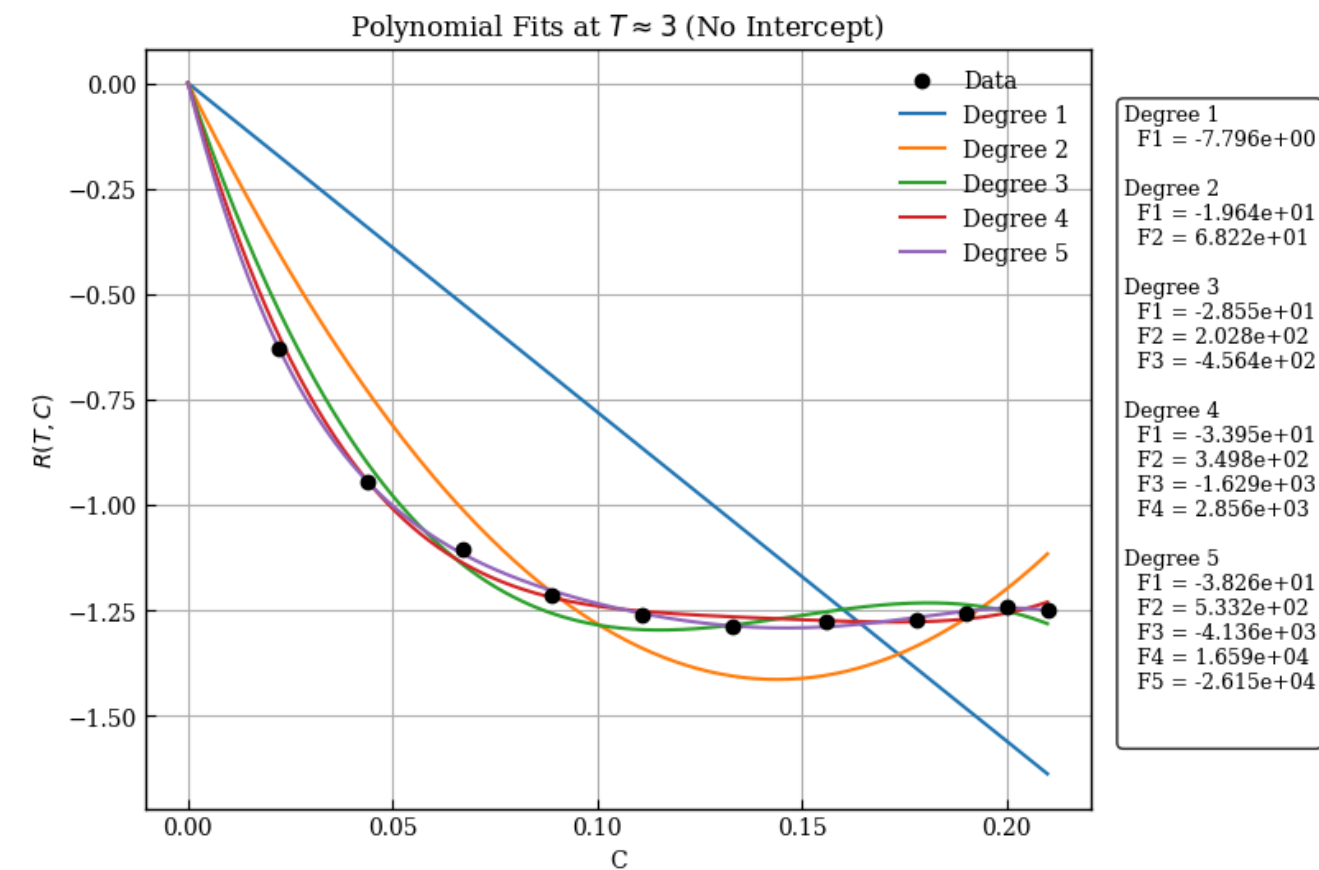
Another filter where E_{noisy} and E_{C0} have same sign.

We need to remove the data with which E_{noisy} and $E_{\text{noiseless}}$ have the different sign. Intuitively, it is due to statistical or numerical error.





using numpy, manually



using Sci-kit learn API, cleaner code

Fitting $R(T, C)$ at various T

✓ Polynomial Fitting of $R(T, C)$

Since $R(T, C = 0) = 0$ for all T ,
the constant term must vanish in the polynomial expansion.

Therefore, we impose $F_0(T) = 0$, and write

$$R(T, C) = C F_1(T) + C^2 F_2(T) + C^3 F_3(T) + C^4 F_4(T) + \cdots$$

This enforces the physical constraint that the noise response vanishes when $C = 0$.

From these data, you can further fit:

$$F_1(T) = F_{10} + F_{11}T + F_{12}T^2$$

$$F_2(T) = F_{20} + F_{21}T + F_{22}T^2$$

and find the coefficients F_{10} , F_{11} , F_{12} , F_{20} , F_{21} , and F_{22} .

I would like you to plot $F_1(T)$ and $F_2(T)$ where:

- **Plot 1:** y-axis is $F_1(T)$ and x-axis is T .
- **Plot 2:** y-axis is $F_2(T)$ and x-axis is T .

This is second degree in C and second degree in T.

We can decrease the number of coefficients if we want.

In general, by increasing the number of coefficients, the fitting may become more difficult.

There may exist some optimal degree.

If we choose second degree in C and second degree in T, there are six coefficients in $R(T,C)$.

This may be too large, but I do not know—let us try.

So, we should always keep in mind that we can decrease the number of coefficients.

(1) First, we fit $R(T,C)$ for different values of T using a polynomial of degree 2 in C.
This gives us $F_1(T)$ and $F_2(T)$ for various T.

(2) Then, we further fit $F_1(T)$ and $F_2(T)$ as functions of T. Possible choices are:

$$F_1(T) = F_{10} + F_{11} T + F_{12} T^2$$

$$F_2(T) = F_{20} + F_{21} T + F_{22} T^2$$

or

$$F_1(T) = F_{10} + F_{11} T$$

$$F_2(T) = F_{20} + F_{21} T + F_{22} T^2$$

or

$$F_1(T) = F_{10} + F_{11} T + F_{12} T^2$$

$$F_2(T) = F_{20} + F_{21} T$$

or

$$F_1(T) = F_{10} + F_{11} T$$

$$F_2(T) = F_{20} + F_{21} T$$

or

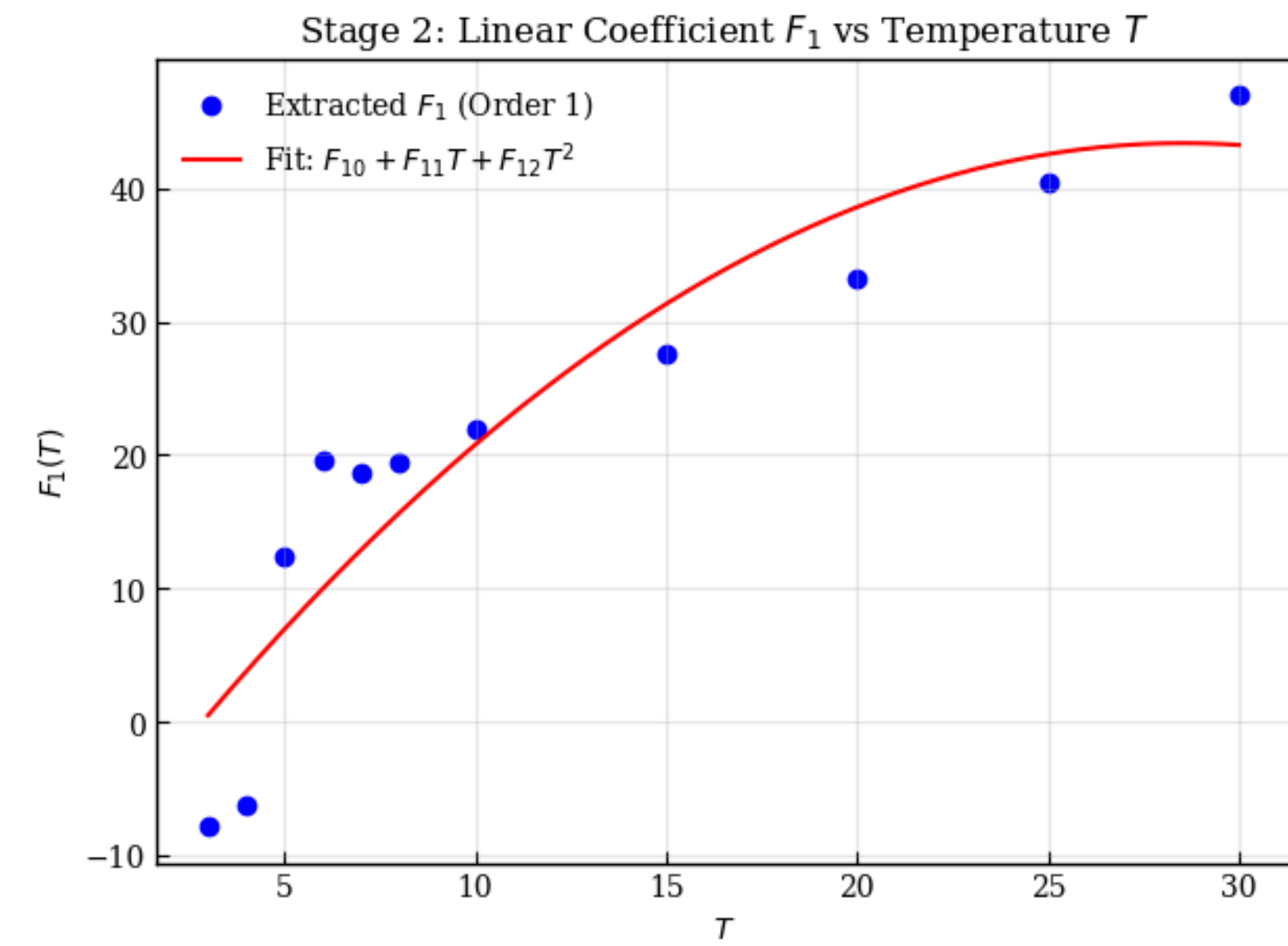
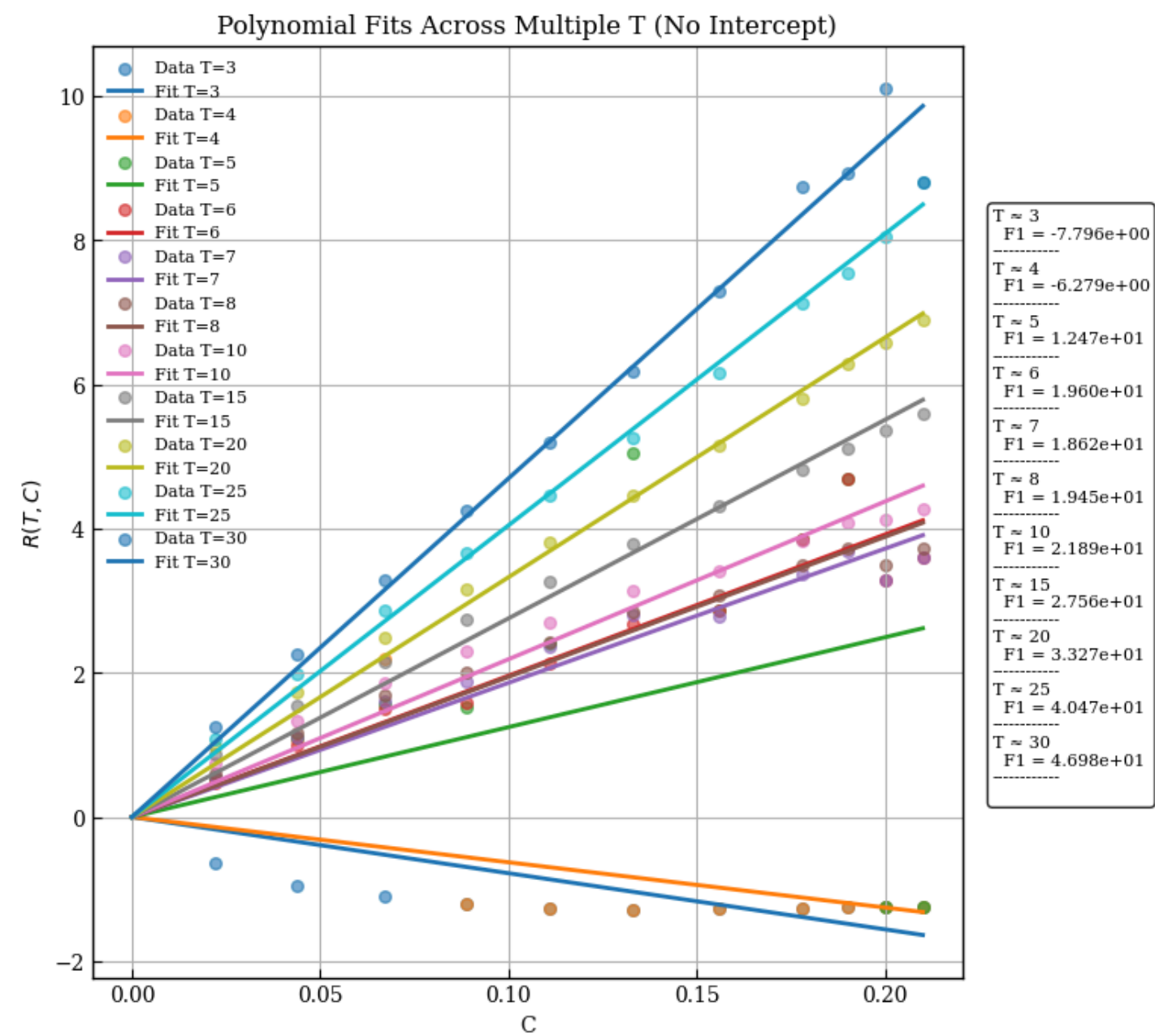
$$F_1(T) = F_{10}$$

$$F_2(T) = F_{20} + F_{21} T$$

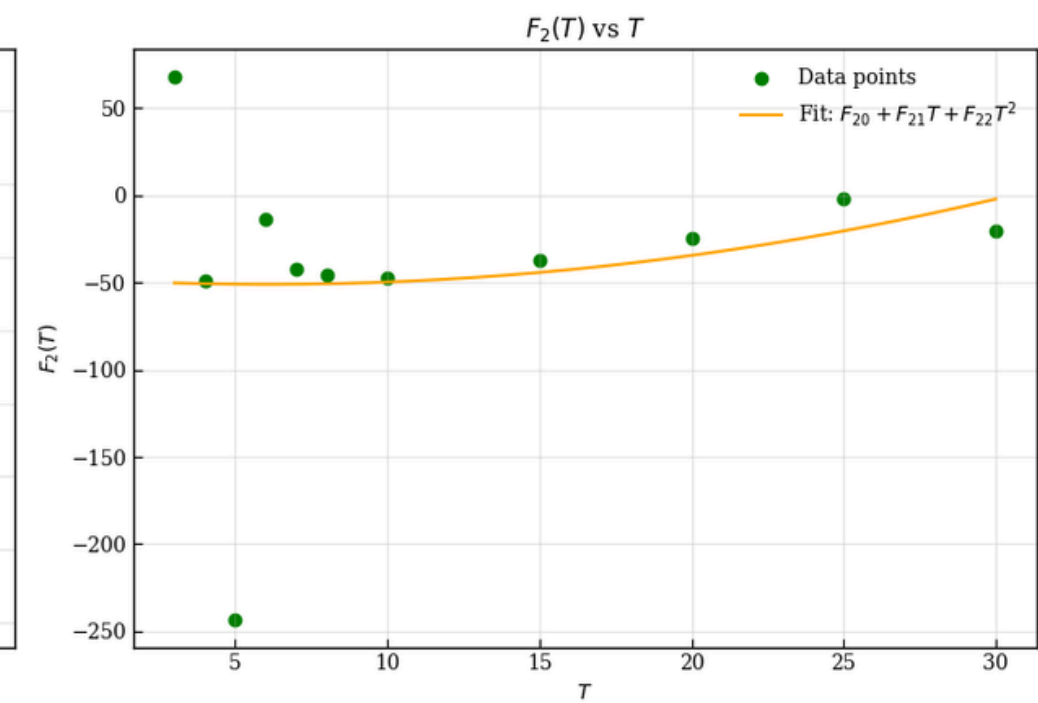
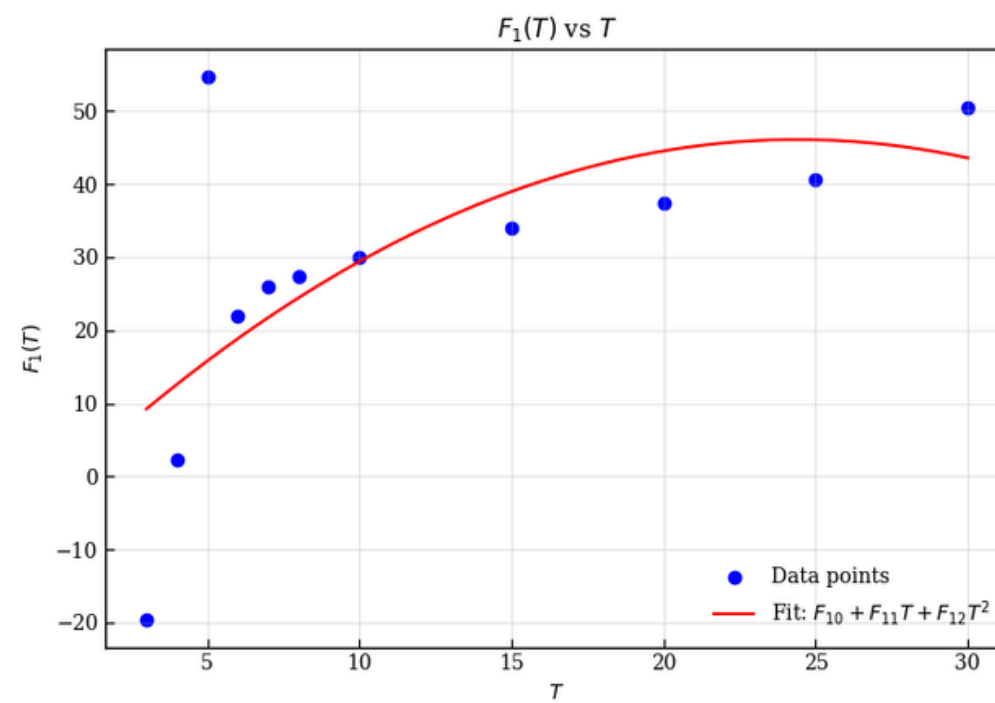
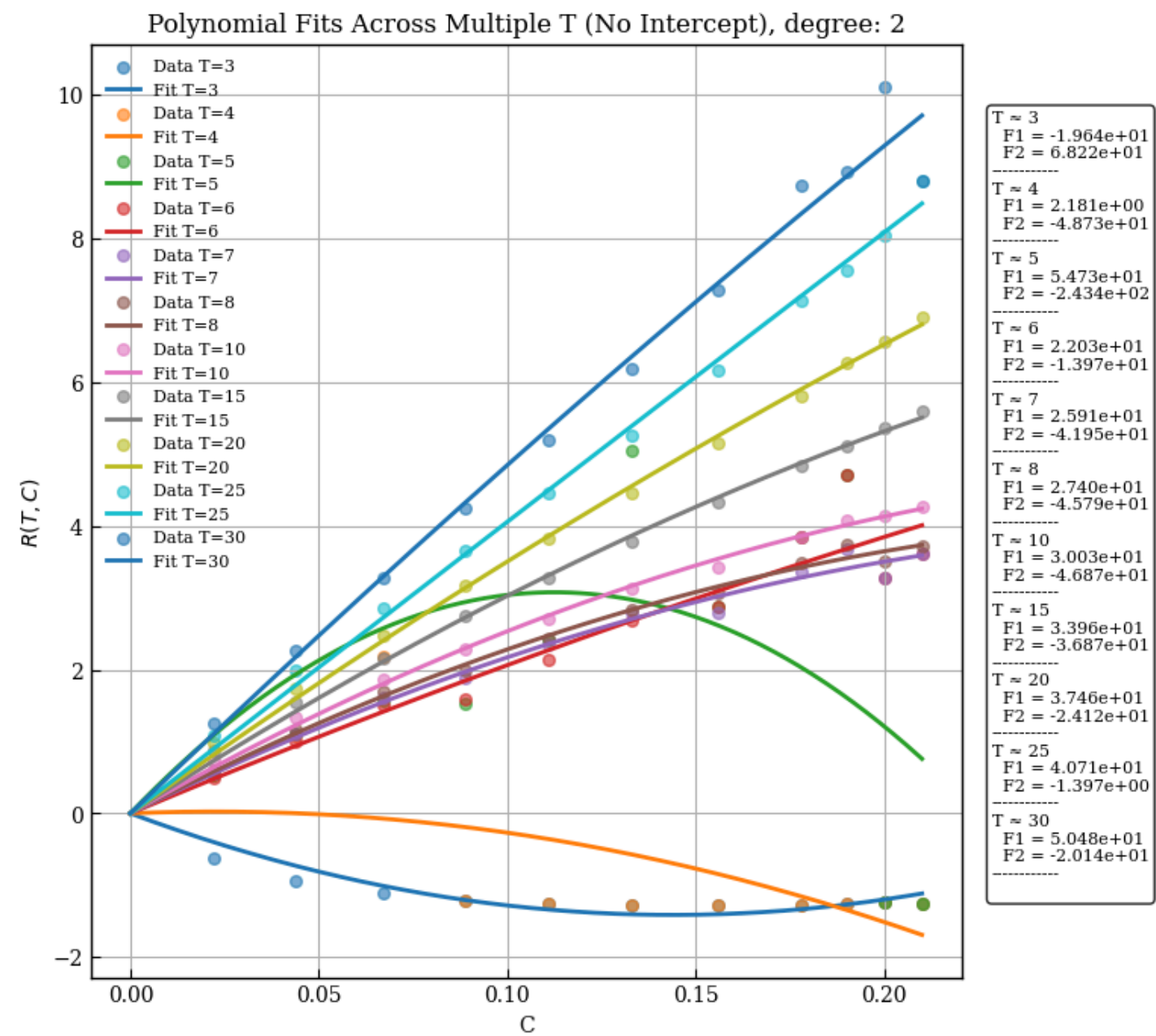
or

$$F_1(T) = F_{10} + F_{11} T$$

$$F_2(T) = F_{20}$$



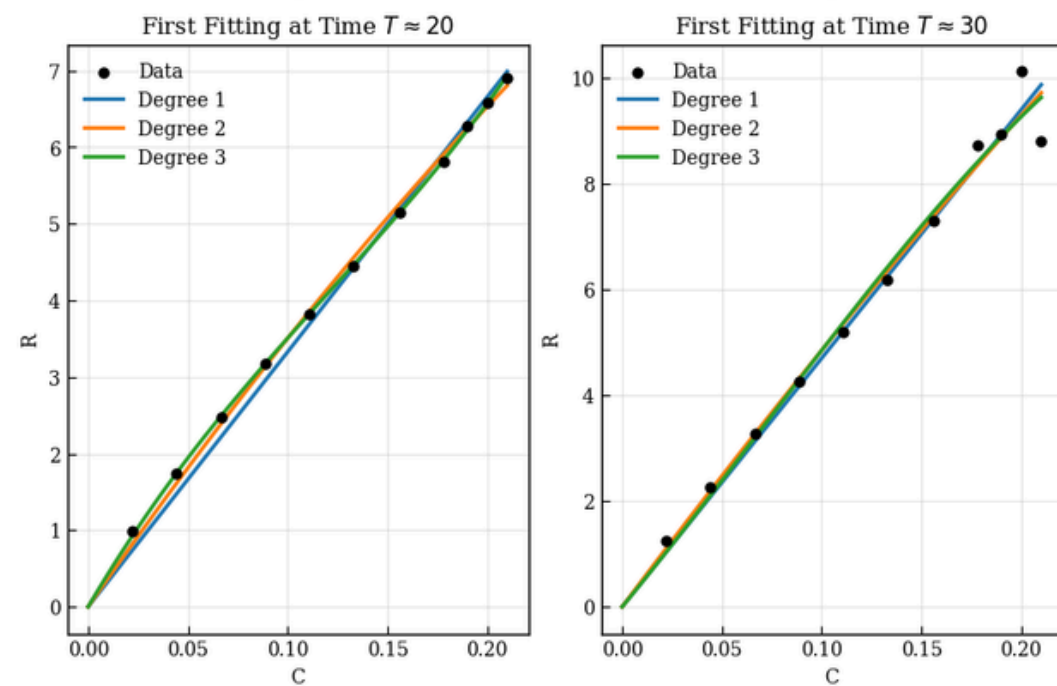
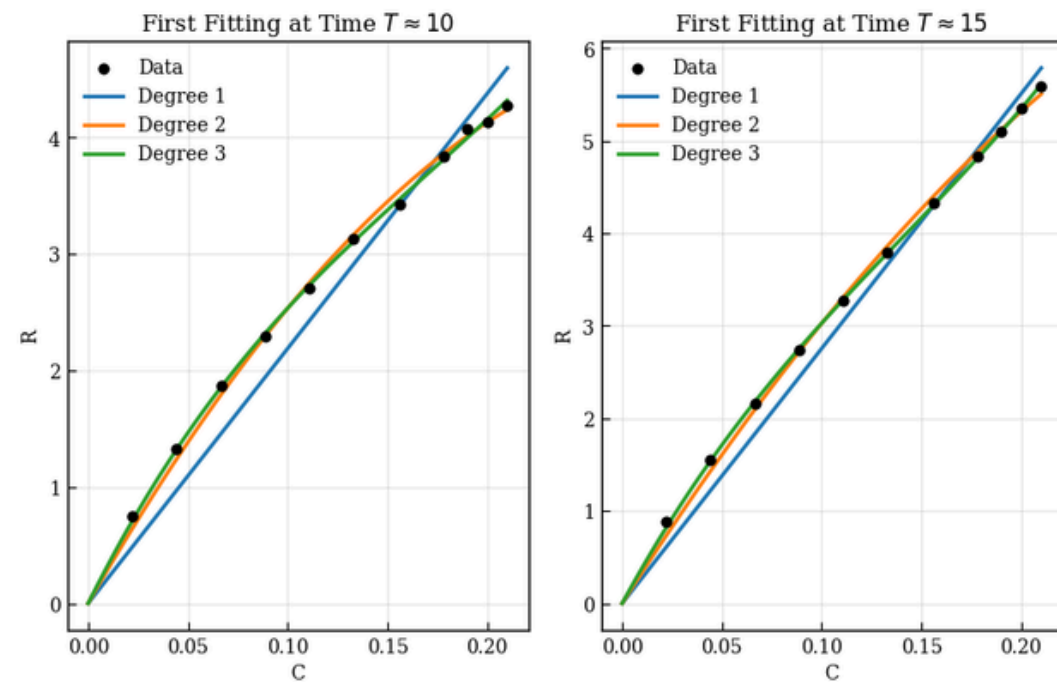
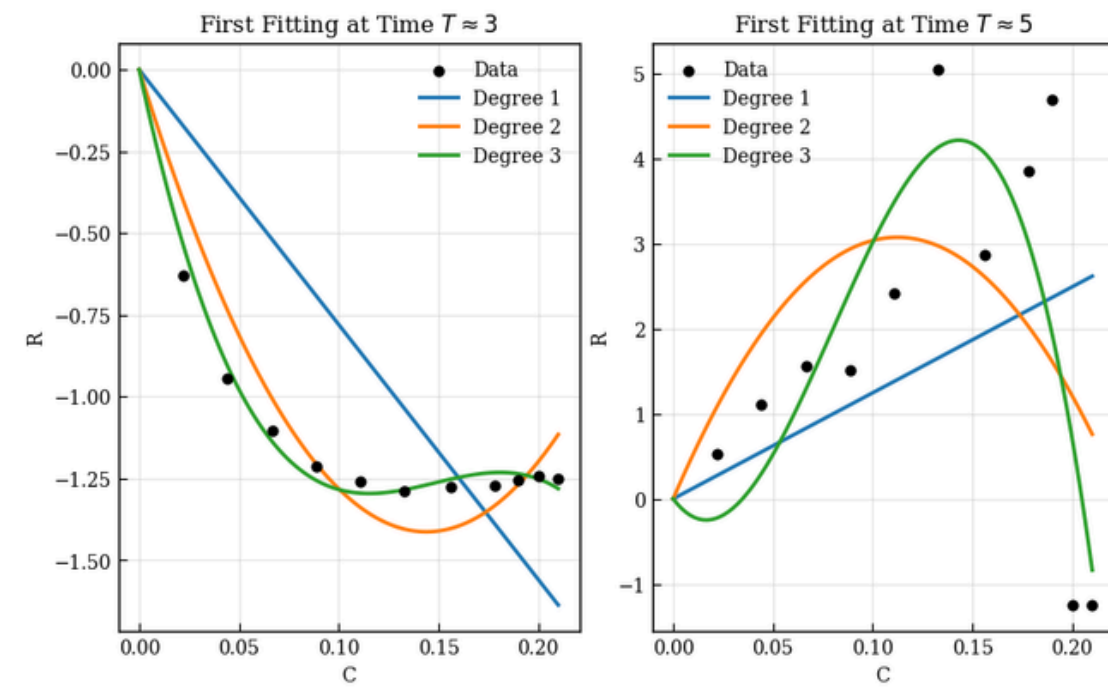
Final Coefficients for $F_1(T)$: $F_{10}=-1.0149e+01$, $F_{11}=3.7504e+00$, $F_{12}=-6.5708e-02$



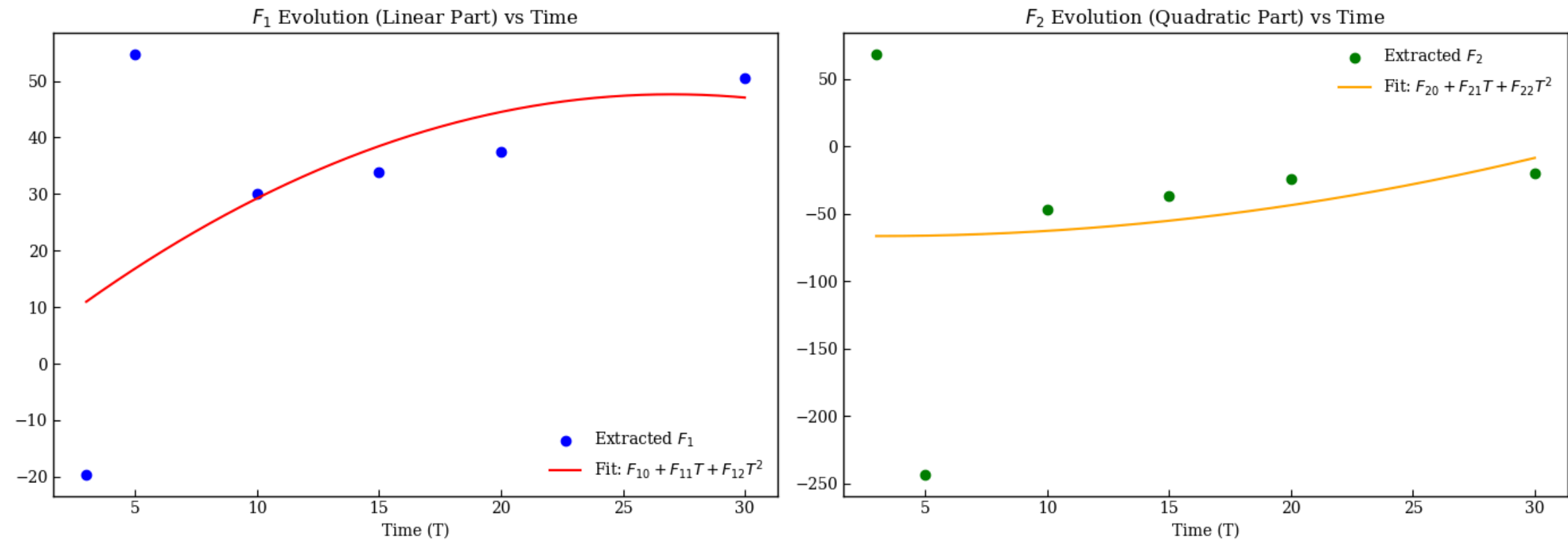
--- Resulting Coefficients ---

F10: -1.840274e+00, F11: 3.930375e+00, F12: -8.054008e-02

F20: -4.785776e+01, F21: -1.036435e+00, F22: 8.543616e-02



Could you give me a polynomial plots of $R(C,T)$ w.r.t. C for fixed T , for several other T ?

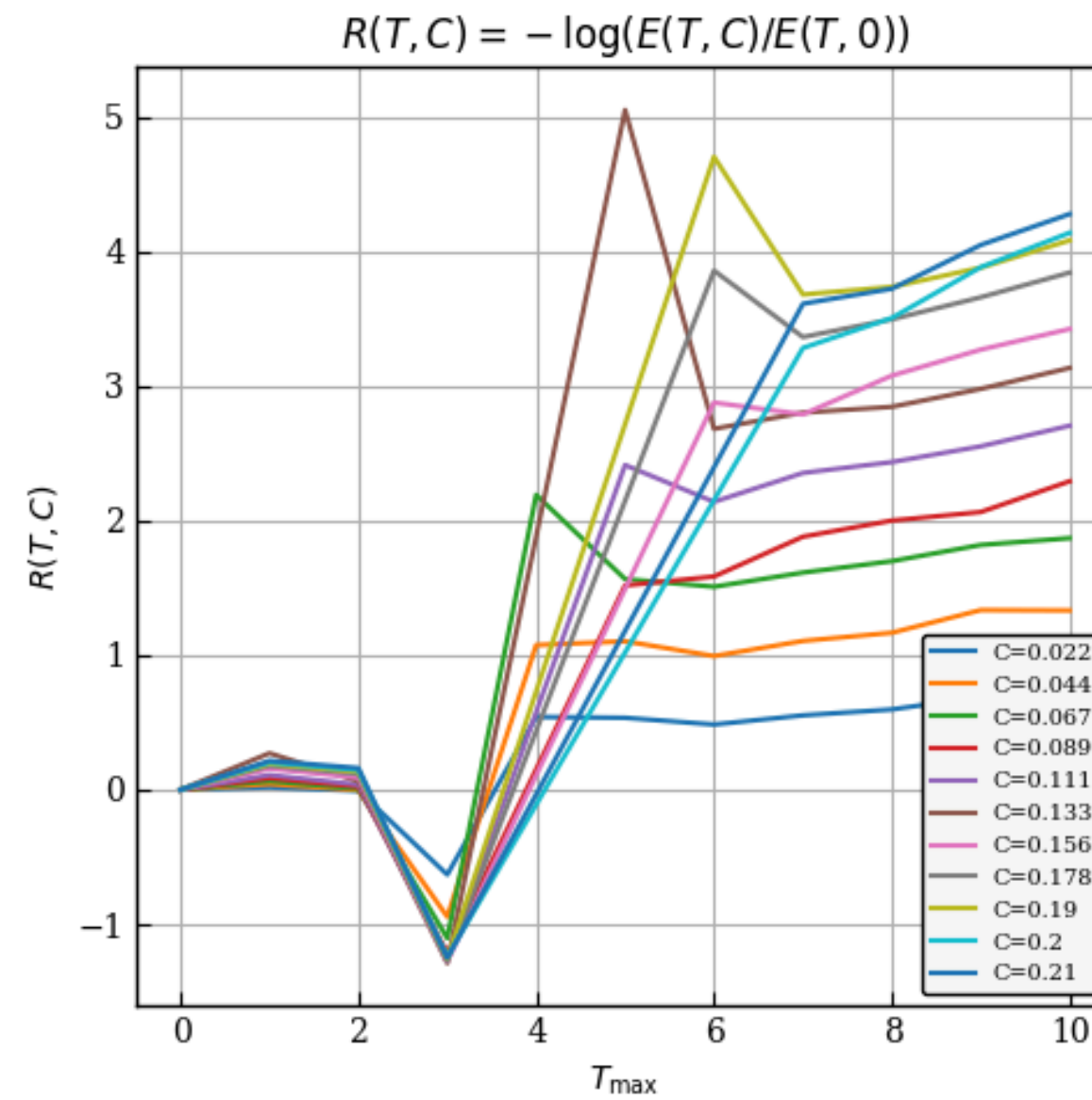


```

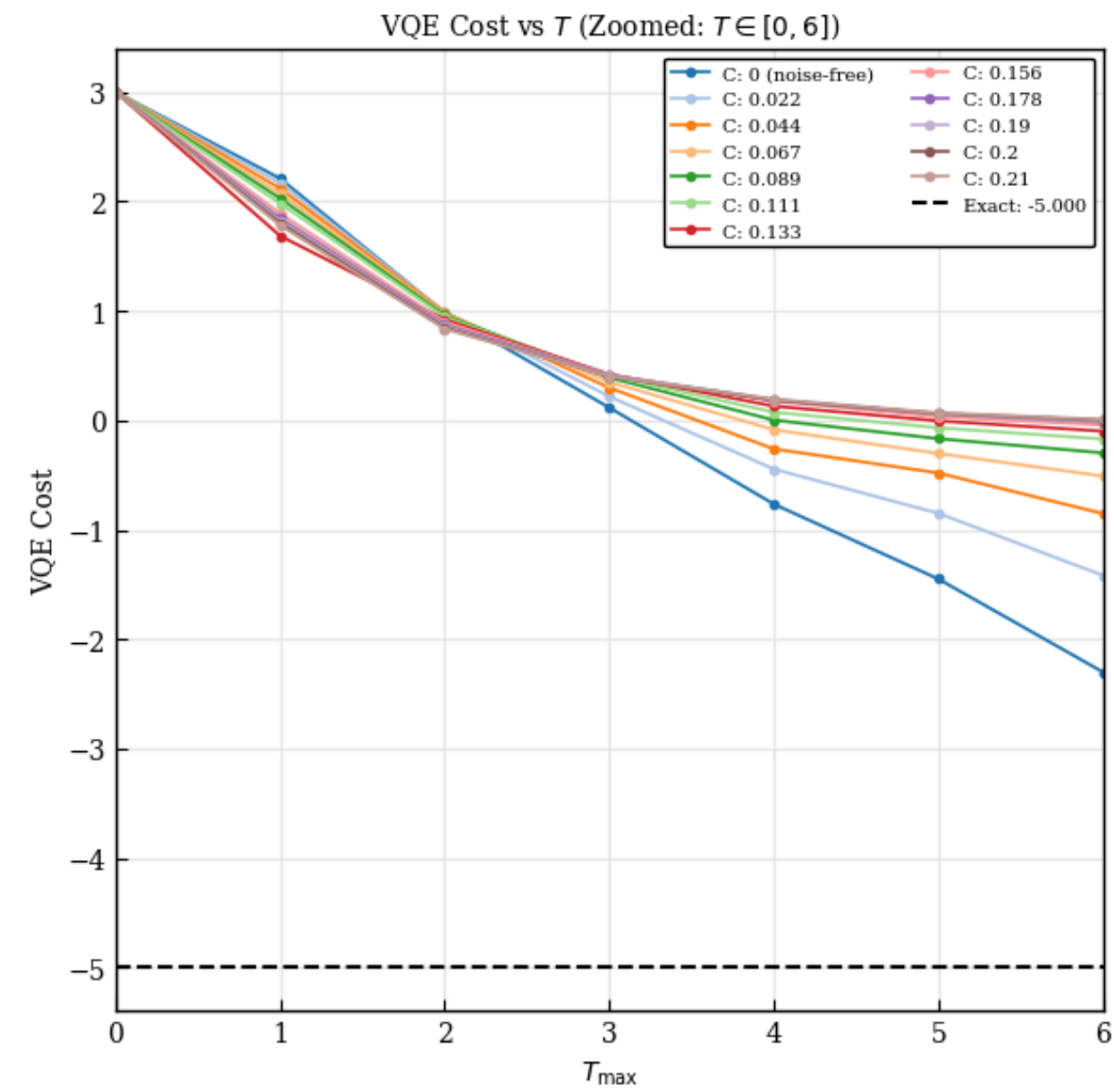
--- Master Coefficients ---
F1(T) = 1.2146e+00 + 3.4382e+00*T + -6.3670e-02*T^2
F2(T) = -6.5828e+01 + -4.9191e-01*T + 7.9924e-02*T^2

```

Could you expand this graph for the range T in $[0,10]$



Could you expand this graph for T in $[0, 6]$



The behavior of polynomial fitting seems very sensitive with respect to T.

Could you calculate the error bar (standard deviation) for R(T,C)?

The standard deviation of R(T,C) may be calculated as follows:

$$\Delta R(T,C) = \sqrt{(\Delta E_{\text{noisy}}(T,C) / E_{\text{noisy}}(T,C))^2 + (\Delta E_{\text{noiseless}}(T,C) / E_{\text{noiseless}}(T,C))^2}$$

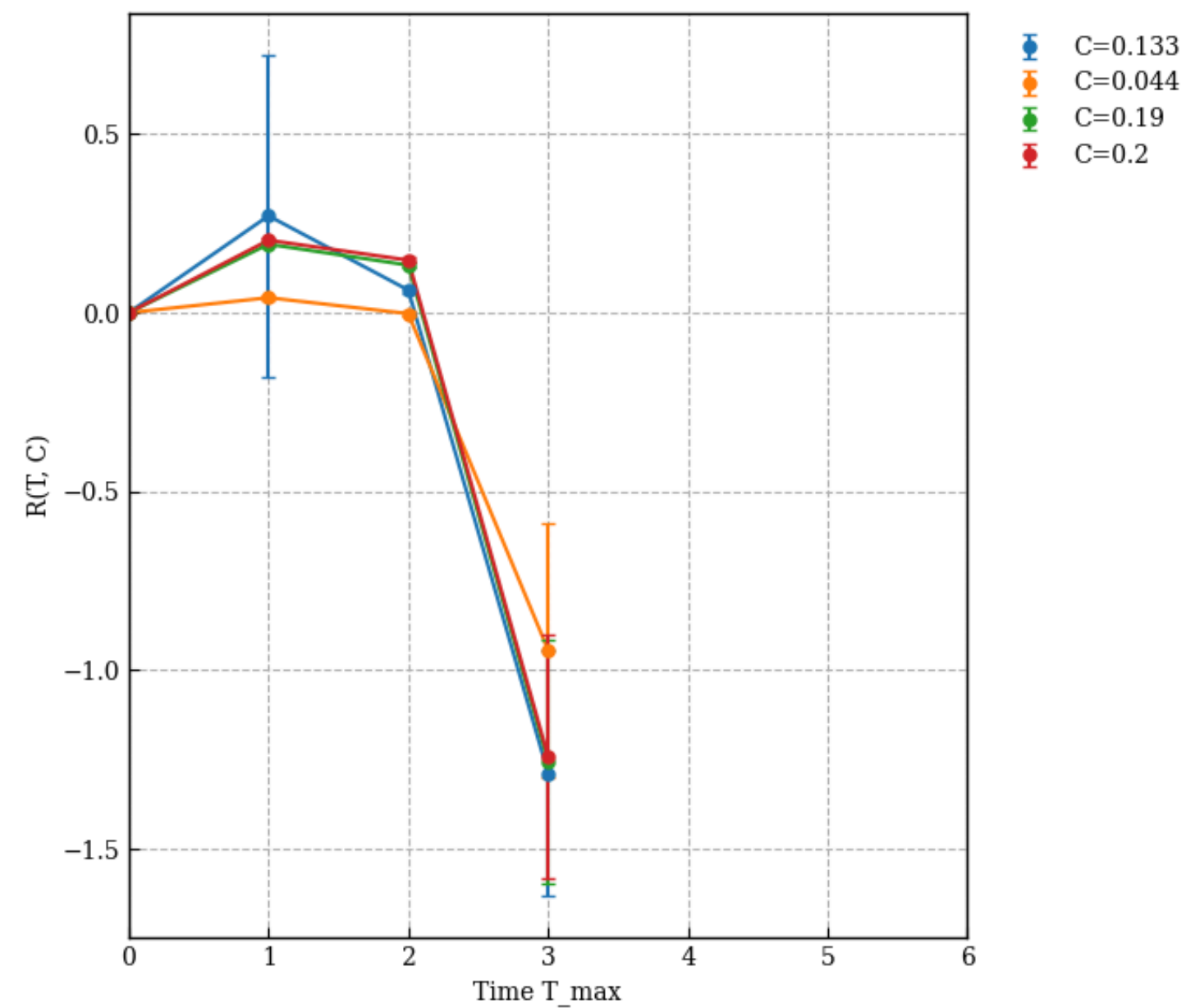
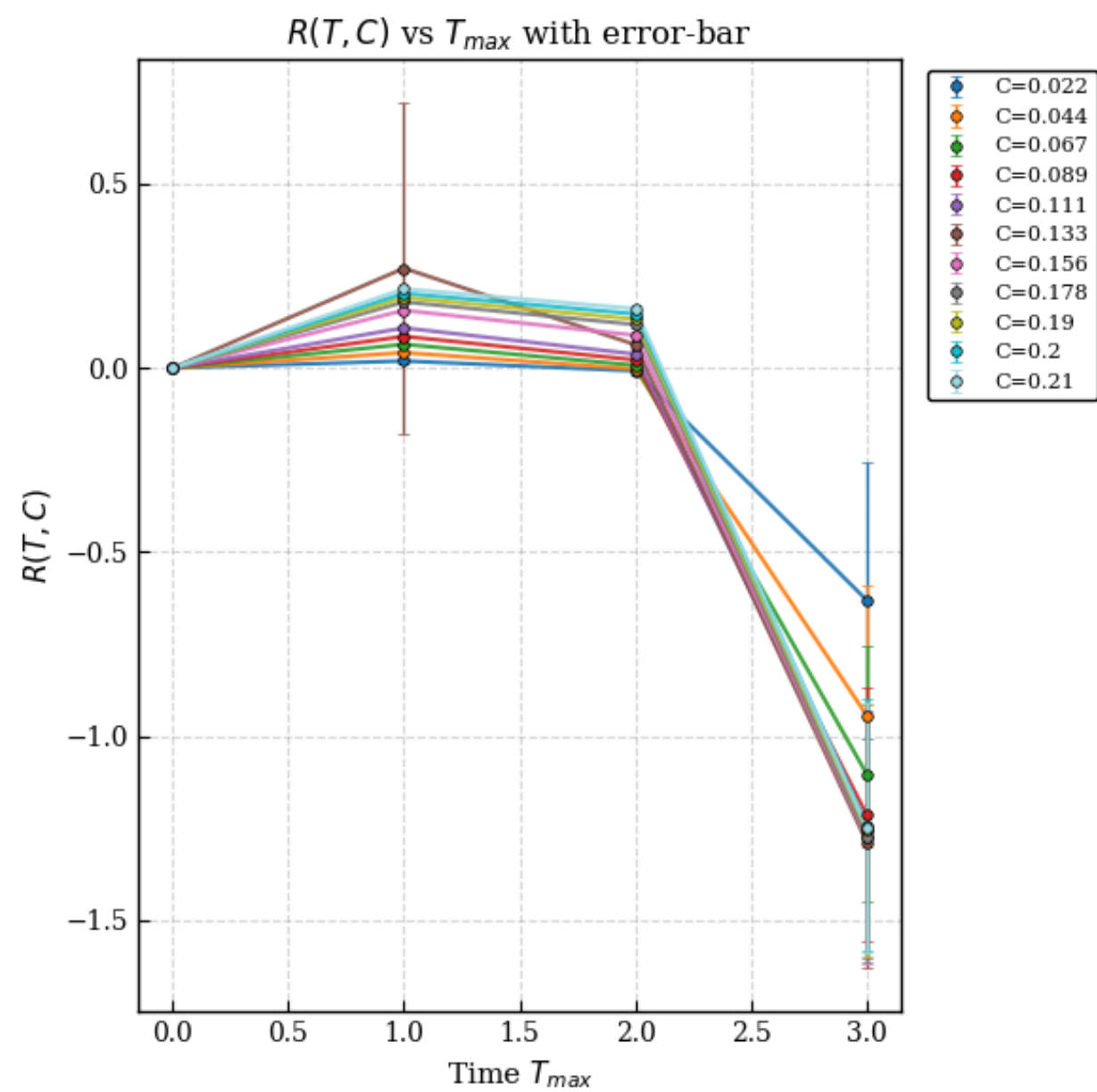
I am slightly wondering about the reliability of data for T = 3 and T = 5.

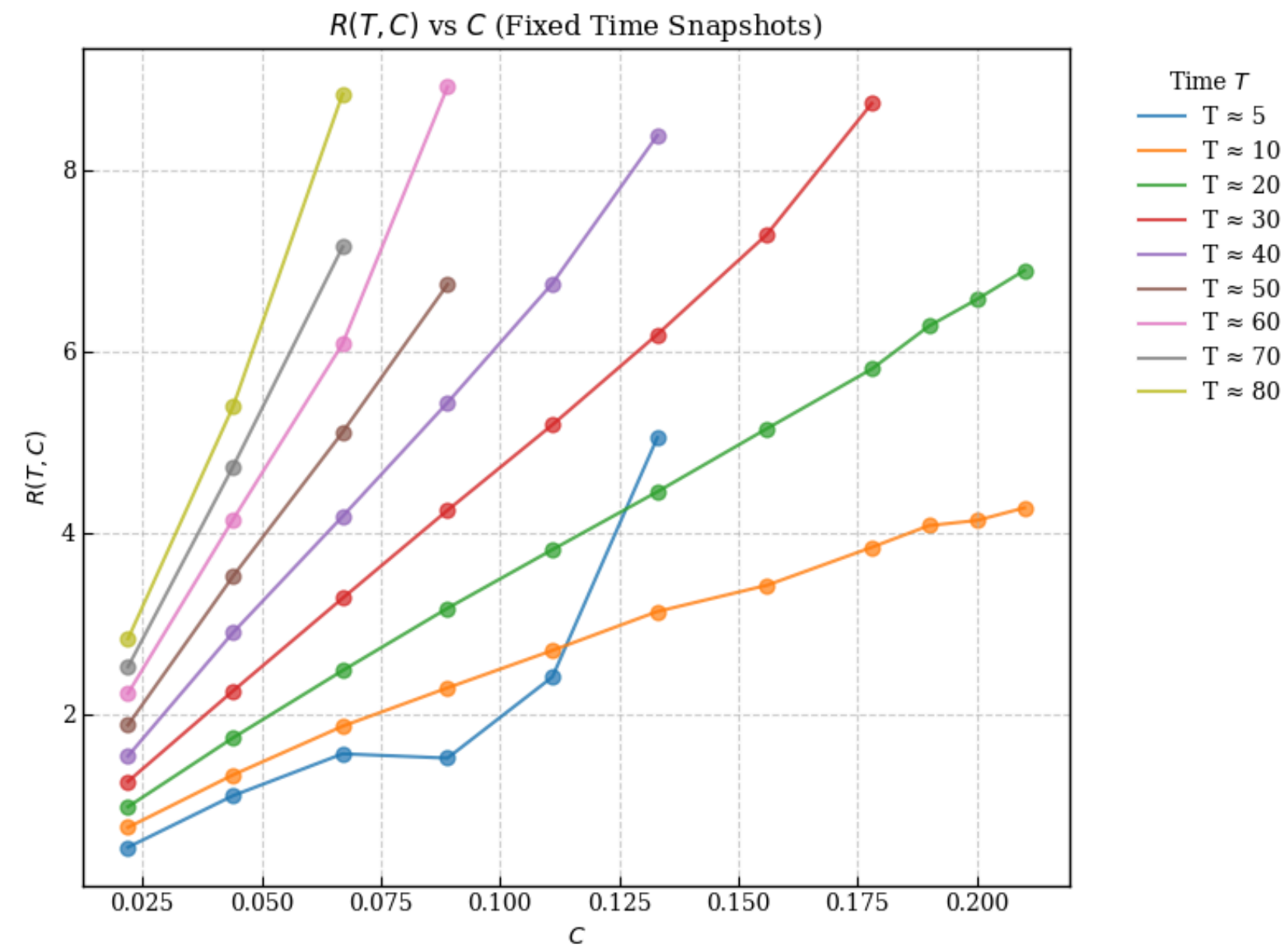
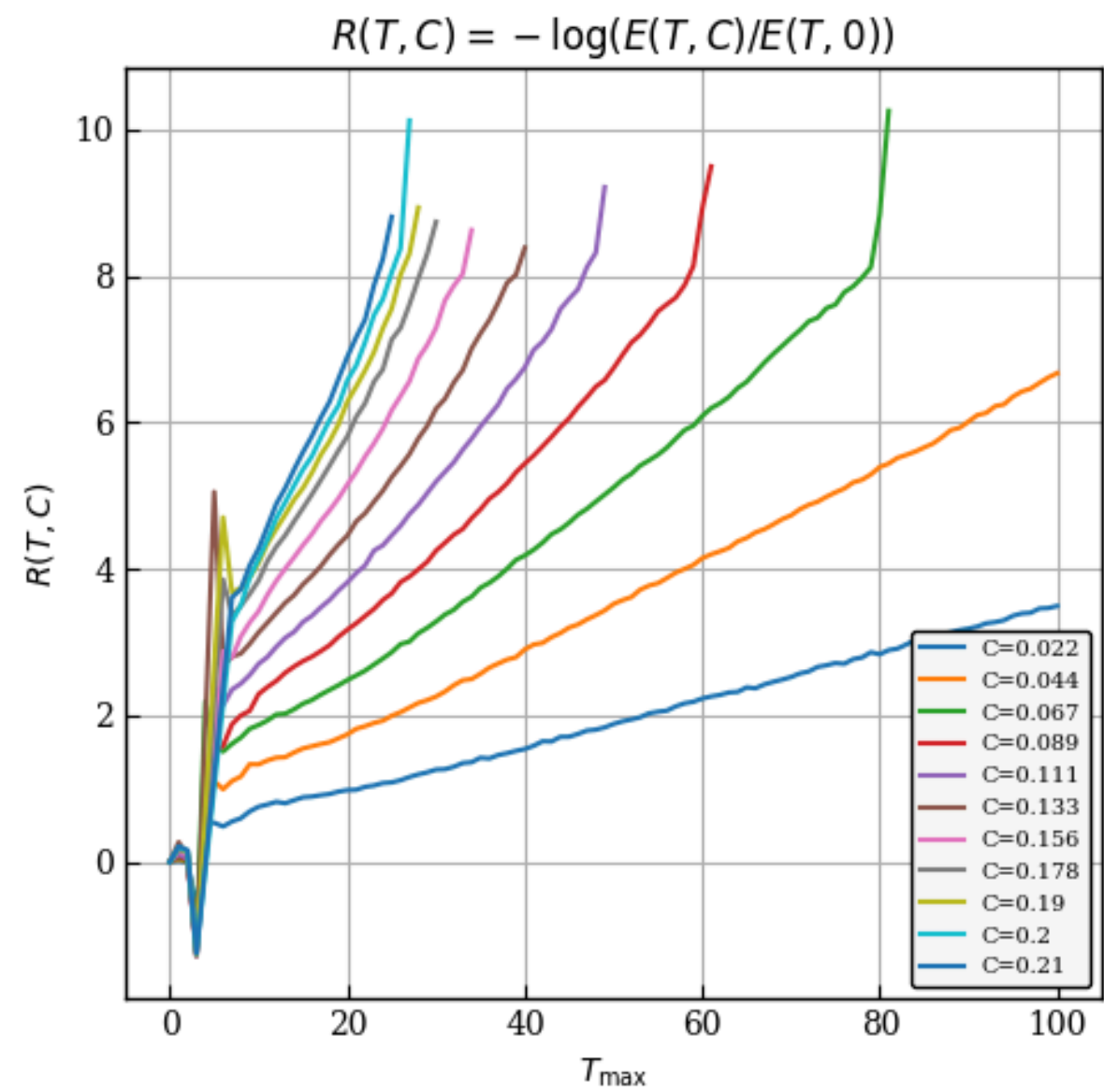
Note: In the above formula, $\Delta E_{\text{noisy}}(T,C)$ and $\Delta E_{\text{noiseless}}(T,C)$ are the standard deviations of E_{noisy} and $E_{\text{noiseless}}$, respectively. Please note that the sum inside the square root:

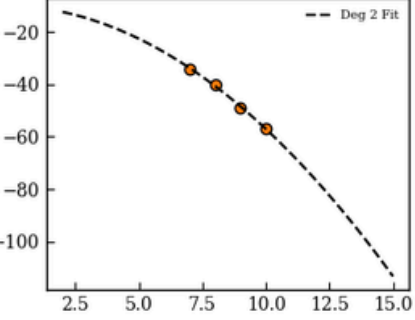
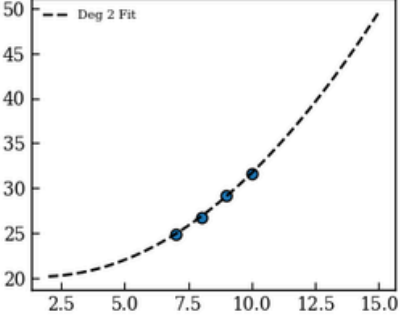
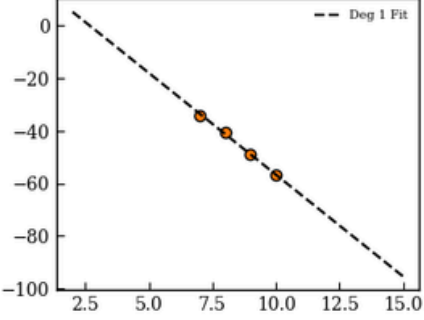
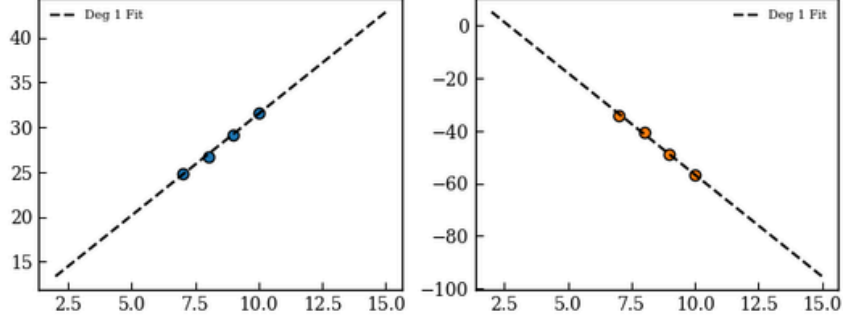
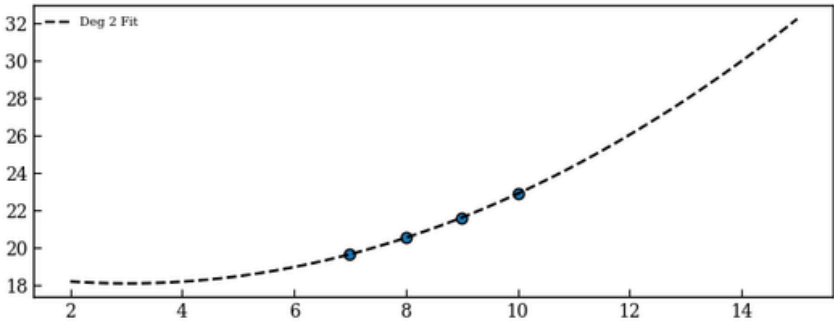
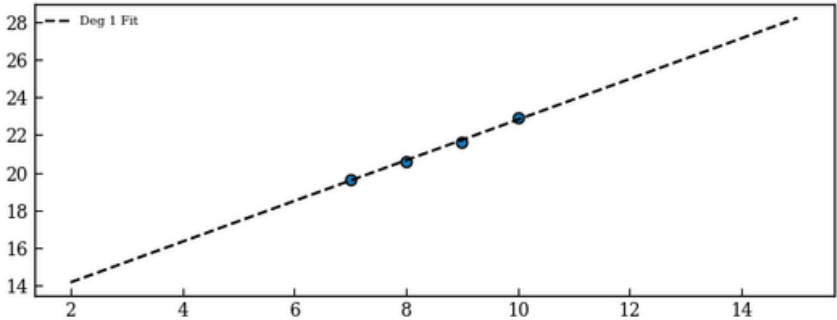
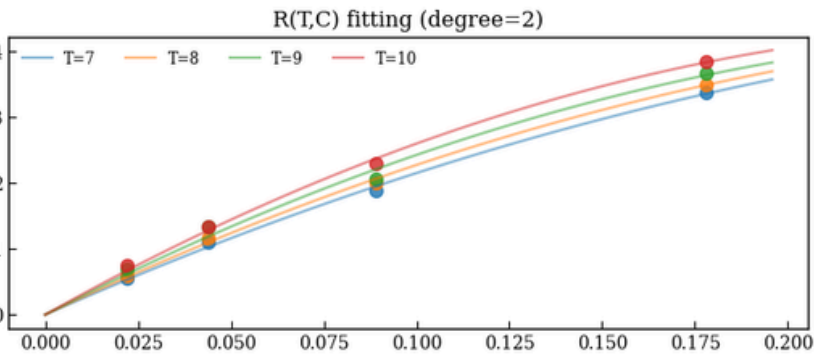
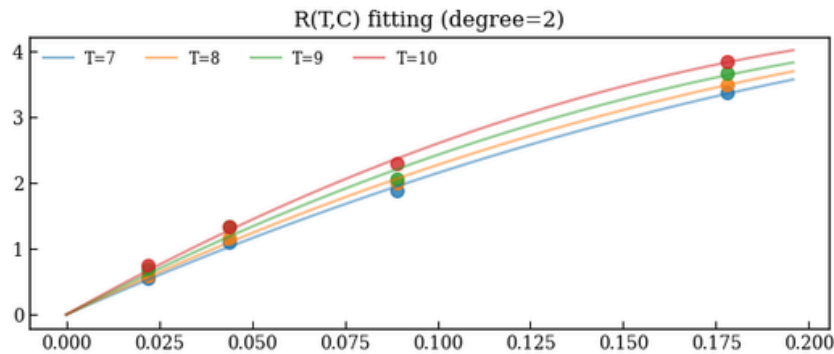
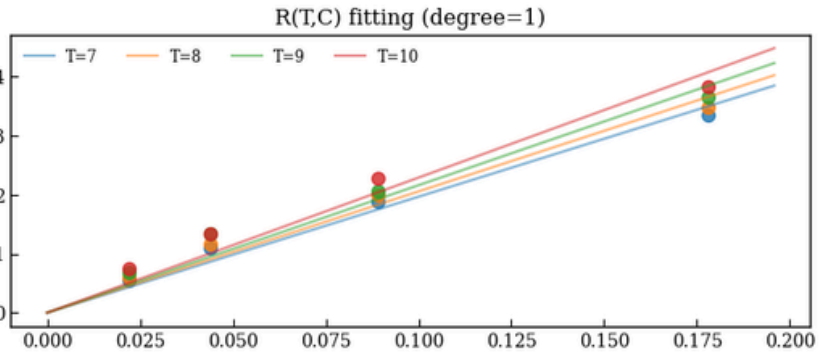
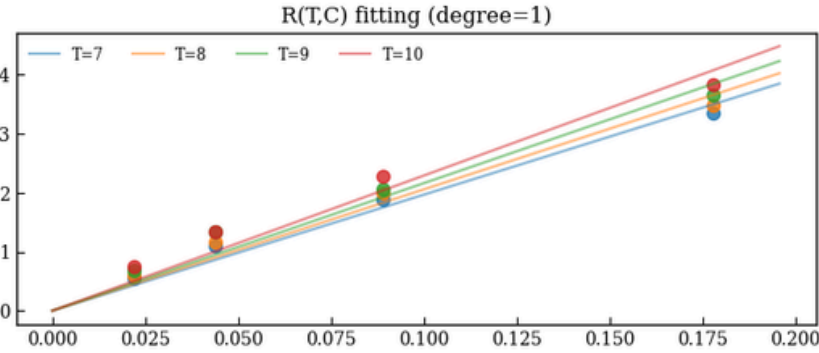
$$(\Delta E_{\text{noisy}}(T,C)/E_{\text{noisy}}(T,C))^2 + (\Delta E_{\text{noiseless}}(T,C)/E_{\text{noiseless}}(T,C))^2$$

is entirely under the $\sqrt{}$.

$$\Delta R(T, C) = \sqrt{\left(\frac{\Delta E_{\text{noisy}}(T, C)}{E_{\text{noisy}}(T, C)}\right)^2 + \left(\frac{\Delta E_{\text{noiseless}}(T, C)}{E_{\text{noiseless}}(T, C)}\right)^2}$$





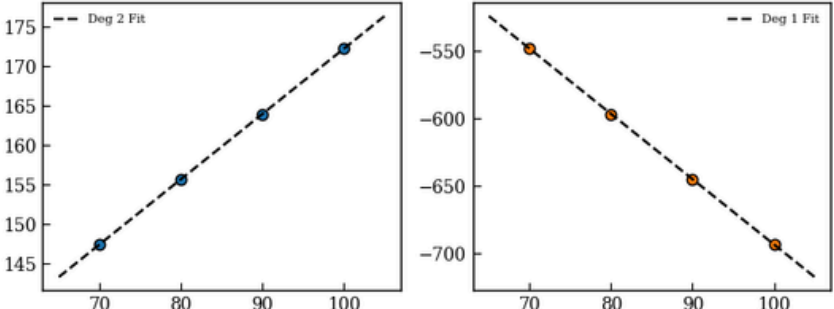
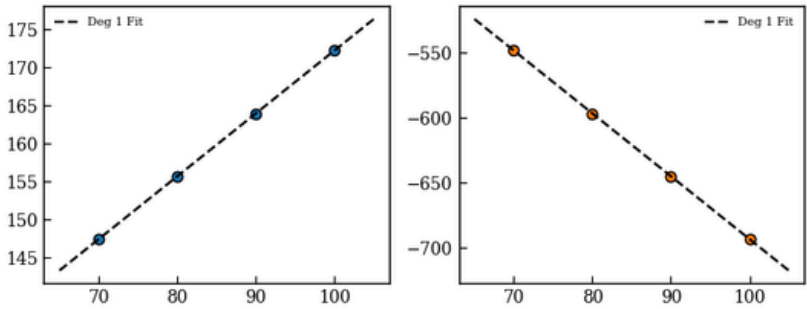
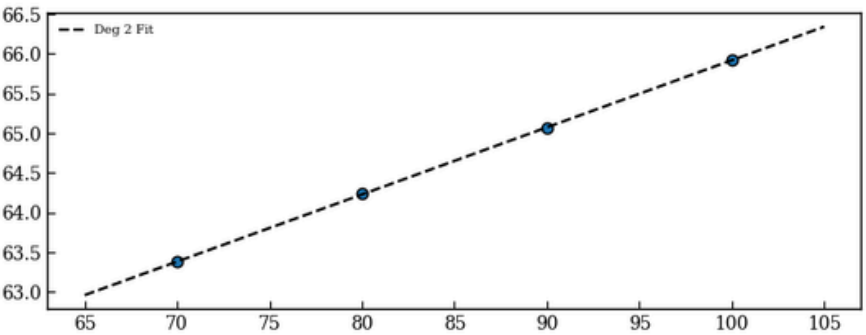
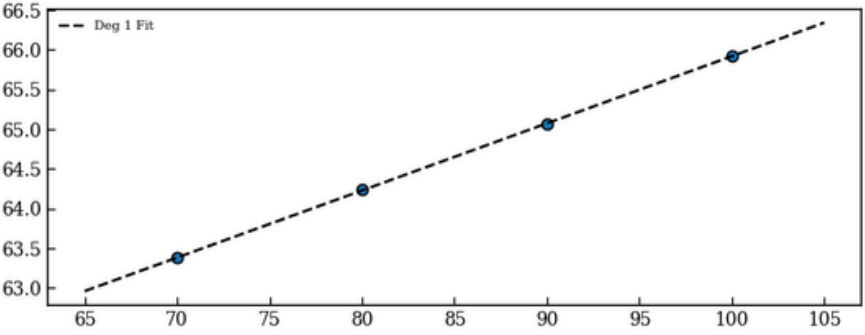
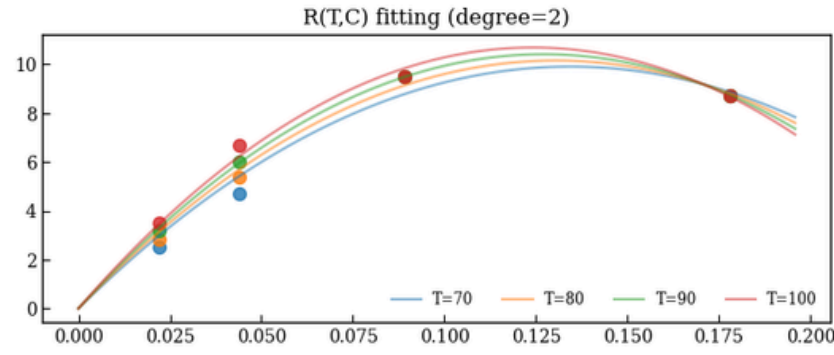
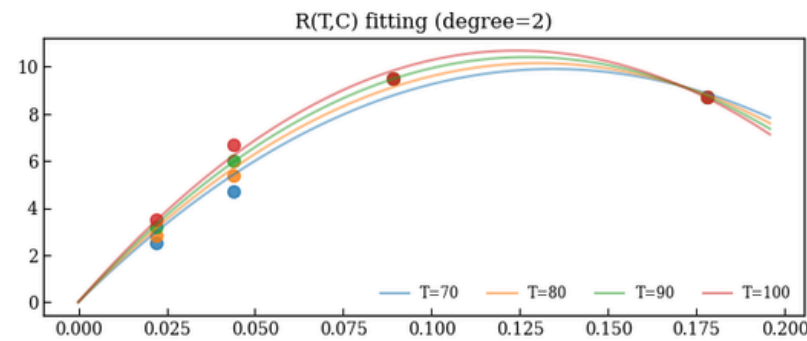
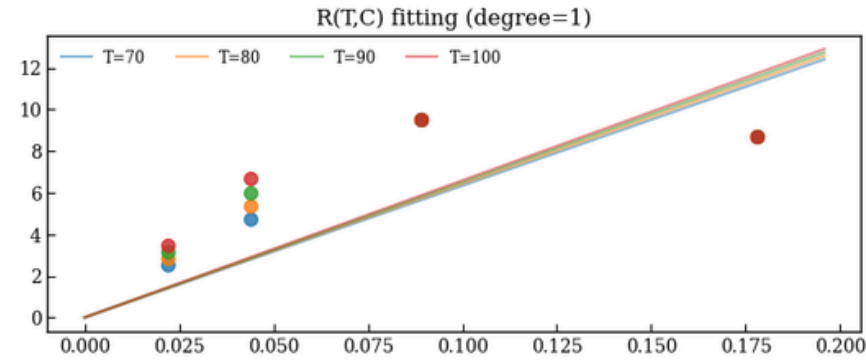
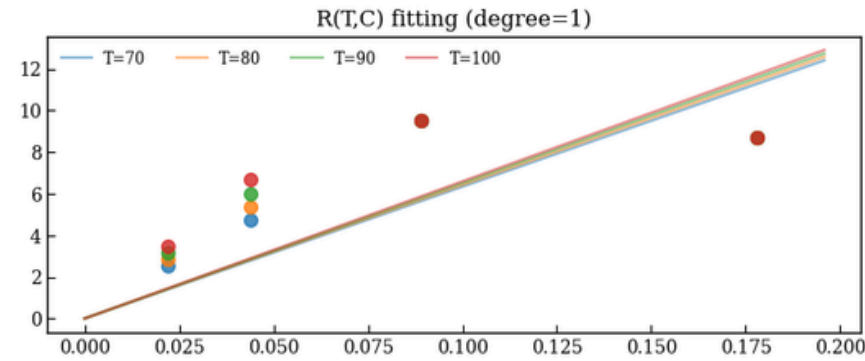


STAGE 1: LOCAL COEFFICIENTS (R vs C)		
T_target	F1	
7	1.9669e+01	
8	2.0567e+01	
9	2.1622e+01	
10	2.2914e+01	
STAGE 2: GLOBAL MODELS (F1 vs T)		
F1(T) [deg 1] = (1.202e+01)*T^0 + (1.079e+00)*T^1		

STAGE 1: LOCAL COEFFICIENTS (R vs C)		
T_target	F1	
7	1.9669e+01	
8	2.0567e+01	
9	2.1622e+01	
10	2.2914e+01	
STAGE 2: GLOBAL MODELS (F1 vs T)		
F1(T) [deg 2] = (1.902e+01)*T^0 + (-5.964e-01)*T^1 + (9.855e-02)*T^2		

STAGE 1: LOCAL COEFFICIENTS (R vs C)		
T_target	F1	F2
7	2.4890e+01	-3.4068e+01
8	2.6749e+01	-4.0336e+01
9	2.9134e+01	-4.9017e+01
10	3.1653e+01	-5.7023e+01
STAGE 2: GLOBAL MODELS (F1 vs T)		
F1(T) [deg 1] = (8.833e+00)*T^0 + (2.267e+00)*T^1		
F2(T) [deg 1] = (2.080e+01)*T^0 + (-7.755e+00)*T^1		

STAGE 1: LOCAL COEFFICIENTS (R vs C)		
T_target	F1	F2
7	2.4890e+01	-3.4068e+01
8	2.6749e+01	-4.0336e+01
9	2.9134e+01	-4.9017e+01
10	3.1653e+01	-5.7023e+01
STAGE 2: GLOBAL MODELS (F1 vs T)		
F1(T) [deg 2] = (2.056e+01)*T^0 + (-5.402e-01)*T^1 + (1.652e-01)*T^2		
F2(T) [deg 2] = (-1.005e+01)*T^0 + (-3.670e-01)*T^1 + (-4.346e-01)*T^2		



STAGE 1: LOCAL COEFFICIENTS (R vs C)		
T_target	F1	
70	6.3378e+01	
80	6.4232e+01	
90	6.5064e+01	
100	6.5918e+01	
STAGE 2: GLOBAL MODELS (F1 vs T)		
F1(T) [deg 1] = (5.746e+01)*T^0 + (8.453e-02)*T^1		

STAGE 1: LOCAL COEFFICIENTS (R vs C)		
T_target	F1	
70	6.3378e+01	
80	6.4232e+01	
90	6.5064e+01	
100	6.5918e+01	
STAGE 2: GLOBAL MODELS (F1 vs T)		
F1(T) [deg 2] = (5.746e+01)*T^0 + (8.458e-02)*T^1 + (-3.380e-07)*T^2		

STAGE 1: LOCAL COEFFICIENTS (R vs C)		
T_target	F1	F2
70	1.4745e+02	-5.4856e+02
80	1.5577e+02	-5.9730e+02
90	1.6392e+02	-6.4505e+02
100	1.7224e+02	-6.9377e+02
STAGE 2: GLOBAL MODELS (F1 vs T)		
F1(T) [deg 1] = (8.969e+01)*T^0 + (8.253e-01)*T^1		
F2(T) [deg 1] = (-2.103e+02)*T^0 + (-4.834e+00)*T^1		

STAGE 1: LOCAL COEFFICIENTS (R vs C)		
T_target	F1	F2
70	1.4745e+02	-5.4856e+02
80	1.5577e+02	-5.9730e+02
90	1.6392e+02	-6.4505e+02
100	1.7224e+02	-6.9377e+02
STAGE 2: GLOBAL MODELS (F1 vs T)		
F1(T) [deg 2] = (8.965e+01)*T^0 + (8.265e-01)*T^1 + (-6.621e-06)*T^2		
F2(T) [deg 1] = (-2.103e+02)*T^0 + (-4.834e+00)*T^1		